

Lecture 3

Local Optimizations, Intro to SSA

- I. Basic blocks & Flow graphs
- II. Abstraction 1: DAG
- III. Abstraction 2: Value numbering
- IV. Intro to SSA

ALSU 8.4-8.5, 6.2.4

I. Basic Blocks & Flow Graphs

Basic block = a sequence of 3-address statements

- only the first statement can be reached from outside the block (no branches into middle of block)
- all the statements are executed consecutively if the first one is (no branches out or halts except perhaps at end of block)
- **We require basic blocks to be *maximal***, i.e., they cannot be made larger without violating the conditions

Flow graph

- **Nodes:** basic blocks
- **Edges:** $B_i \rightarrow B_j$, iff B_j can follow B_i immediately in *some* execution
 - Either first instruction of B_j is target of a goto at end of B_i
 - Or, B_j physically follows B_i , which does not end in an unconditional goto.

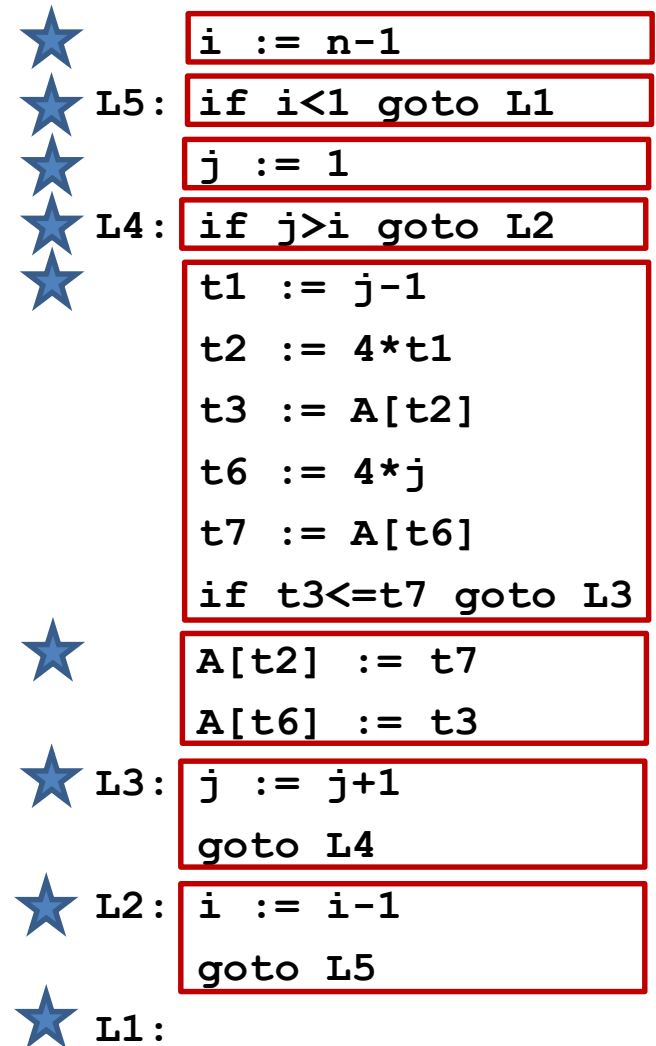
Partitioning into Basic Blocks

Identify the leader of each basic block

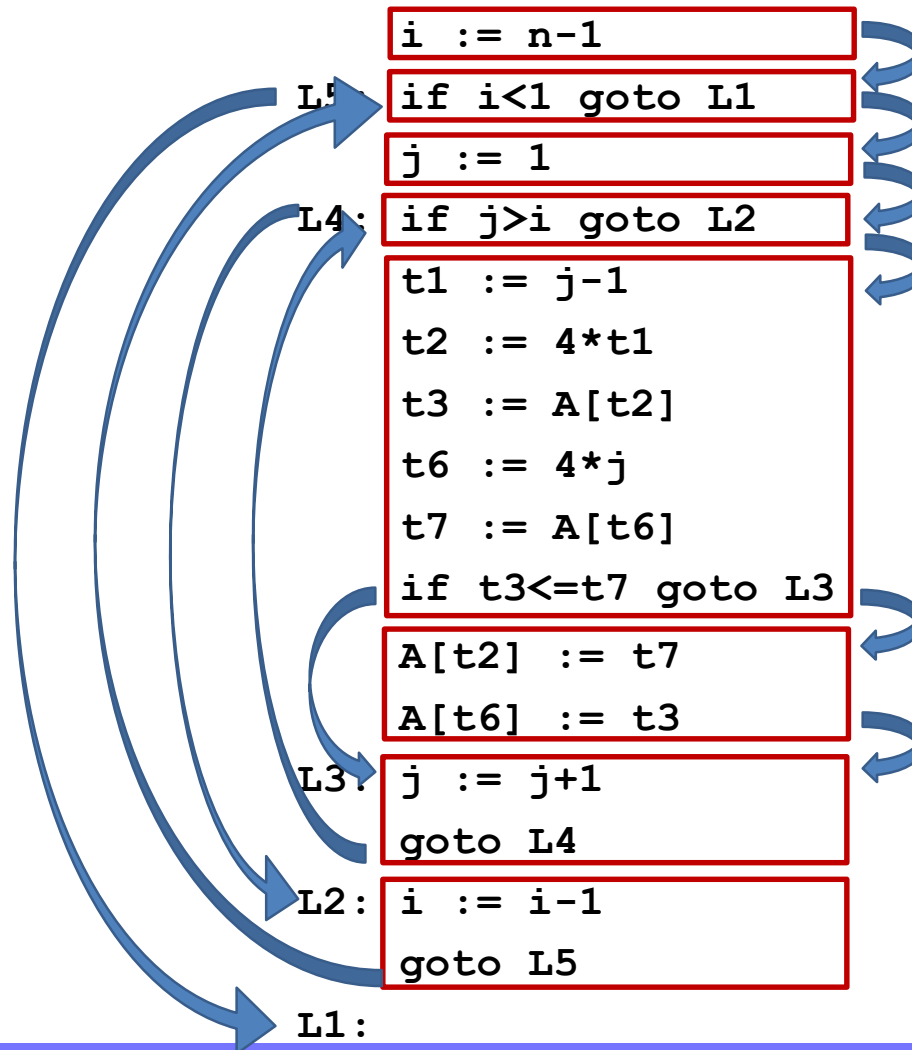
- First instruction
- Any target of a jump
- Any instruction immediately following a jump

Basic block starts at leader & ends at instruction immediately before a leader (or the last instruction)

ALSU 8.4



Flow Graph



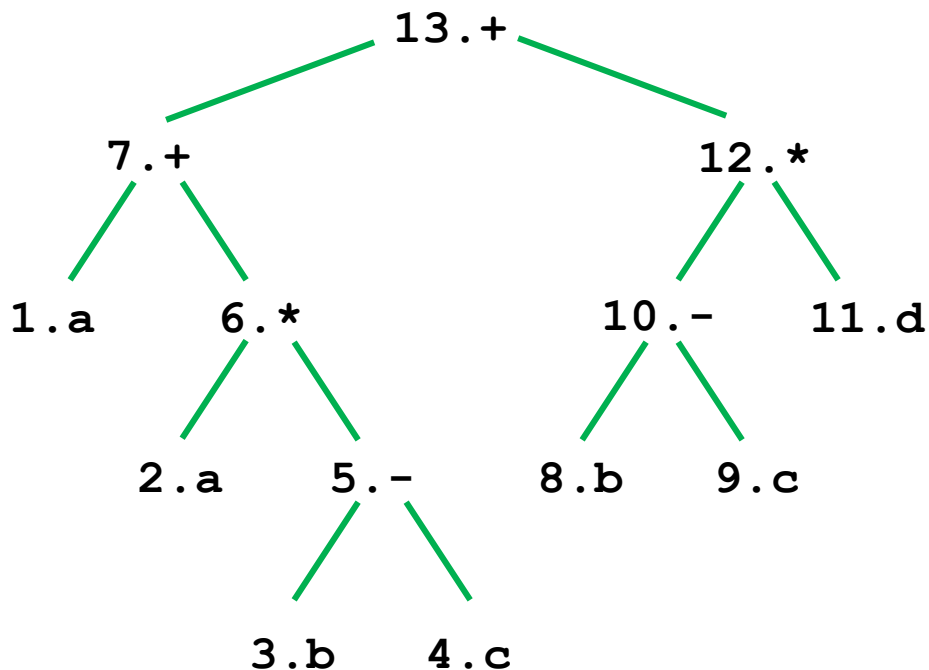
II. Local Optimizations (within basic block)

- **Common subexpression elimination**
 - array expressions
 - field access in records
 - access to parameters

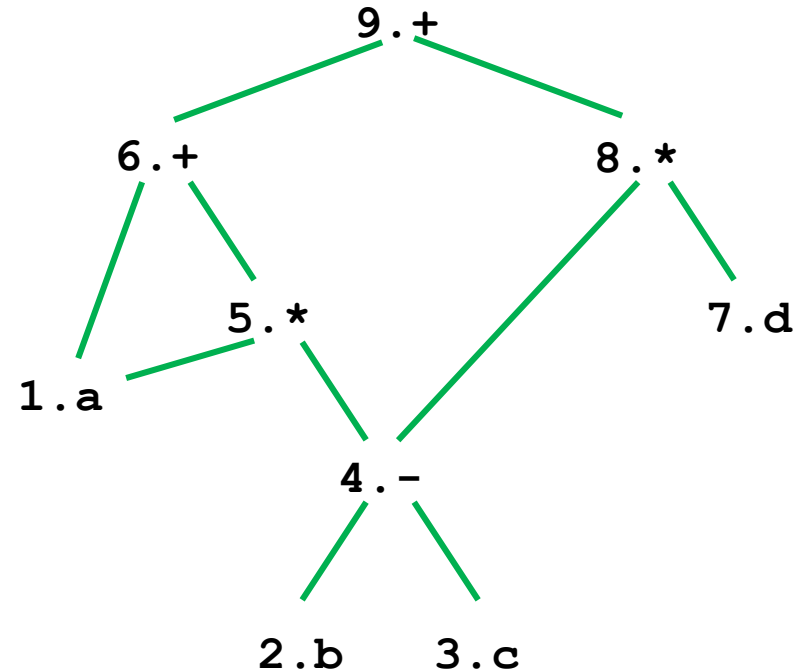
Graph Abstractions

Example 1:

- grammar (for bottom-up parsing): $E \rightarrow E + T \mid E - T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid id$
- expression: $a + a * (b - c) + (b - c) * d$



Parse tree



Expression DAG

ALSU 4.5

ALSU 6.1.1

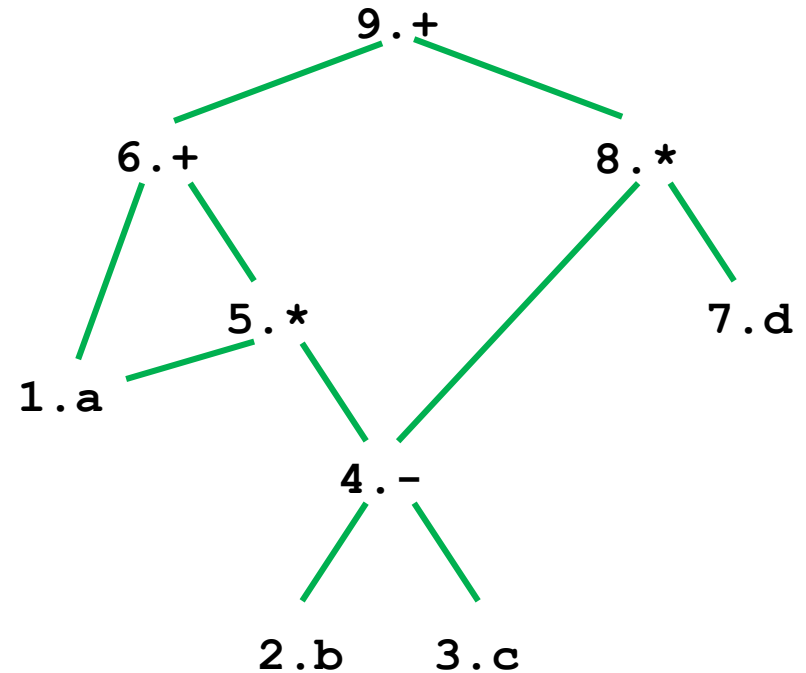
Carnegie Mellon

Graph Abstractions

Expression: $a + a * (b - c) + (b - c) * d$

Optimized code:

```
t1 = b - c
t2 = a * t1
t3 = a + t2
t4 = t1 * d
t5 = t3 + t4
```



How well do DAGs hold up across statements?

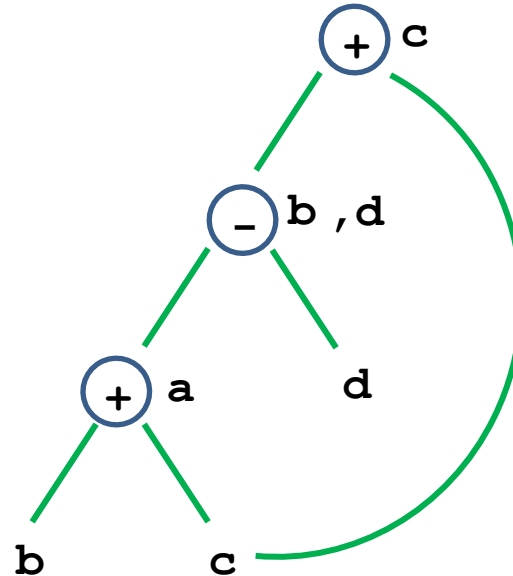
Example 2:

`a = b+c;`

`b = a-d;`

`c = b+c;`

`d = a-d;`



Is this optimized code correct?

`a = b+c;`

`d = a-d;`

`c = d+c;`

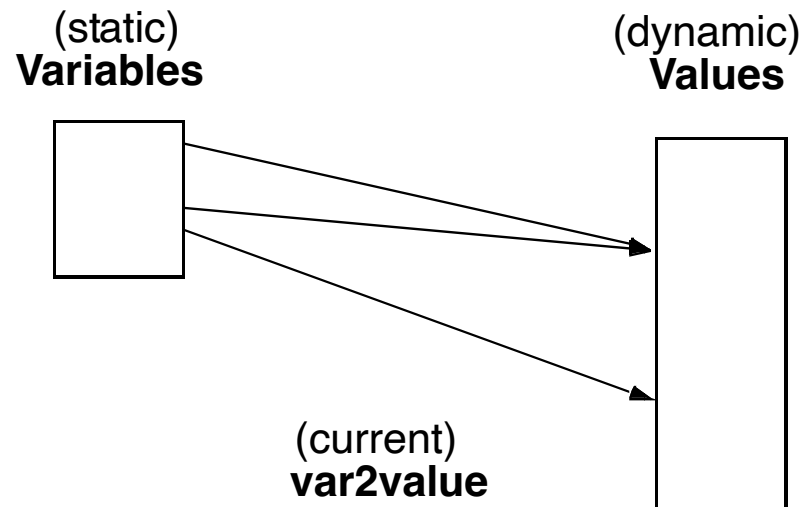
Depends on whether `b` is
live on exit from the block

Critique of DAGs

- **Cause of problems**
 - Assignment statements
 - Value of variable depends on TIME
- **How to fix problem?**
 - build graph in order of execution
 - attach variable name to latest value
- **Final graph created is not very interesting**
 - Key: variable->value mapping across time
 - loses appeal of abstraction

III. Value Number: Another Abstraction

- John Cocke & Jack Schwartz in unpublished book: “Programming Languages and their Compilers”, (1970)
- More explicit with respect to VALUES, and TIME



- each value has its own “number”
 - common subexpression means same value number
- var2value: current map of variable to value
 - used to determine the value number of current expression

$r1 + r2 \Rightarrow \text{var2value}(r1) + \text{var2value}(r2)$

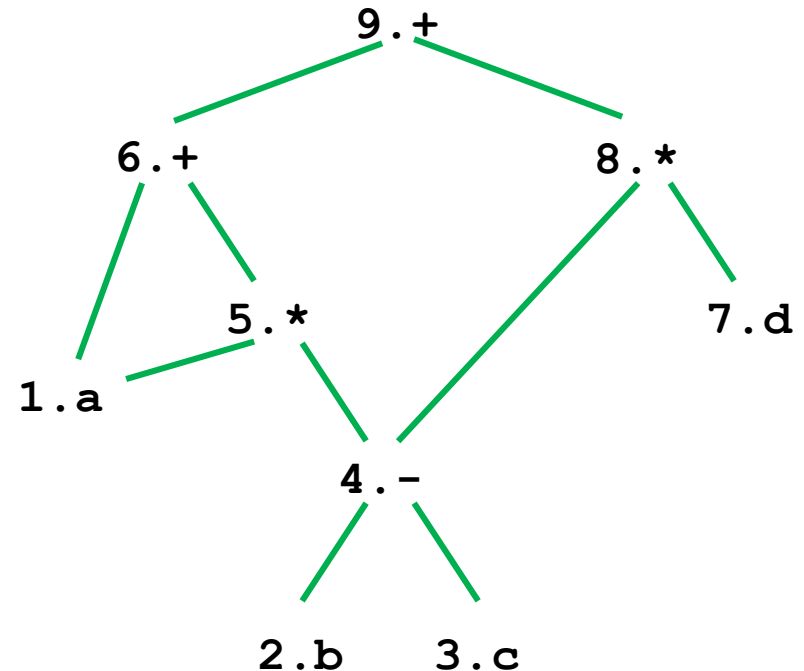
ALSU 6.1.2

Value Numbering: Expression Example

Expression: $a + a * (b - c) + (b - c) * d$

Optimized code:

```
t4 = b - c
t5 = a * t4
t6 = a + t5
t8 = t4 * d
t9 = t6 + t8
```



Value Numbering Algorithm

Data structure:

```
VALUES = Table of
    expression    /* [OP, valnum1, valnum2] */
    var           /* name of variable currently holding expr */
```

For each instruction (dst = src1 OP src2) in execution order

```
valnum1=var2value(src1); valnum2=var2value(src2)
```

```
IF [OP, valnum1, valnum2] is in VALUES
```

```
    v = the index of expression
```

```
    Replace instruction with: dst = VALUES[v].var
```

```
ELSE
```

```
    Add
```

```
        expression = [OP, valnum1, valnum2]
```

```
        var         = tv
```

```
    to VALUES
```

```
    v = index of new entry; tv is new temporary for v
```

```
    Replace instruction with: tv = VALUES[valnum1].var OP VALUES[valnum2].var
                               dst = tv
```

```
set_var2value (dst, v)
```

More Details

- **What are the initial values of the variables?**
 - values at beginning of the basic block
- **Possible implementations:**
 - Initialization: create “initial values” for all variables
 - Or dynamically create them as they are used
- **Implementation of VALUES and var2value: hash tables**

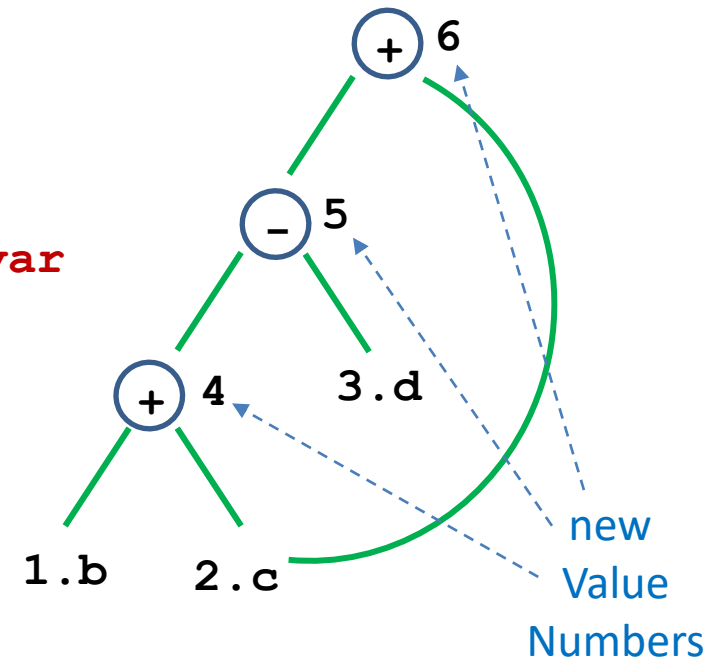
Value Numbering: Basic Block Example

```
a = b+c          t4 = b + c  // tv = VALUES[valnum1].var
                                OP VALUES[valnum2].var
                                a = t4          // dst = tv
b = a-d          t5 = t4 - d
b = t5
c = b+c          t6 = t5 + c
c = t6
d = a-d          d = t5      // dst =
                                VALUES[v].var
```

VALUES[5] = ([-,4,3], t5)

Q: Assigning to a temporary and then copying to the destination increases the number of instructions—so why do it?

A: If dst is overwritten later, would lose opportunity to eliminate common subexpression since no variable would hold the result



Question

- How do you extend value numbering to constant folding?

a = 1

b = 2

c = a+b

Answer: Can add a field to the VALUES table indicating when an expression is a constant and what its value is

DAGs vs. Value Numbering

- **Comparisons of two abstractions**
 - DAGs
 - Value numbering
- **Value numbering**
 - VALUE: distinguish between variables and VALUES
 - TIME
 - Interpretation of instructions in order of execution
 - Keep dynamic state information

IV. Intro to SSA

Global Optimizations: look beyond the basic block

- **Global versions of local optimizations**

- global common subexpression elimination
- global constant propagation
- dead code elimination

We will cover these
optimizations
in later lectures

- **Loop optimizations**

- reduce code to be executed in each iteration
- code motion
- induction variable elimination

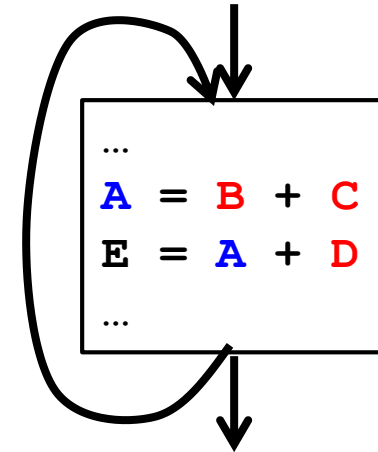
- **Other control structures**

- Code hoisting: eliminates copies of identical code on parallel paths in a flow graph to reduce code size.

Recurring Optimization Theme: Where Is a Variable Defined or Used?

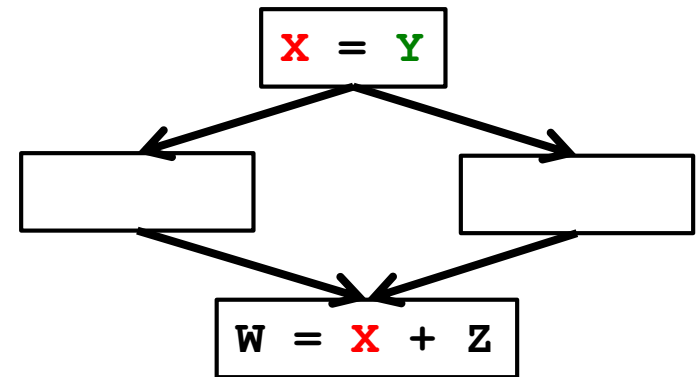
- Example: Loop-Invariant Code Motion

- Are **B**, **C**, and **D** only defined outside the loop?
- Other definitions of **A** inside the loop?
- Uses of **A** inside the loop?



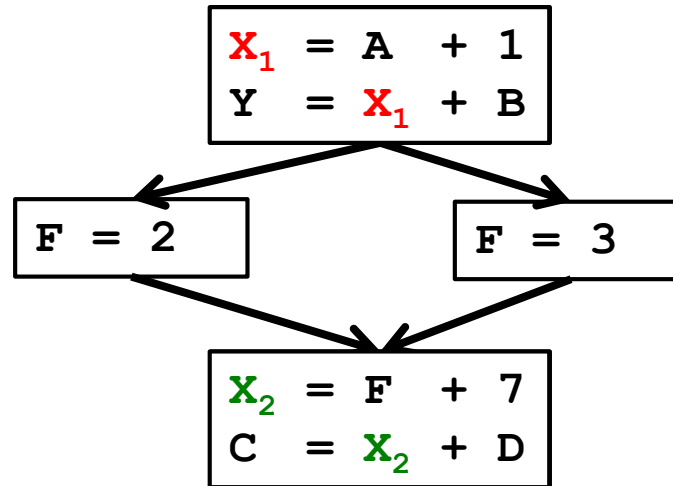
- Example: Copy Propagation

- For a given use of **X**:
 - Are all reaching definitions of **X**:
 - copies from same variable: e.g., **X** = **Y**
 - Where **Y** is not redefined since that copy?
- If so, substitute use of **X** with use of **Y**



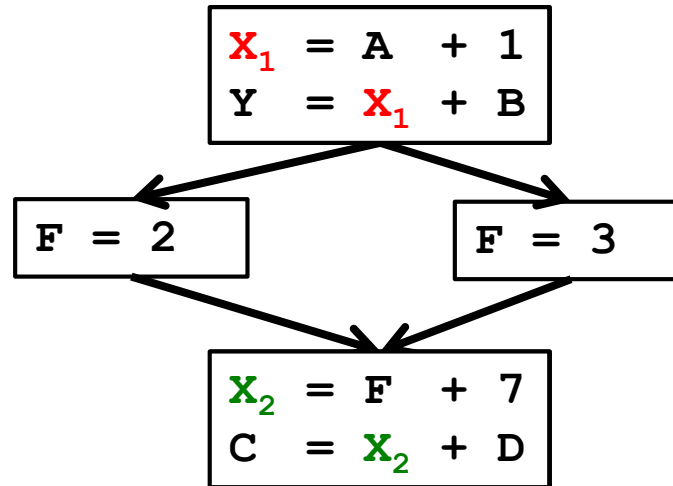
- It would be nice if we could *traverse directly* between related uses and def's
 - this would enable a form of *sparse* code analysis (skip over “don’t care” cases)

Appearances of Same Variable Name May Be Unrelated



- The values in reused storage locations may be provably independent
 - in which case the compiler can optimize them as separate values
- Compiler could use renaming to make these different versions more explicit

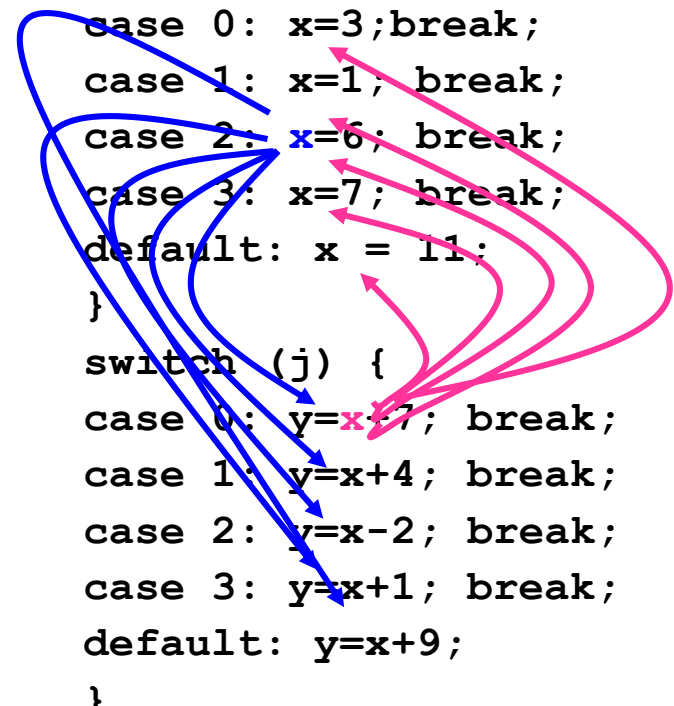
Definition-Use and Use-Definition Chains



- Definition-Use (DU) Chains:
 - for a given definition of a variable X , what are all of its uses?
- Use-Definition (UD) Chains:
 - for a given use of a variable X , what are all of the reaching definitions of X ?

Unfortunately DU and UD Chains Can Be Expensive

```
foo(int i, int j) {  
  ...  
  switch (i) {  
    case 0: x=3; break;  
    case 1: x=1; break;  
    case 2: x=6; break;  
    case 3: x=7; break;  
    default: x = 11;  
  }  
  switch (j) {  
    case 0: y=x*7; break;  
    case 1: y=x+4; break;  
    case 2: y=x-2; break;  
    case 3: y=x+1; break;  
    default: y=x+9;  
  }  
  ...  
}
```



In general,

N defs

M uses

⇒ $O(NM)$ space and time

One solution: limit each variable to ONE definition site

Unfortunately DU and UD Chains Can Be Expensive

```
foo(int i, int j) {
```

```
...
```

```
  switch (i) {
```

```
    case 0: x=3; break;
```

```
    case 1: x=1; break;
```

```
    case 2: x=6;
```

```
    case 3: x=7;
```

```
    default: x = 11;
```

```
  }
```

x1 is one of the above x's

```
  switch (j) {
```

```
    case 0: y=x1+7;
```

```
    case 1: y=x1+4;
```

```
    case 2: y=x1-2;
```

```
    case 3: y=x1+1;
```

```
    default: y=x1+9;
```

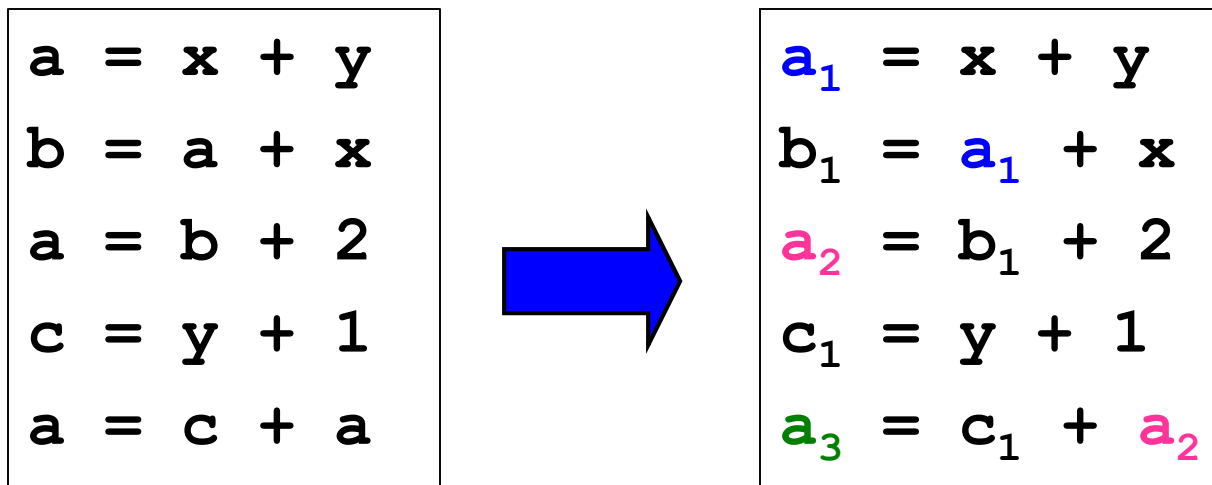
```
  }
```

```
...
```

One solution: limit each variable to ONE definition site

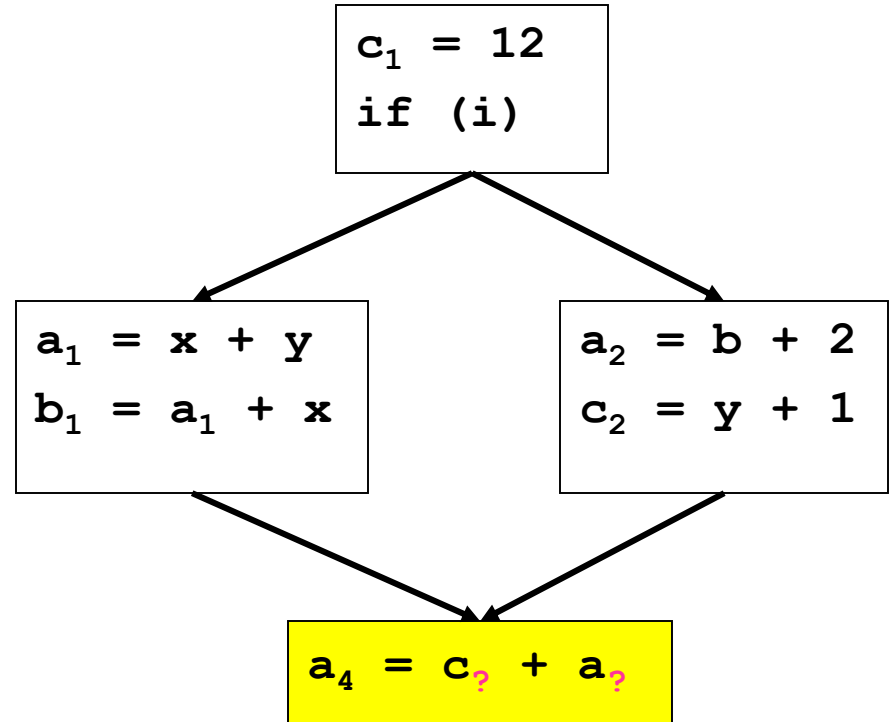
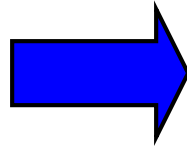
Static Single Assignment (SSA)

- Static single assignment is an IR where every variable is assigned a value at most once in the program text
- Easy for a basic block (reminiscent of Value Numbering):
 - Visit each instruction in program order:
 - LHS: assign to a *fresh version* of the variable
 - RHS: use the *most recent version* of each variable



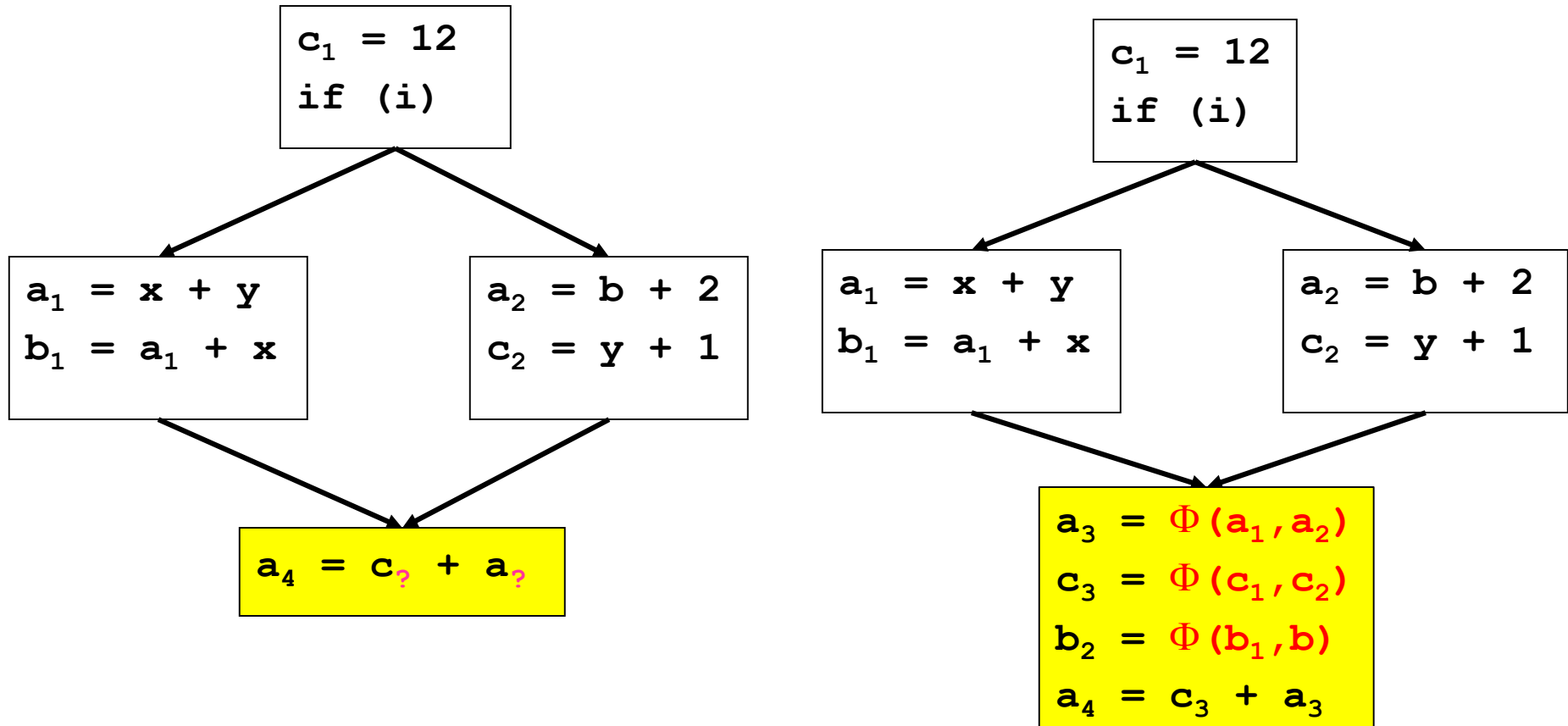
What about Joins in the CFG?

```
c = 12
if (i) {
    a = x + y
    b = a + x
} else {
    a = b + 2
    c = y + 1
}
a = c + a
```



→ Use a notational convention (fiction): a Φ function

Merging at Joins: the Φ function



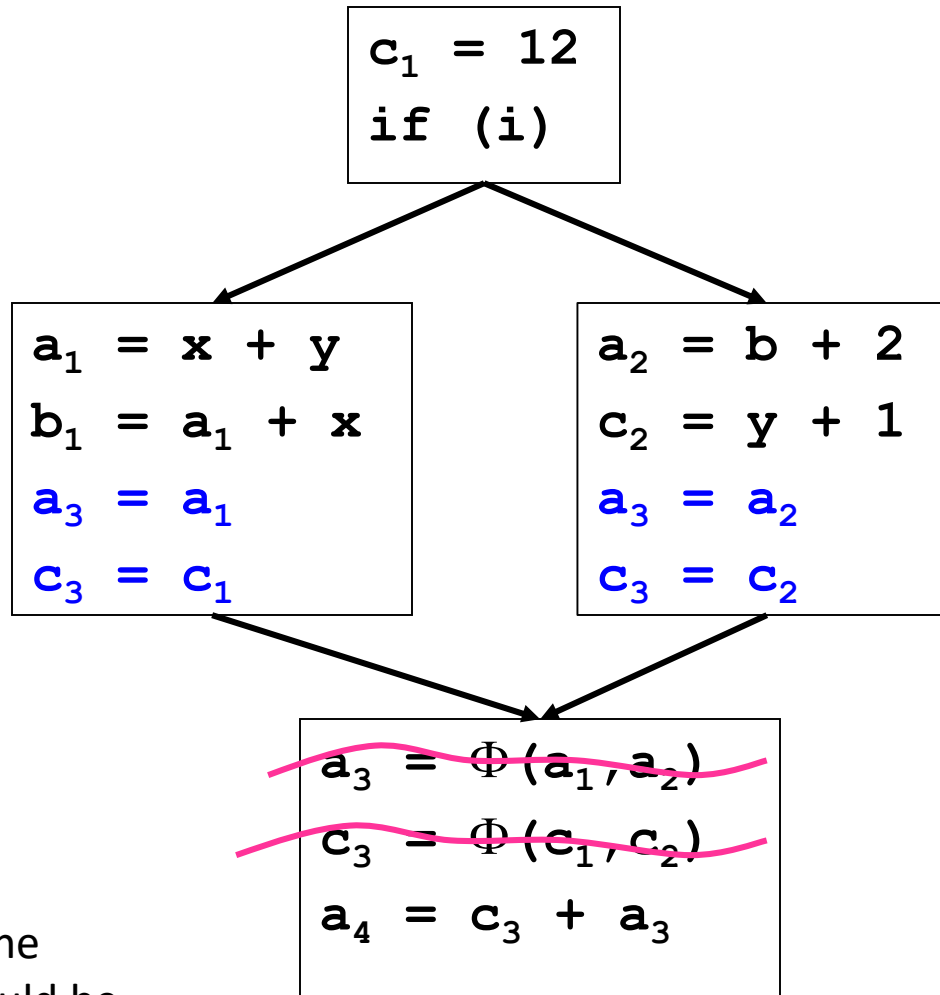
The Φ function

- Φ merges multiple definitions along multiple control paths into a single definition.
- At a basic block with p predecessors, there are p arguments to the Φ function.

$$x_{\text{new}} = \Phi(x_1, x_2, x_3, \dots, x_p)$$

- How do we choose which x_i to use?
 - We don't really care!
- How do we emit code for this?

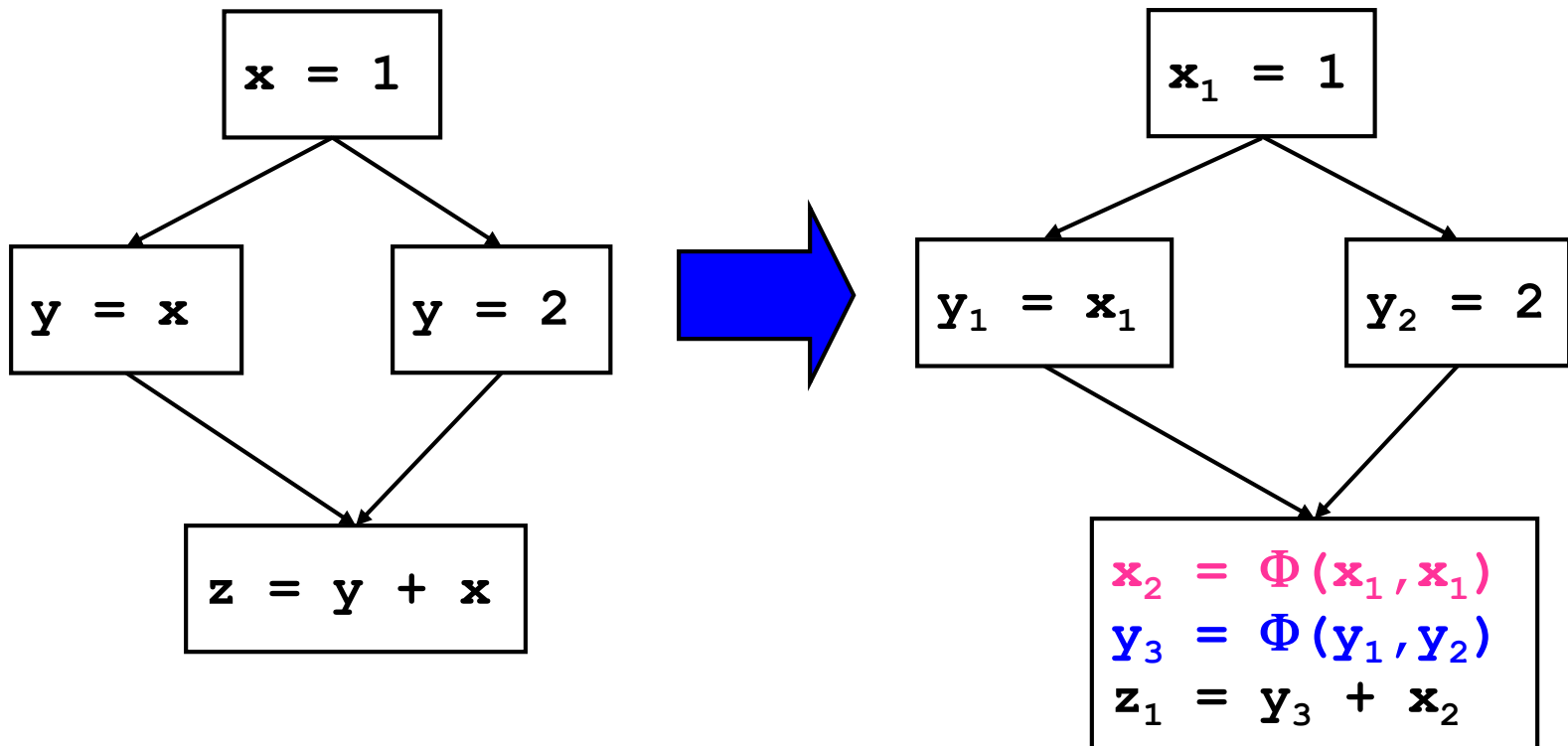
“Implementing” Φ



Never really done
this way, but could be

Trivial SSA

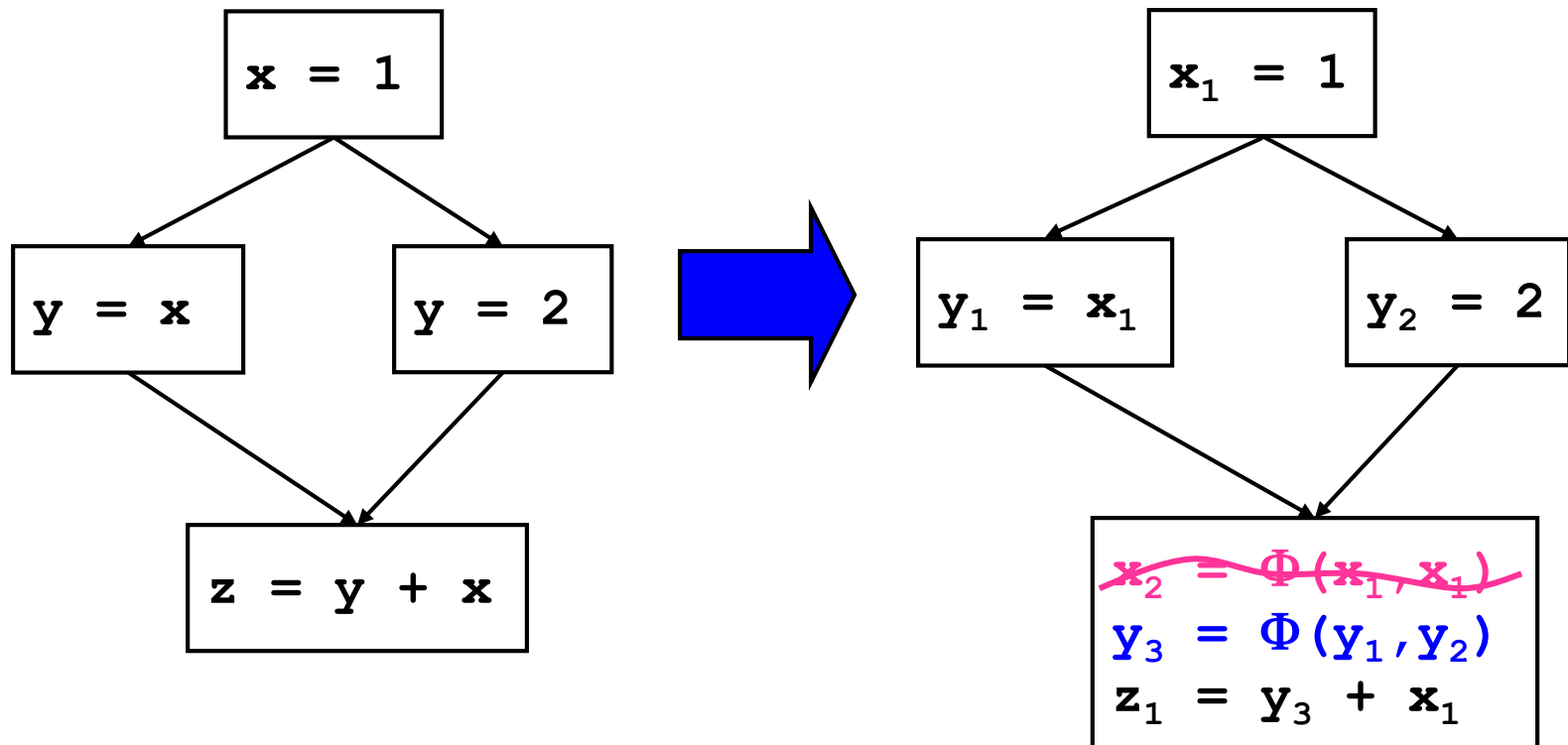
- Each assignment generates a fresh variable
- At each join point insert Φ functions for **all live variables**



In general, too many Φ functions inserted

Minimal SSA

- Each assignment generates a fresh variable
- At each join point insert Φ functions for **all live variables** with **multiple outstanding defs**



Lecture 5

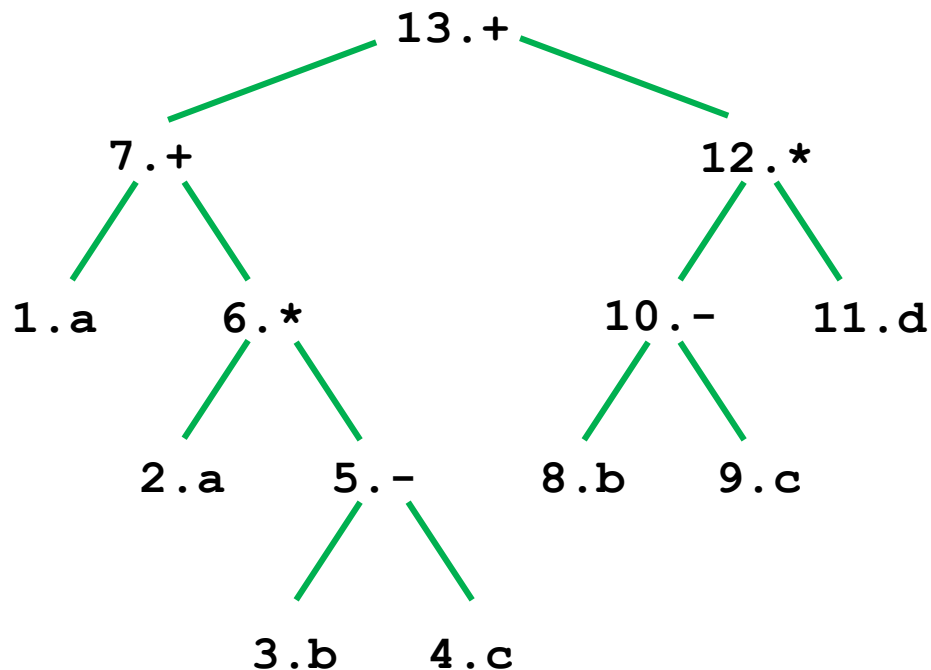
Introduction to Data Flow Analysis

- I. Structure of data flow analysis
- II. Example 1: Reaching definition analysis
- III. Example 2: Liveness analysis
- IV. Framework

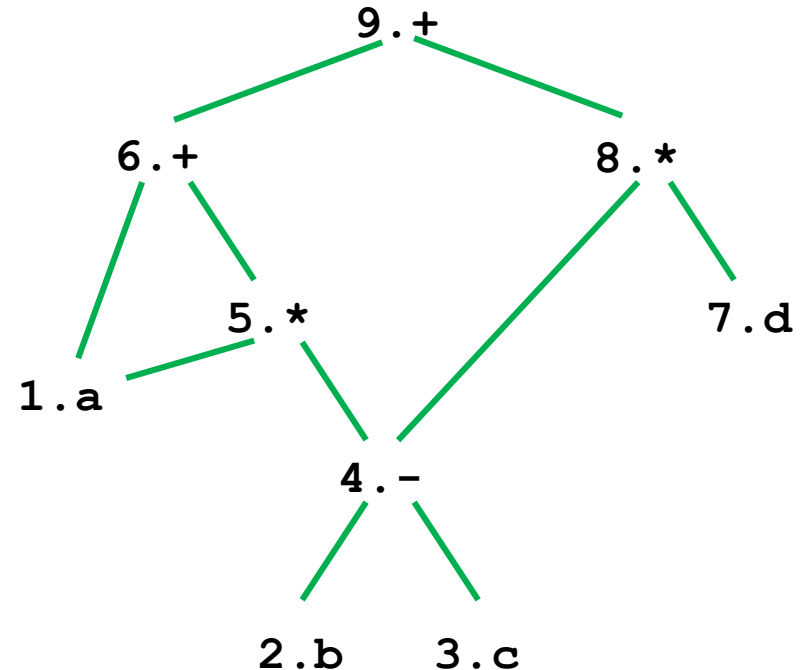
Review: Expression DAG

Example 1:

- grammar (for bottom-up parsing): $E \rightarrow E + T \mid E - T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid id$
- expression: $a + a * (b - c) + (b - c) * d$



Parse tree



Expression DAG

Review: Value Numbering

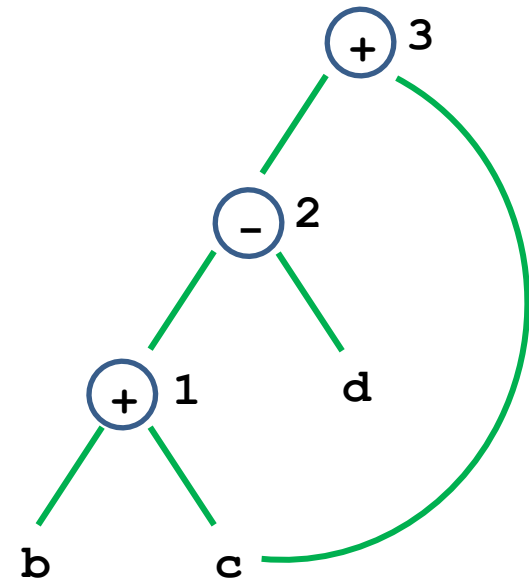
Data structure:

VALUES = Table of

expression	/* [OP, valnum1, valnum2] */
var	/* name of variable currently holding expr */

var2value() /* variable's current value number */

a = b+c	t1 = b + c
	a = t1
b = a-d	t2 = t1 - d
	b = t2
c = b+c	t3 = t2 + c
	c = t3
d = a-d	d = t2



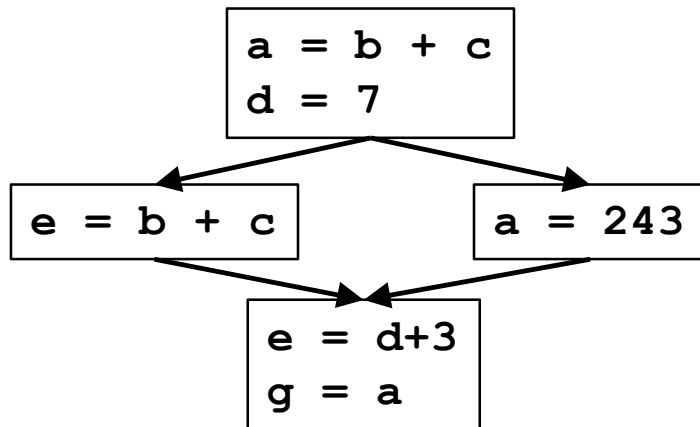
What is Data Flow Analysis?

- **Local analysis (e.g. value numbering)**
 - analyze effect of each instruction
 - compose effects of instructions to derive information from beginning of basic block to each instruction
- **Data flow analysis**
 - analyze effect of each basic block
 - compose effects of basic blocks to derive information at basic block boundaries
 - from basic block boundaries, apply local technique to generate information on instructions

[ALSU 9.2]

What is Data Flow Analysis? (Cont.)

- **Data flow analysis:**
 - Flow-sensitive: sensitive to the control flow in a function
 - Intraprocedural analysis
- **Examples of optimizations:**
 - Constant propagation
 - Common subexpression elimination
 - Dead code elimination



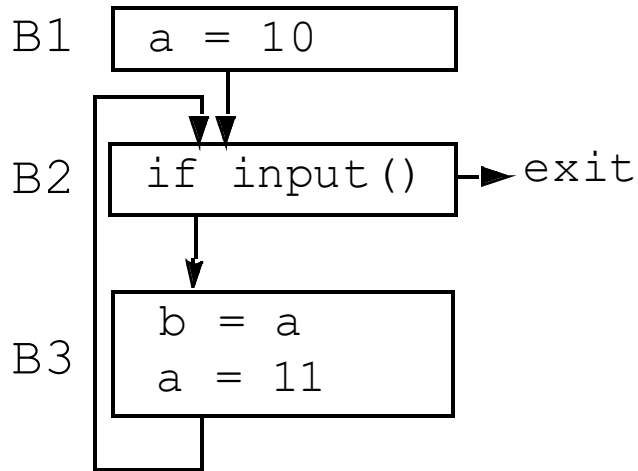
For each variable x , determine:

Value of x ?

Which “definition” defines x ?

Is the definition still meaningful (live)?

Static Program vs. Dynamic Execution



- **Statically:** Finite program
- **Dynamically:** Can have infinitely many possible execution paths
- **Data flow analysis abstraction:**
 - For each point in the program:
combines information of all the instances of the same program point.
- **Example of a data flow question:**
 - Which definition defines the value used in statement “`b = a`”?

Effects of a Basic Block

- Effect of a statement: **a = b+c**
 - **Use** source variables (b and c)
 - **Kills** an old definition (old definition of a)
 - Defines a new **definition** (a)
- Compose effects of statements -> Effect of a basic block
 - A **locally exposed use** in a b.b. is a use of a data item that is not preceded in the b.b. by a definition of the data item.
 - Any definition of a data item in the b.b. **kills** all other definitions of the same data item.
 - A **locally available definition** = last definition of data item in b.b.

```
t1 = r1+r2
r2 = t1
t2 = r2+r1
r1 = t2
t3 = r1*r1
r2 = t3
if r2>100 goto L1
```



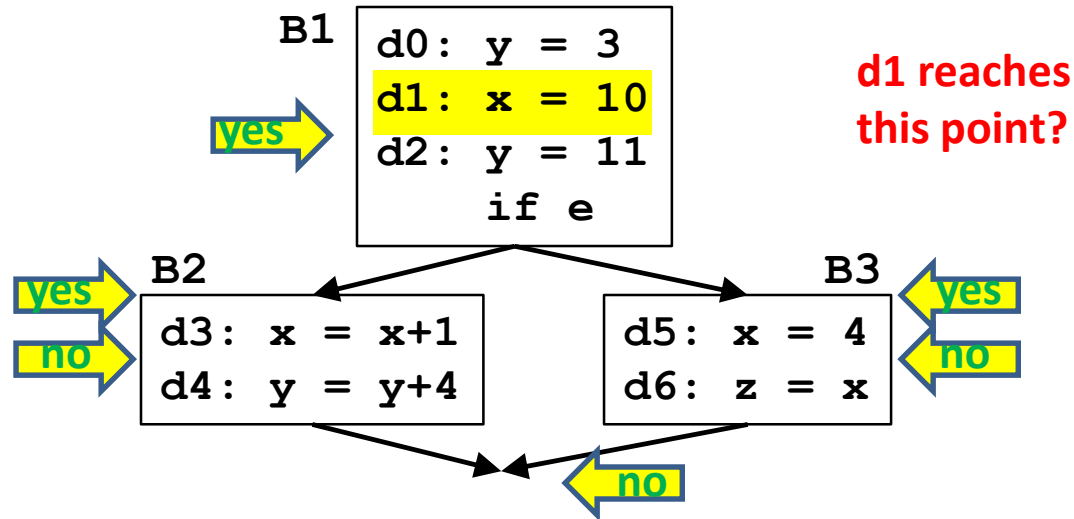
locally exposed uses? r1

kills any definitions? Any other
defn of t2
in program

locally available definition? t2

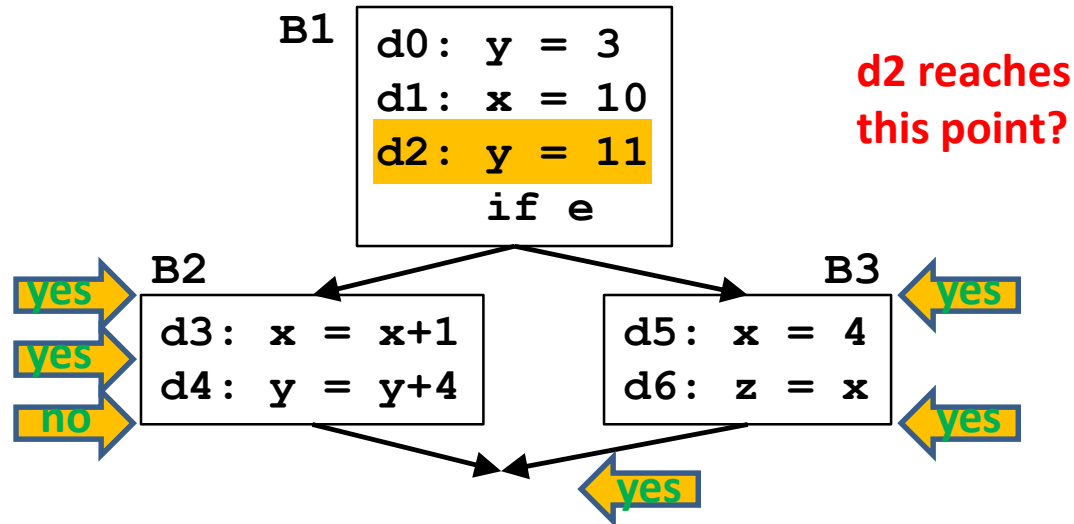
II. Reaching Definitions

ALSU 9.2.4



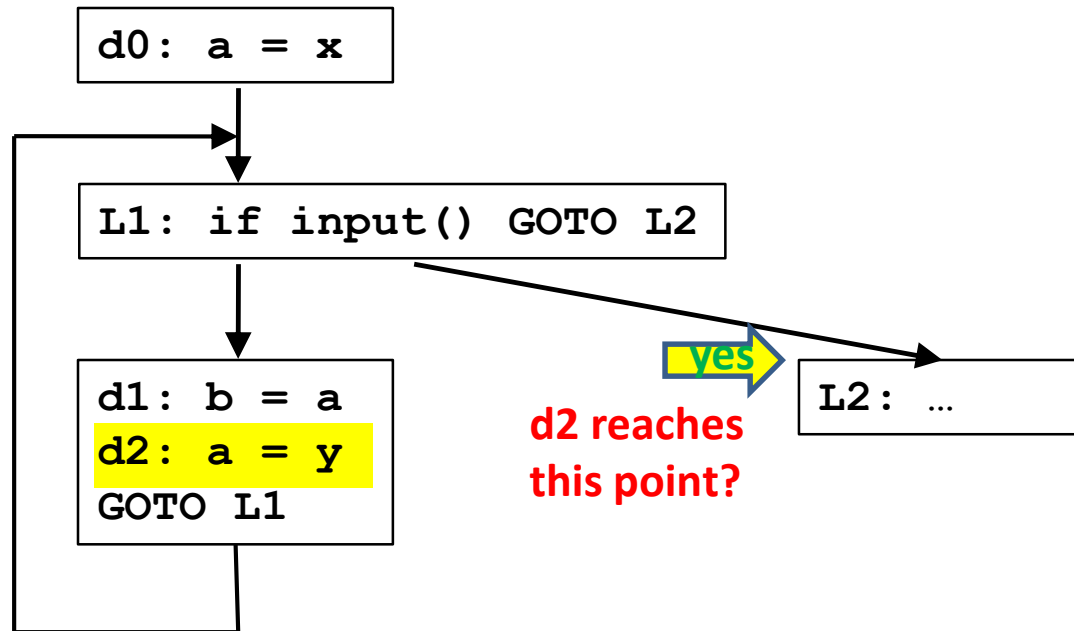
- Every assignment is a **definition**
- A **definition** d **reaches** a point p if **there exists** path from the point immediately following d to p such that d is **not killed** (overwritten) along that path.
- Problem statement
 - For each point in the program, determine if each definition in the program reaches the point
 - A bit vector per program point, vector-length = #defs

Reaching Definitions

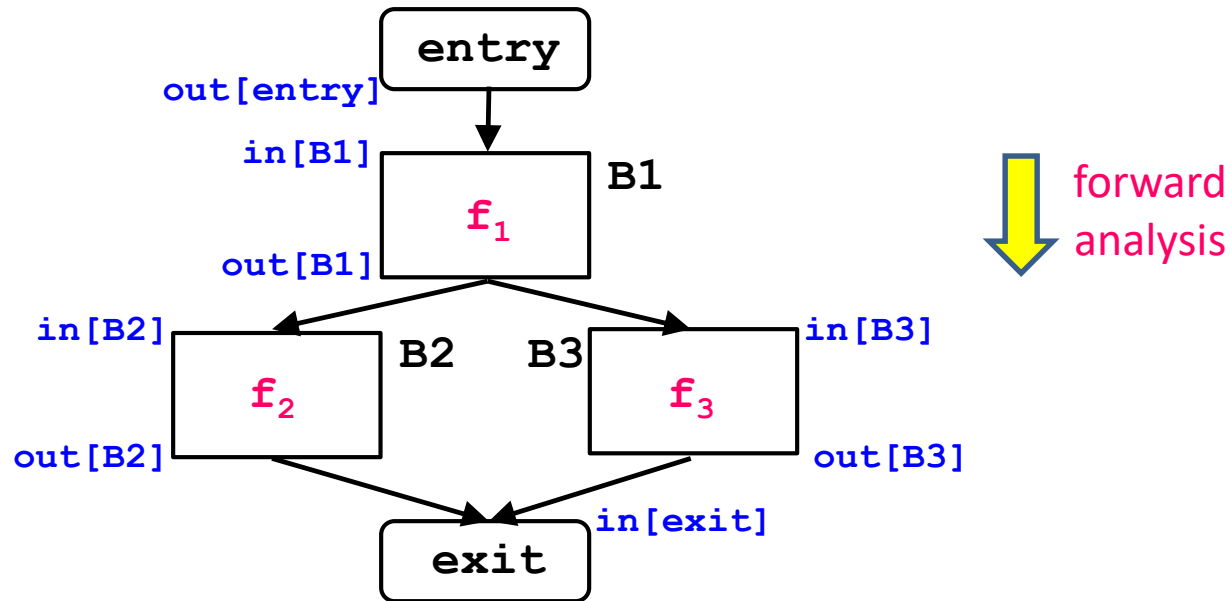


- Every assignment is a **definition**
- A **definition** d **reaches** a point p if **there exists** path from the point immediately following d to p such that d is **not killed** (overwritten) along that path.
- Problem statement
 - For each point in the program, determine if each definition in the program reaches the point
 - A bit vector per program point, vector-length = #defs

Reaching Definitions: Another Example

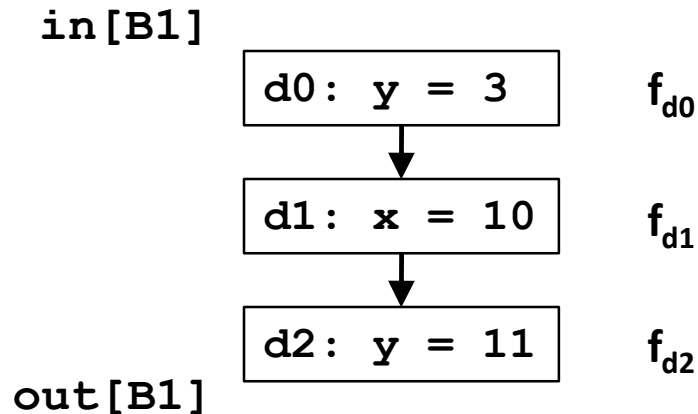


Data Flow Analysis Schema



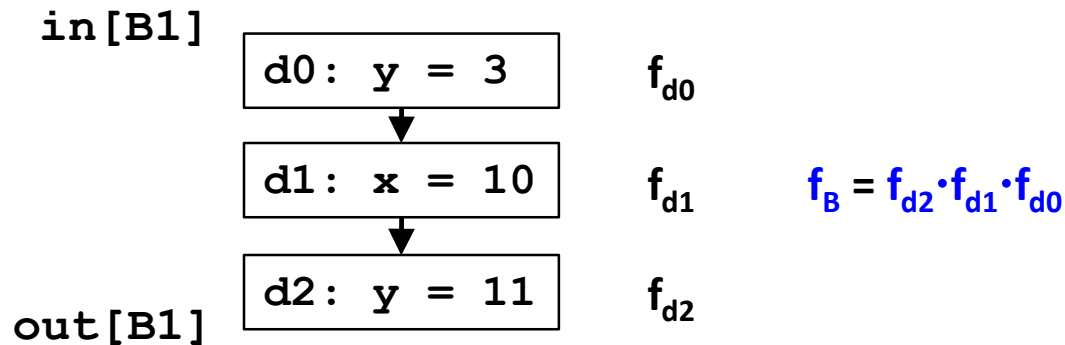
- Build a **flow graph** (nodes = basic blocks, edges = control flow)
- Set up a set of equations between $in[b]$ and $out[b]$ for all basic blocks b
 - Effect of **code in basic block**:
 - **Transfer function** f_b relates $in[b]$ and $out[b]$, for same b
 - Effect of **flow of control**:
 - relates $out[b]$, $in[b']$ if b and b' are **adjacent**
- Find a solution to the equations

Effects of a Statement



- f_s : A transfer function of a statement
 - abstracts the execution with respect to the problem of interest
- Consider Reaching Definitions. For a statement s (e.g., $d: x = y + z$):
 $out[s] = f_s(in[s]) = Gen[s] \cup (in[s] - Kill[s])$
 - **Gen[s]**: definitions generated: $Gen[s] = \{d\}$
 - **Propagated** definitions: $in[s] - Kill[s]$,
where **Kill[s]** = set of all other defs to x in the rest of program

Effects of a Basic Block



- Transfer function of a statement *s*:
 - $\text{out}[s] = f_s(\text{in}[s]) = \text{Gen}[s] \cup (\text{in}[s] - \text{Kill}[s])$
- Transfer function of a **basic block B**:
 - Composition of transfer functions of statements in B
- **out[B] = f_B(in[B]) = f_{d2}f_{d1}f_{d0}(in[B])**

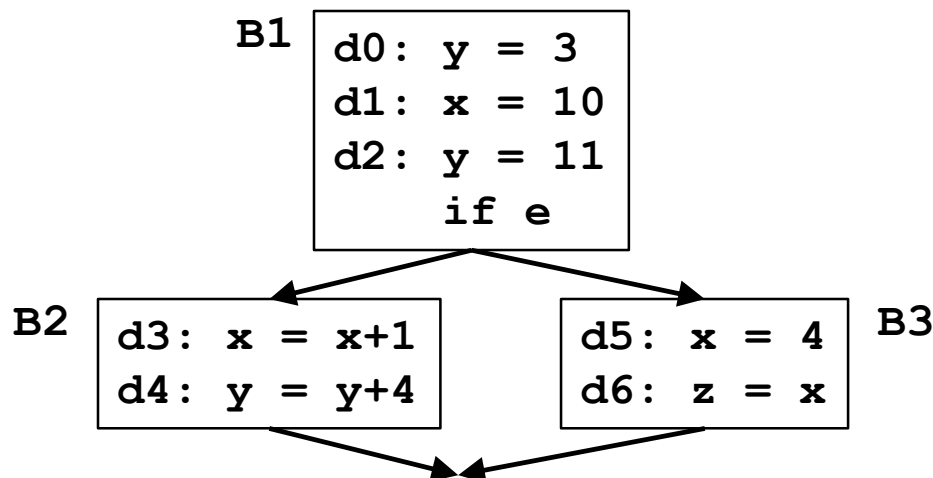
$$= \text{Gen}[d_2] \cup ((\text{Gen}[d_1] \cup ((\text{Gen}[d_0] \cup (\text{in}[B] - \text{Kill}[d_0])) - \text{Kill}[d_1])) - \text{Kill}[d_2])$$

$$= \text{Gen}[d_2] \cup (\text{Gen}[d_1] \cup (\text{Gen}[d_0] - \text{Kill}[d_1]) - \text{Kill}[d_2]) \cup$$

$$(\text{in}[B] - (\text{Kill}[d_0] \cup \text{Kill}[d_1] \cup \text{Kill}[d_2]))$$

$$= \text{Gen}[B] \cup (\text{in}[B] - \text{Kill}[B])$$
 - **Gen[B]**: locally available definitions (defined locally & reaches end of bb)
 - **Kill[B]**: set of definitions killed by B

Reaching Definitions Example

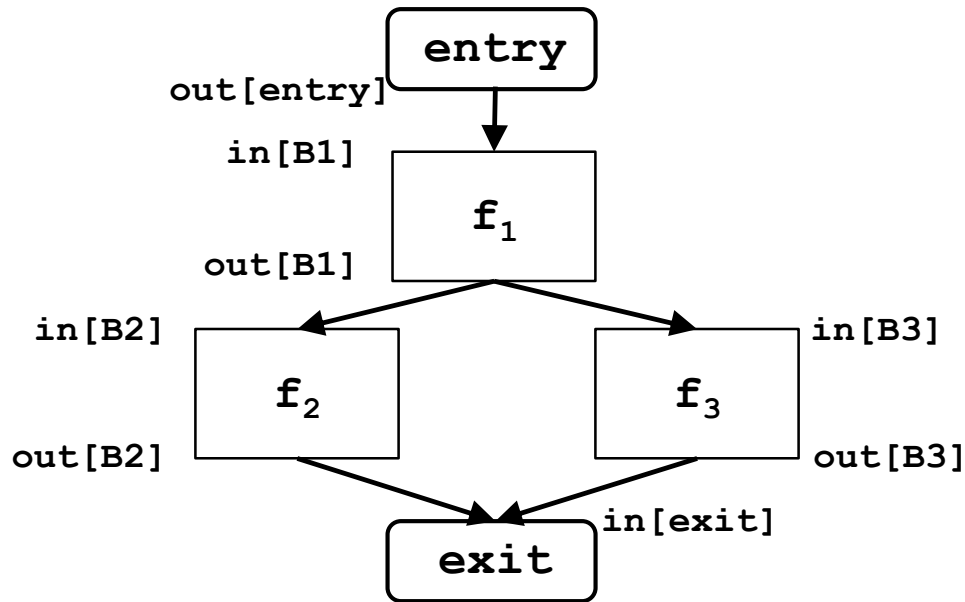


f	Gen	Kill
1	{1,2}	{0,2,3,4,5}
2	{3,4}	{0,1,2,5}
3	{5,6}	{1,3}

Subtlety: d0 & d2
kill each other, but
d2 is still generated

- **transfer function** f_b of a basic block b : $OUT[b] = f_b(IN[b])$
incoming reaching definitions $\rightarrow$ outgoing reaching definitions
- A basic block b
 - **generates** definitions: $Gen[b]$,
– set of definitions in b that reach end of b
 - **kills** definitions: $in[b] - Kill[b]$,
where $Kill[b]$ = set of defs killed by defs in b
- **$out[b] = Gen[b] \cup (in[b] - Kill[b])$**

Effects of the Edges (acyclic)

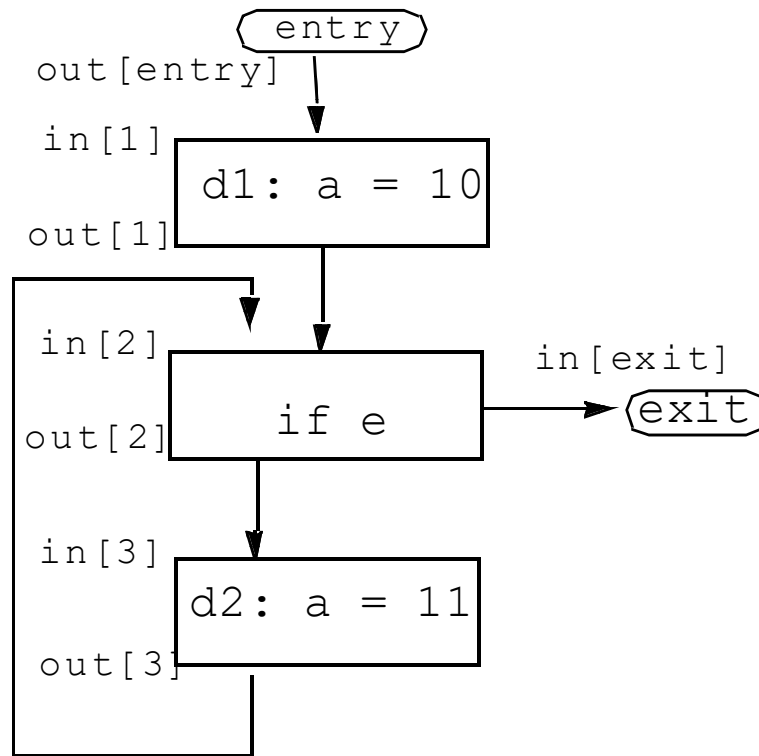


- $out[b] = f_b(in[b])$
- Join node: a node with multiple predecessors
- **meet** operator:

$in[b] = out[p_1] \cup out[p_2] \cup \dots \cup out[p_n]$, where
 $p_1, \dots, p_n$ are all the predecessors of b

$in[exit] = out[B2] \cup out[B3]$

Cyclic Graphs



- Equations still hold
 - $out[b] = f_b(in[b])$
 - $in[b] = out[p_1] \cup out[p_2] \cup \dots \cup out[p_n], p_1, \dots, p_n \text{ pred.}$
- Find: fixed point solution

$$in[B2] = out[B1] \cup out[B3]$$

Reaching Definitions: Iterative Algorithm

input: control flow graph $CFG = (N, E, \text{Entry}, \text{Exit})$

// Boundary condition

$\text{out}[\text{Entry}] = \emptyset$

// Initialization for iterative algorithm

For each basic block B other than Entry

$\text{out}[B] = \emptyset$

// iterate

While (changes to any $\text{out}[]$ occur) {

For each basic block B other than Entry {

$\text{in}[B] = U(\text{out}[p])$, for all predecessors p of B

$\text{out}[B] = f_B(\text{in}[B])$ // $\text{out}[B] = \text{gen}[B] \cup (\text{in}[B] - \text{kill}[B])$

}

Reaching Definitions: Worklist Algorithm

```
input: control flow graph CFG = (N, E, Entry, Exit)

// Initialize
  out[Entry] =  $\emptyset$            // could set out[Entry] to special def
                                // if reaching then undefined use

  For all nodes i
    out[i] =  $\emptyset$            // could optimize by out[i]=gen[i]
  ChangedNodes = N

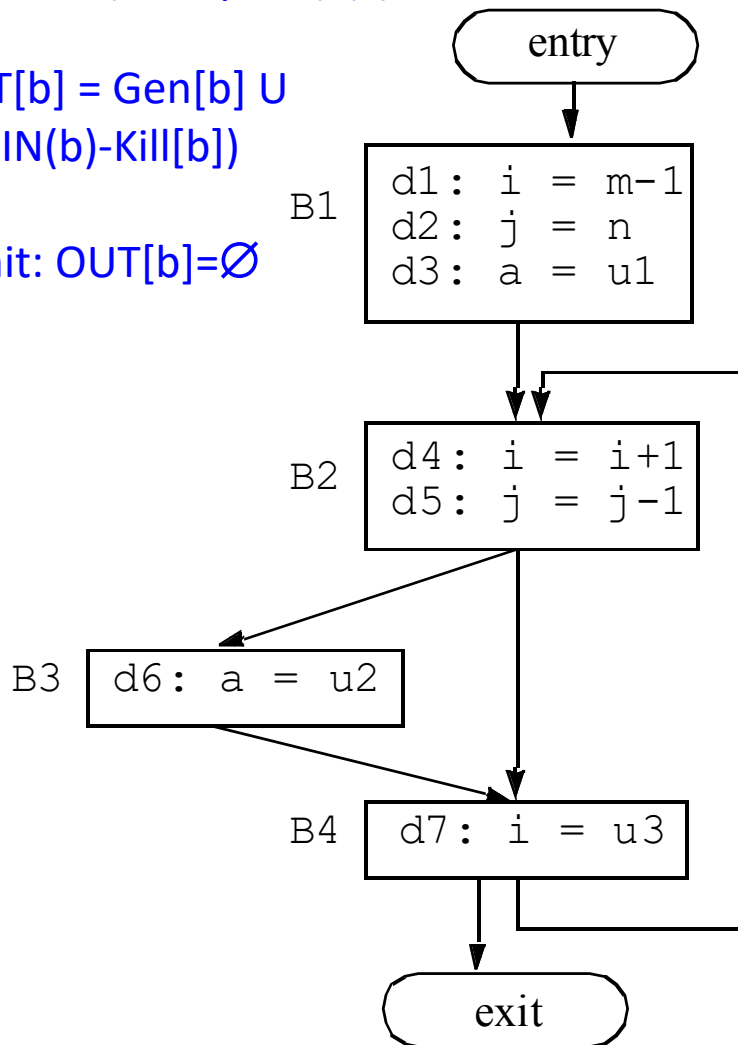
// iterate
  While ChangedNodes  $\neq \emptyset$  {
    Remove i from ChangedNodes
    in[i] = U(out[p]), for all predecessors p of i
    oldout = out[i]
    out[i] =  $f_i$ (in[i])         // out[i]=gen[i] U (in[i]-kill[i])
    if (oldout  $\neq$  out[i]) {
      for all successors s of i
        add s to ChangedNodes
    }
  }
```

Reaching Definitions Example

$IN[b] = U(OUT[pred(b)])$

$OUT[b] = Gen[b] \cup (IN(b) - Kill[b])$

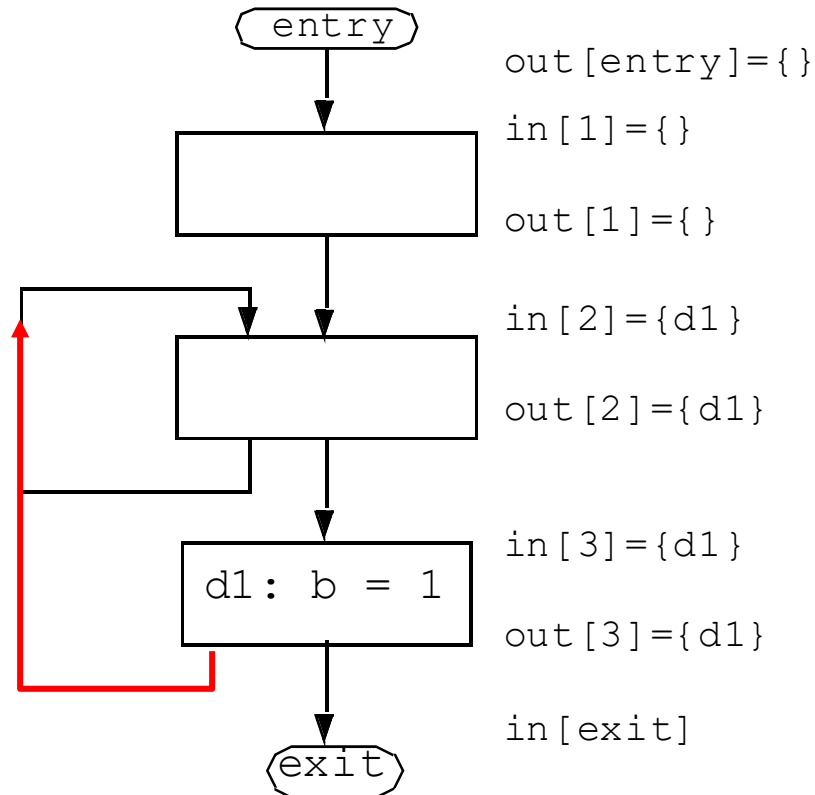
Init: $OUT[b] = \emptyset$



	First Pass	Second Pass
$IN[B1]$	000 00 0 0	000 00 0 0
$OUT[B1]$	111 00 0 0	111 00 0 0
$IN[B2]$	111 00 0 0	111 01 1 1
$OUT[B2]$	001 11 0 0	001 11 1 0
$IN[B3]$	001 11 0 0	001 11 1 0
$OUT[B3]$	000 11 1 0	000 11 1 0
$IN[B4]$	001 11 1 0	001 11 1 0
$OUT[B4]$	001 01 1 1	001 01 1 1
$IN[exit]$	001 01 1 1	001 01 1 1

Fixed point!

A valid solution to Reaching Definitions?



Fixed point:
another iteration
of algorithm
won't change
in/out values

- Is the solution a fixed point? **yes** Is it valid? **no**
- Will the worklist algorithm generate this answer? **no**
- What if add control flow edge shown in red? **yes**

III. Live Variable Analysis

- **Definition**

- A variable v is **live** at point p if
 - the value of v is used along some path in the flow graph starting at p .
- Otherwise, the variable is **dead**.

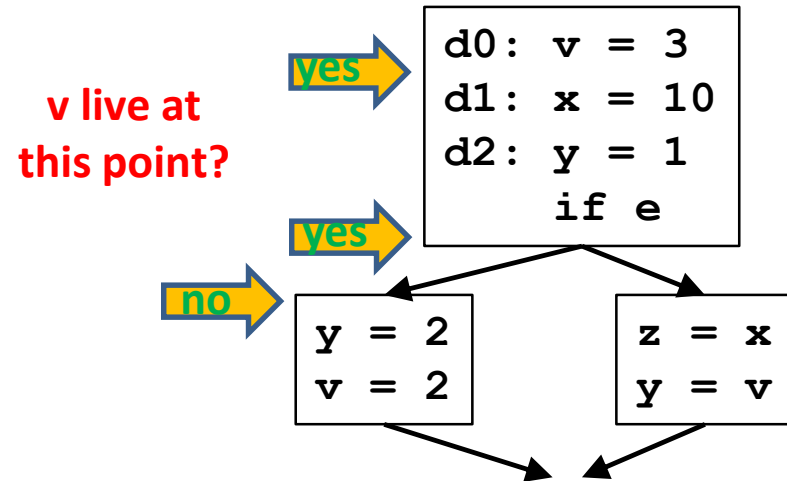
- **Motivation**

- e.g. register allocation

```
for i = 0 to n
  ... i ...
...
for i = 0 to n
  ... i ...
```

- **Problem statement**

- For each basic block
 - determine if each variable is live in each basic block
- Size of bit vector: one bit for each variable

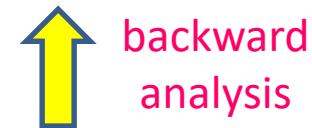
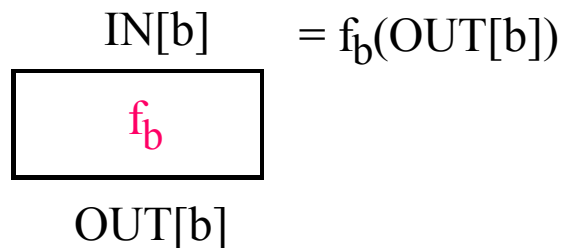
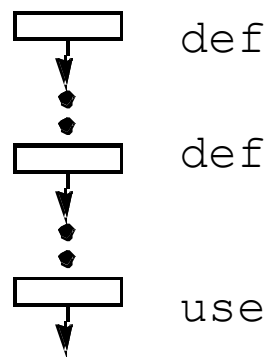


Live Variables: Effects of a Basic Block (Transfer Function)

- Insight: Trace uses **backwards** to the definitions

an execution path

control flow



- A basic block **b** can
 - generate live variables: **Use[b]**
 - set of locally exposed uses in **b**
 - propagate incoming live variables: **OUT[b] - Def[b]**,
 - where **Def[b]** = set of variables defined in **b.b**.
- transfer function** for block **b**:
 $in[b] = Use[b] \cup (out[b] - Def[b])$

example

d4: d = 1

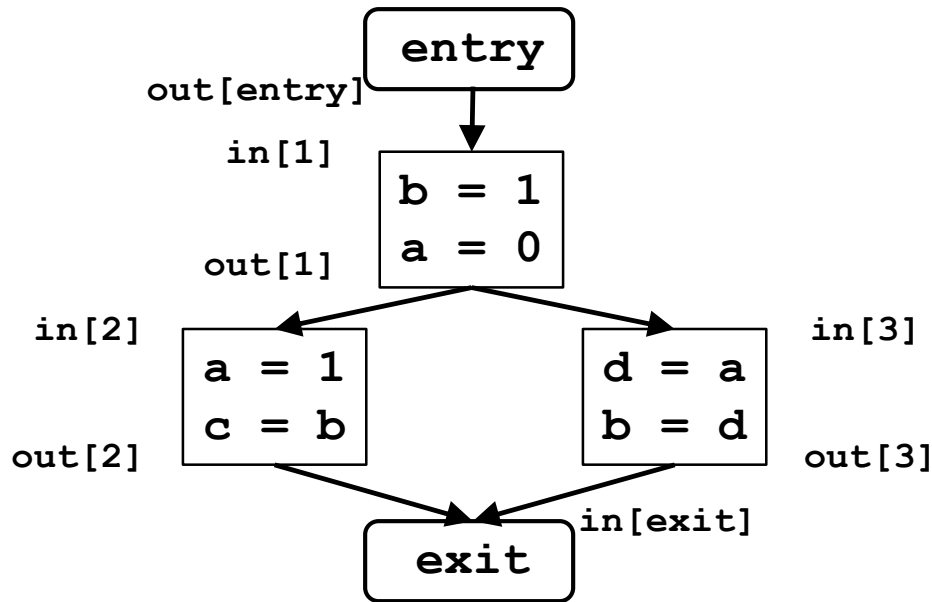
d5: c = a

d6: a = 4

$IN[b] =$

$\{a\} \cup (OUT[b] - \{a, c, d\})$

Flow Graph



f	Use	Def
1	{}	{a,b}
2	{b}	{a,c}
3	{a}	{b,d}

- $\text{in}[b] = f_b(\text{out}[b])$
- **Join node**: a node with multiple successors
- **meet** operator:
$$\text{out}[b] = \text{in}[s_1] \cup \text{in}[s_2] \cup \dots \cup \text{in}[s_n], \text{ where}$$
$$s_1, \dots, s_n \text{ are all successors of } b$$

Live Variables: Iterative Algorithm

input: control flow graph $CFG = (N, E, Entry, Exit)$

// Boundary condition

$in[Exit] = \emptyset$

// Initialization for iterative algorithm

For each basic block B other than Exit

$in[B] = \emptyset$

// iterate

While (changes to any $in[]$ occur) {

For each basic block B other than Exit {

$out[B] = \bigcup (in[s])$, for all successors s of B

$in[B] = f_B(out[B])$ // $in[B] = Use[B] \cup (out[B] - Def[B])$

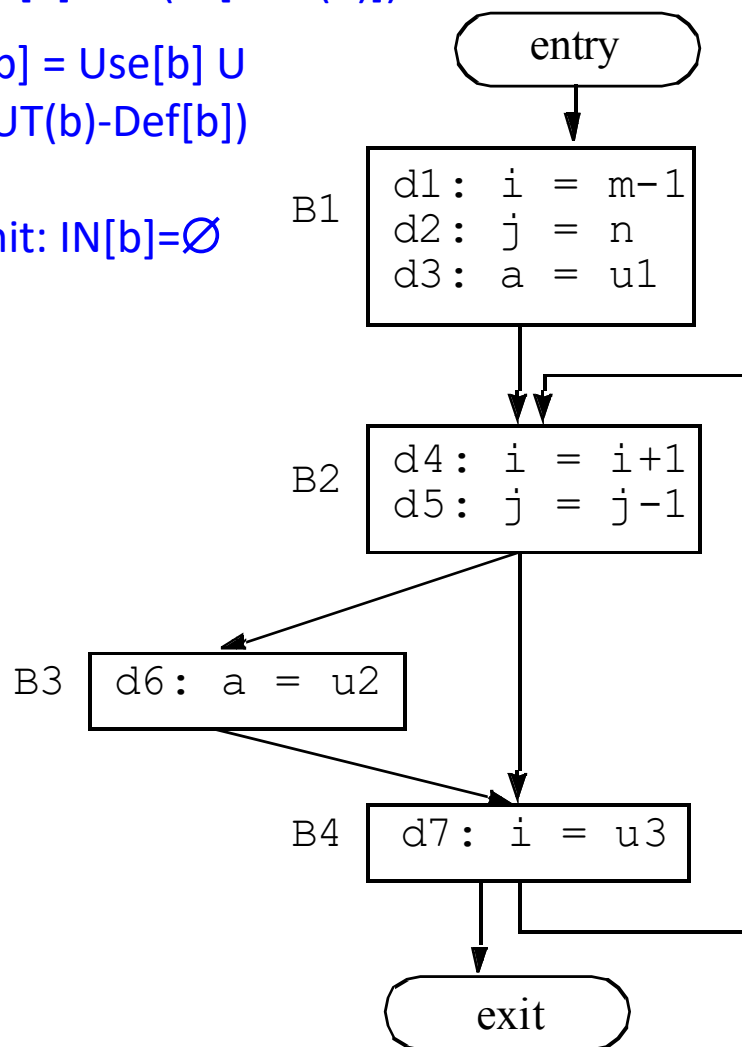
}

Live Variables Example

$OUT[b] = U(IN[succ(b)])$

$IN[b] = Use[b] \cup (OUT[b] - Def[b])$

Init: $IN[b] = \emptyset$



	First Pass	Second Pass
OUT[entry]	{m,n,u1,u2,u3}	{m,n,u1,u2,u3}
IN[B1]	{m,n,u1,u2,u3}	{m,n,u1,u2,u3}
OUT[B1]	{i,j,u2,u3}	{i,j,u2,u3}
IN[B2]	{i,j,u2,u3}	{i,j,u2,u3}
OUT[B2]	{u2,u3}	{j,u2,u3}
IN[B3]	{u2,u3}	{j,u2,u3}
OUT[B3]	{u3}	{j,u2,u3}
IN[B4]	{u3}	{j,u2,u3}
OUT[B4]	{}	{i,j,u2,u3}

Fixed point!

IV. Framework

	Reaching Definitions	Live Variables
Domain	Sets of definitions	Sets of variables
Direction	forward: $\text{out}[b] = f_b(\text{in}[b])$ $\text{in}[b] = \wedge \text{out}[\text{pred}(b)]$	backward: $\text{in}[b] = f_b(\text{out}[b])$ $\text{out}[b] = \wedge \text{in}[\text{succ}(b)]$
Transfer function	$f_b(x) = \text{Gen}_b \cup (x - \text{Kill}_b)$	$f_b(x) = \text{Use}_b \cup (x - \text{Def}_b)$
Meet Operation ($\wedge$)	$\cup$	$\cup$
Boundary Condition	$\text{out}[\text{entry}] = \emptyset$	$\text{in}[\text{exit}] = \emptyset$
Initial interior points	$\text{out}[b] = \emptyset$	$\text{in}[b] = \emptyset$

Other Data Flow Analysis problems fit into this general framework,
e.g., Available Expressions [ALSU 9.2.6]

Lecture 6

Foundations of Data Flow Analysis

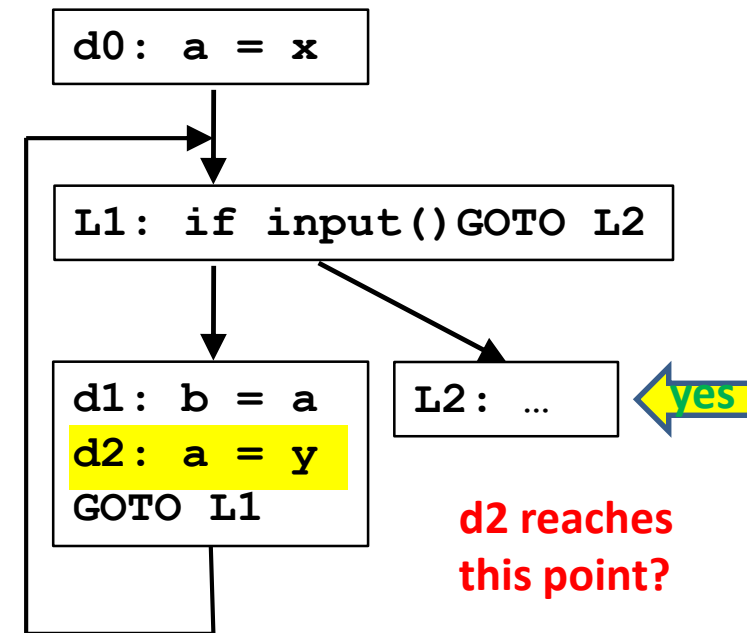
- I. Meet operator
- II. Transfer functions
- III. Correctness, Precision, Convergence
- IV. Efficiency

ALSU 9.3

Review: Reaching Definitions

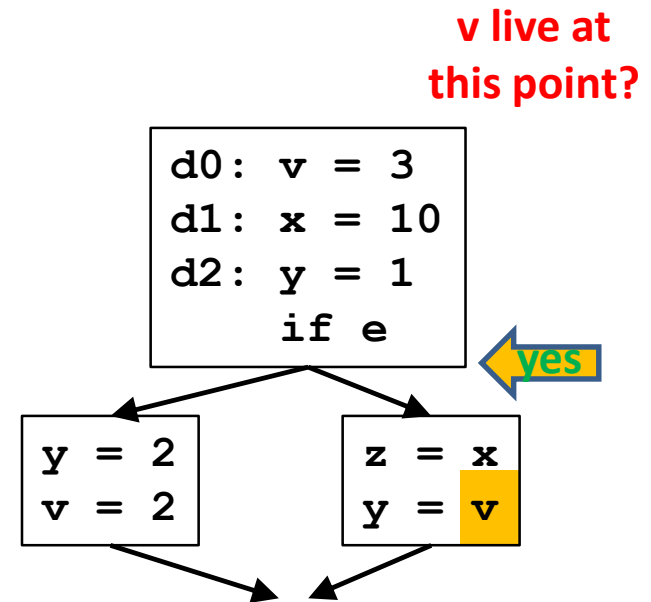
- A definition **d** **reaches** a point *p* if
 - **there exists** a path from the point immediately following *d* to *p* such that *d* is **not killed** (overwritten) along that path.
- A basic block **b** can
 - **generate** new definitions: **Gen[b]**
 - set of definitions in *b* that reach end of *b*
 - **propagate** incoming definitions: **in[b] - Kill[b]**,
 - where **Kill[b]** = set of defs killed by defs in *b*
- **Forward analysis**
 - **transfer function** for block *b*:
$$\text{out}[b] = \text{Gen}[b] \cup (\text{in}[b] - \text{Kill}[b])$$
- **meet** operator:
$$\text{in}[b] = \text{out}[p_1] \cup \text{out}[p_2] \cup \dots \cup \text{out}[p_n], \text{ where }$$

$p_1, \dots, p_n$ are all the predecessors of *b*



Review: Live Variable Analysis

- A variable v is **live** at point p if
 - the value of v is used along some path in the flow graph starting at p .
- A basic block b can
 - generate live variables: **Use[b]**
 - set of locally exposed uses in b
 - propagate incoming live variables: **out[b] - Def[b]**,
 - where **Def[b]** = set of variables defined in $b.b$.
- **Backward analysis**
 - **transfer function** for block b :
$$\text{in}[b] = \text{Use}[b] \cup (\text{out}[b] - \text{Def}[b])$$
- **meet** operator:
$$\text{out}[b] = \text{in}[s_1] \cup \text{in}[s_2] \cup \dots \cup \text{in}[s_n], \text{ where}$$
$$s_1, \dots, s_n \text{ are all successors of } b$$



Review: Data Flow Analysis Framework

	Reaching Definitions	Live Variables
Domain	Sets of definitions	Sets of variables
Direction	forward: $\text{out}[b] = f_b(\text{in}[b])$ $\text{in}[b] = \wedge \text{out}[\text{pred}(b)]$	backward: $\text{in}[b] = f_b(\text{out}[b])$ $\text{out}[b] = \wedge \text{in}[\text{succ}(b)]$
Transfer function	$f_b(x) = \text{Gen}_b \cup (x - \text{Kill}_b)$	$f_b(x) = \text{Use}_b \cup (x - \text{Def}_b)$
Meet Operation ($\wedge$)	$\cup$	$\cup$
Boundary Condition	$\text{out}[\text{entry}] = \emptyset$	$\text{in}[\text{exit}] = \emptyset$
Initial interior points	$\text{out}[b] = \emptyset$	$\text{in}[b] = \emptyset$

Other Data Flow Analysis problems fit into this general framework,
e.g., Available Expressions & Constant Propagation (Lecture 7)

A Unified Framework

- **Data flow problems are defined by**
 - Domain of values: V
 - Meet operator ($V \wedge V \rightarrow V$), initial value
 - A set of transfer functions ($V \rightarrow V$)
- **Usefulness of unified framework**
 - To answer questions such as
correctness, precision, convergence, speed of convergence
for a family of problems
 - If meet operators and transfer functions have properties X, then we know Y about the above.
 - Reuse code

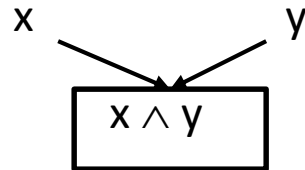
Overview: A Check List for Data Flow Problems

- **Semi-lattice**
 - set of values
 - meet operator
 - top, bottom
 - finite descending chain?
- **Transfer functions**
 - function of each basic block
 - monotone
 - distributive?
- **Algorithm**
 - initialization step (entry/exit, other nodes)
 - visit order: rPostOrder
 - depth of the graph

I. Meet Operator

- Properties of the meet operator

- commutative: $x \wedge y = y \wedge x$

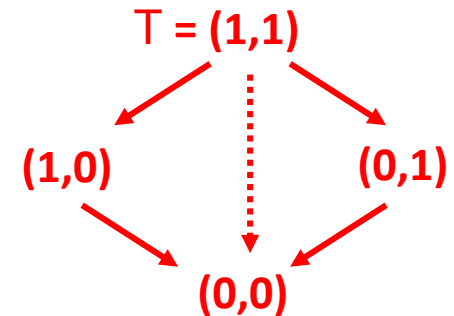


- idempotent: $x \wedge x = x$
- associative: $x \wedge (y \wedge z) = (x \wedge y) \wedge z$
- there is a **Top** element T such that $x \wedge T = x$

- Meet operator defines a partial ordering on values

- $x \leq y$ if and only if $x \wedge y = x$
 - Transitivity: if $x \leq y$ and $y \leq z$ then $x \leq z$
 - Antisymmetry: if $x \leq y$ and $y \leq x$ then $x = y$
 - Reflexivity: $x \leq x$

Partial Order

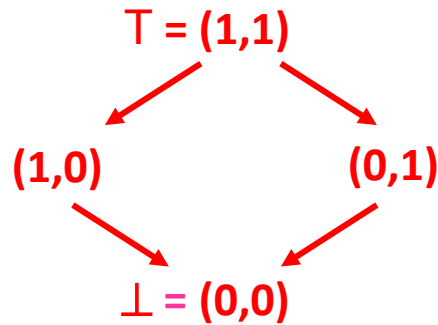


Meet Operator:
Elementwise-min

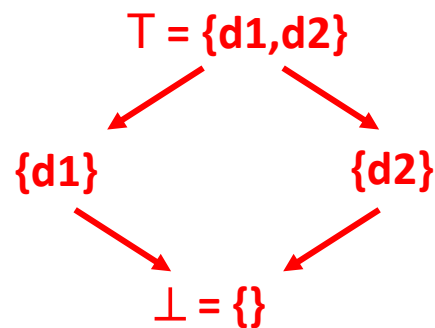
Note: $x < y$ is depicted as
 $y \rightarrow x$ in diagram

Note: typically show only
minimal (i.e., transitively
reduced) set of edges.
[not the dashed edge above]

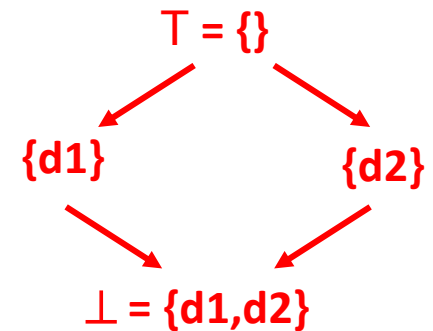
Partial Order



Meet Operator:
Elementwise-min



Meet Operator?
Intersection



Meet Operator?
Union

- Top and Bottom elements
 - Top T such that: $x \wedge T = x$
 - Bottom $\perp$ such that: $x \wedge \perp = \perp$
- Values and meet operator in a data flow problem define a semi-lattice:
 - there exists a T , but not necessarily a $\perp$.
- x, y are ordered: $x \leq y$ then $x \wedge y = x$

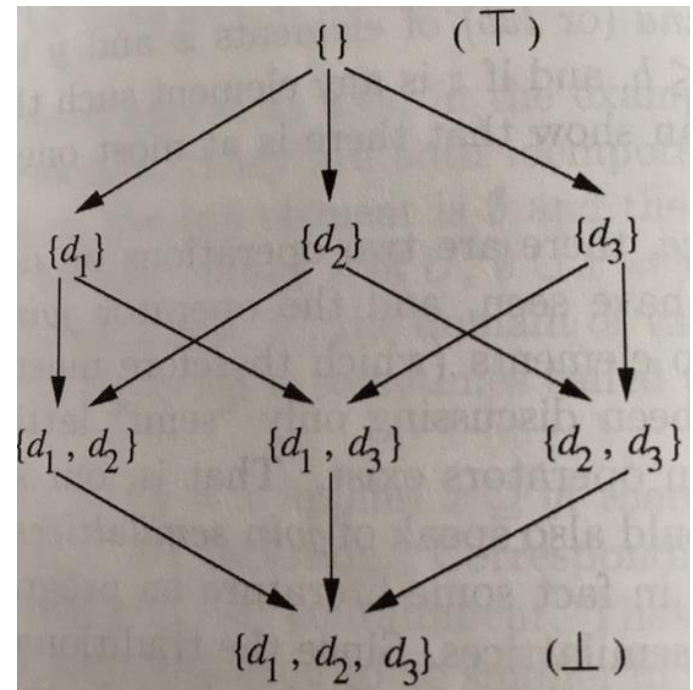
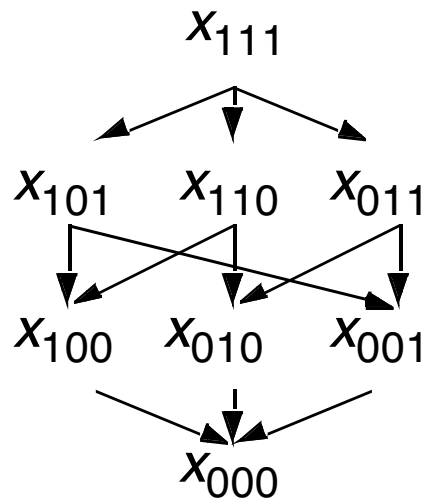
Note: $x < y$ is depicted as
 $y \rightarrow x$ in diagram

One vs. All Variables/Definitions

- Lattice for each variable: e.g. intersection



- Lattice for three variables for intersection:

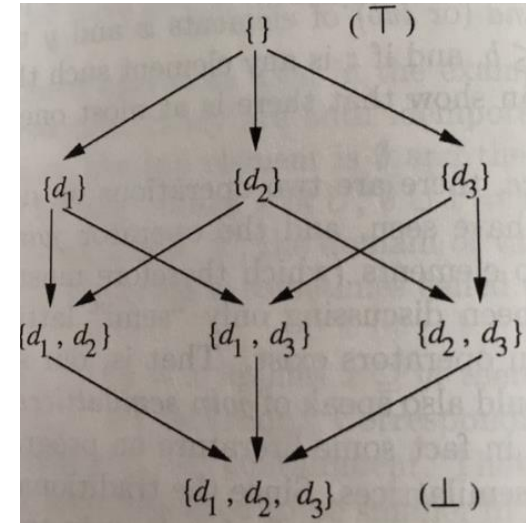


How many variables? **3**

Meet operator? **union**

Further Properties of Meet Operators

- $x < y$ means x is **more conservative** than y
 - Prefer x only if necessary for correctness
 - Otherwise y is **more precise**
- x, y are **ordered**: $x \leq y$ iff $x \wedge y = x$
 - E.g., $\{d_1, d_2\} \leq \{d_2\}$ and $\{d_1, d_2\} \wedge \{d_2\} = \{d_1, d_2\}$
- For all a, b (even if unordered): $a \wedge b \leq a$
 - Why? $(a \wedge b) \wedge a = (b \wedge a) \wedge a = b \wedge (a \wedge a)$
 $= b \wedge a = a \wedge b$
 - Thus, considering more paths decreases result of $\wedge$ (or leaves unchanged)
- What if x and y are not ordered? $x \wedge y \leq x$ and $x \wedge y \leq y$
 - If $w \leq x$ and $w \leq y$ then $w \leq x \wedge y$
 - E.g., $x = \{d_1\}$, $y = \{d_3\}$, and $w = \{d_1, d_3\}$ or $\{d_1, d_2, d_3\}$



Union
 $\leq$: **superset**

Descending Chain

- **Definition**

- The **height** of a lattice is the largest number of **> relations** that will fit in a descending chain.

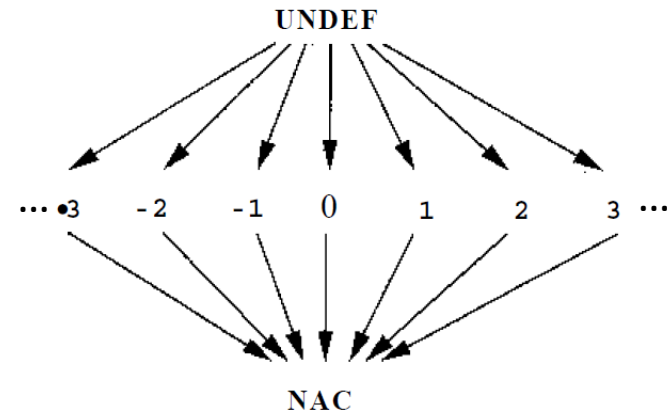
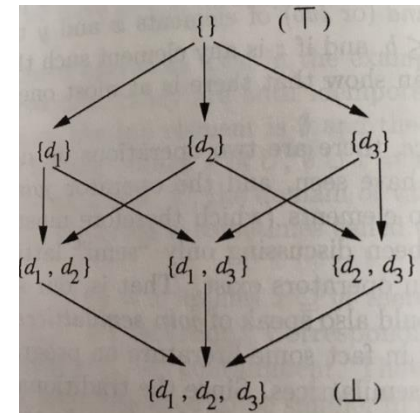
$$x_0 > x_1 > x_2 > \dots$$

- **Height of values in reaching definitions?**

Height=n, where n is the number of definitions

- Important property: **finite descending chain**

- Can an infinite lattice have a finite descending chain? **yes**
- Example: Constant Propagation/Folding
 - To determine if an integer variable is a constant
- Domain of values:
 - undef, ... -1, 0, 1, 2, ..., not-a-constant



II. Transfer Functions

e.g., $\text{out}[b] = \text{Gen}[b] \cup (\text{in}[b] - \text{Kill}[b])$

- **Basic Properties** $f: V \rightarrow V$
 - Has an identity function
 - There exists an f such that $f(x) = x$, for all x .
 - Closed under composition
 - if $f_1, f_2 \in F$, then $f_1 \cdot f_2 \in F$

Monotonicity

Transfer function $f: V \rightarrow V$
e.g., $\text{out}[b] = \text{Gen}[b] \cup (\text{in}[b] - \text{Kill}[b])$

- A framework $(F, V, \wedge)$ is **monotone** if and only if
 - $x \leq y$ implies $f(x) \leq f(y)$
 - i.e. a “smaller or equal” input to the same function will always give a “smaller or equal” output
- **Equivalently**, a framework $(F, V, \wedge)$ is **monotone** if and only if
 - $f(x \wedge y) \leq f(x) \wedge f(y)$
 - i.e. merge input, then apply f is **small than or equal to** apply the transfer function individually and then merge the result

Example: Reaching Definitions is Monotone

- **Reaching definitions:** $f(x) = \text{Gen} \cup (x - \text{Kill})$, $\wedge = \cup$

- Definition 1: $x \leq y$ implies $f(x) \leq f(y)$
 - $x \leq y$ implies $(x - \text{Kill}) \leq (y - \text{Kill})$
implies $\text{Gen} \cup (x - \text{Kill}) \leq \text{Gen} \cup (y - \text{Kill})$

- Definition 2: $f(x \wedge y) \leq f(x) \wedge f(y)$

- $(\text{Gen} \cup ((x \cup y) - \text{Kill}))$
 $= (\text{Gen} \cup (x - \text{Kill})) \cup (\text{Gen} \cup (y - \text{Kill}))$

- **Note: Monotone framework does not mean that $f(x) \leq x$**

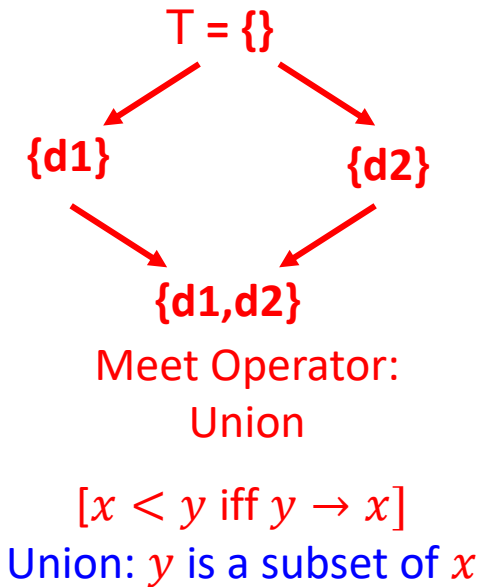
- E.g., consider reaching definitions, where d_1 and d_2 define the same variable
- Then the transfer function $f(x)$ for a basic block that defines only d_1 has

$$\text{Gen} = \{d_1\} \text{ and } \text{Kill} = \{d_2\}$$

- Let $x = \{d_2\}$. Then $f(x) = \{d_1\}$ which is **unordered** w.r.t. $x = \{d_2\}$.

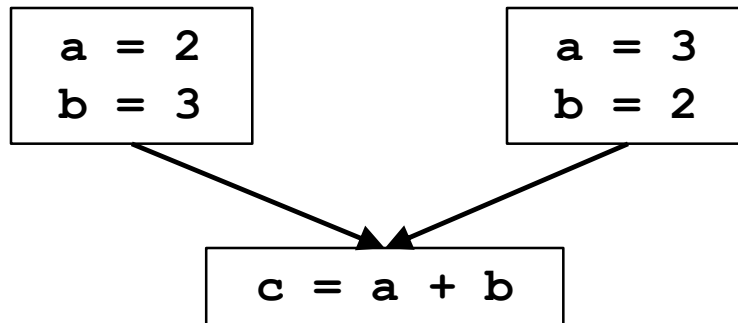
- **If $\text{input}(\text{second iteration}) \leq \text{input}(\text{first iteration})$**

- $\text{result}(\text{second iteration}) \leq \text{result}(\text{first iteration})$



Distributivity

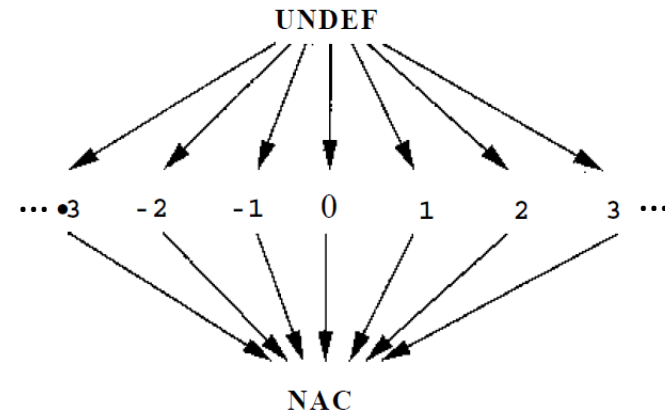
- A framework $(F, V, \wedge)$ is **distributive** if and only if
 - $f(x \wedge y) = f(x) \wedge f(y)$
 - i.e., merge input, then apply f is **equal to** apply the transfer function individually then merge result
- Is Reaching Definitions distributive? **yes**
- Is Constant Propagation distributive? **no**



Consider c :

$f(x) \wedge f(y) = 5$

$f(x \wedge y) = \text{NAC}$



III. Data Flow Analysis

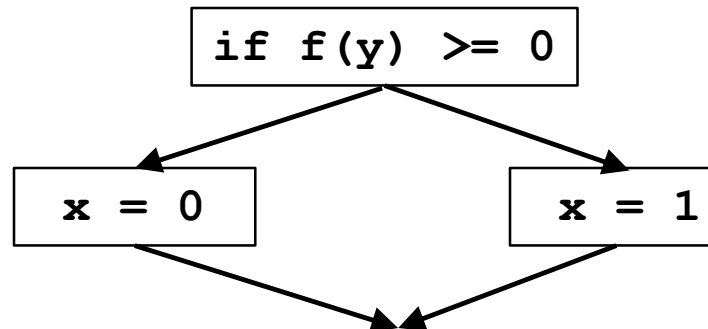
- **Definition**

- Let $f_1, \dots, f_m : \in F$, where f_i is the transfer function for node i
 - $f_p = f_{n_k} \cdot \dots \cdot f_{n_1}$, where p is a path through nodes $n_1, \dots, n_k$
 - $f_p = \text{identify function}$ if p is an empty path

- **Perfect data flow answer:**

- For each node n :

$\wedge f_{p_i}(T)$, for all possibly executed paths p_i in the program reaching n .



If $f(y)$ is always non-negative then right path never taken

- In general: Determining all possibly executed paths is **undecidable**

Meet-Over-Paths (MOP)

- Err in the conservative direction

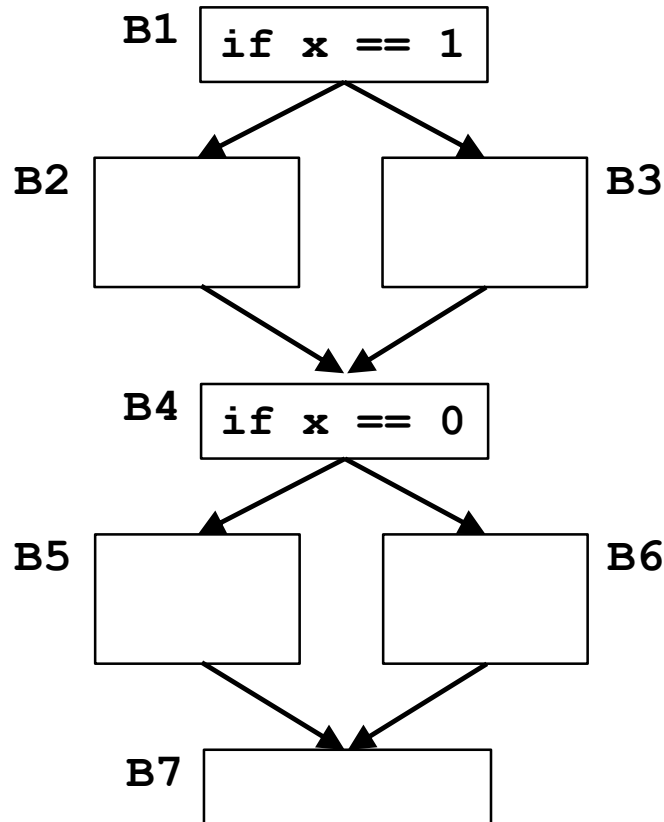
- **Meet-Over-Paths (MOP):**

- For each node n :

$$\text{MOP}(n) = \bigwedge f_{p_i}(\top), \text{ for all paths } p_i \text{ in data flow graph reaching } n$$

- a path exists as long there is an edge in the code
 - $\text{MOP} = \text{Perfect-Solution} \wedge \text{Solution-to-Unexecuted-Paths}$
 - $\text{MOP} \leq \text{Perfect-Solution}$
 - Considers more paths than necessary, hence solution is **conservative**
 - Meet = union: Definition may reach; Variable may be live
 - Meet = intersection: Expression is always available even when consider extra paths
 - Considering too few paths ($>$ Perfect-Solution) would not be **safe!**
- **Desirable solution: as close to MOP as possible**

Example: MOP considers more paths than Perfect



Perfect considers only which 2 paths?

B1-B2-B4-B6-B7 (i.e., $x=1$)

B1-B3-B4-B5-B7 (i.e., $x=0$)

MOP also considers which other (unexecuted) paths?

B1-B2-B4-B5-B7

B1-B3-B4-B6-B7

What changes if $x \in \{0,1,2\}$?

B1-B3-B4-B6-B7 is also considered by Perfect

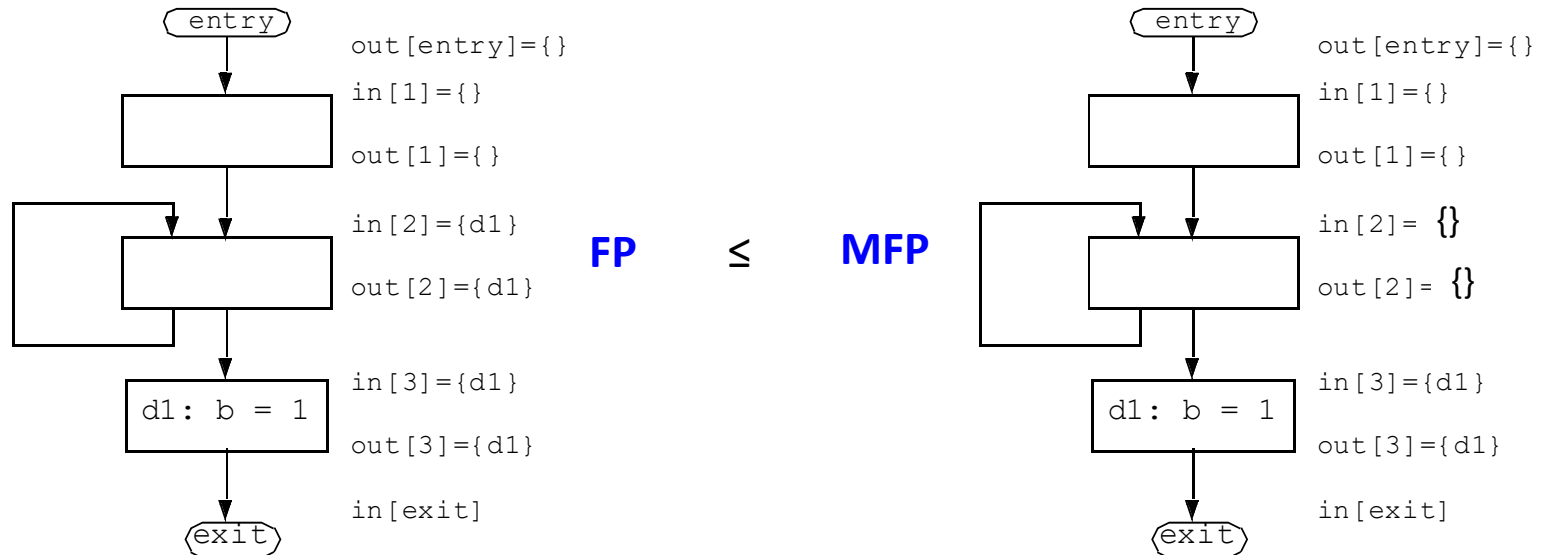
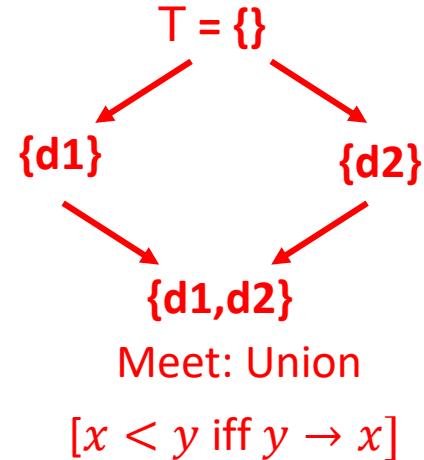
Assume $x \in \{0,1\}$ and B2 & B3 do not update x

Solving Data Flow Equations

- **Framework** $(F, V, \wedge)$ defines set of equations relating $\text{in}[b]$'s and $\text{out}[b]$'s
- **Any solution satisfying equations = Fixed Point Solution (FP)**
- **Iterative algorithm for forward analysis** (backward analysis case is symmetric)
 - initializes $\text{out}[b]$ to T for all b
 - if framework is monotone & algorithm converges, then it computes **Maximum Fixed Point (MFP)**:
 - **MFP** is the **largest of all solutions to equations** (in any other solution, the values of $\text{IN}[b]$ and $\text{OUT}[b]$ are $\leq$ the corresponding values of the MFP)
- **Properties:**
 - $\text{FP} \leq \text{MFP} \leq \text{MOP} \leq \text{Perfect-solution}$
 - FP, MFP are safe
 - If monotone & converges, then $\text{in}[b] \leq \text{MOP}[b]$

Solving Data Flow Equations

- **Example: Reaching definitions**
 - **Values** = {subsets of definitions}. Init $\text{out}[b] = \{\}$
 - **Meet operator**: $\text{in}[b] = \cup \text{out}[p]$, for all predecessors p of b
 - **Transfer functions**: $\text{out}[b] = \text{gen}_b \cup (\text{in}[b] - \text{kill}_b)$
- **Any solution satisfying equations = Fixed Point Solution (FP)**
- **Iterative algorithm computes Maximum Fixed Point (MFP)**:
 - In any other solution, the values of $\text{IN}[b]$ and $\text{OUT}[b]$ are $\leq$ the corresponding values of the MFP

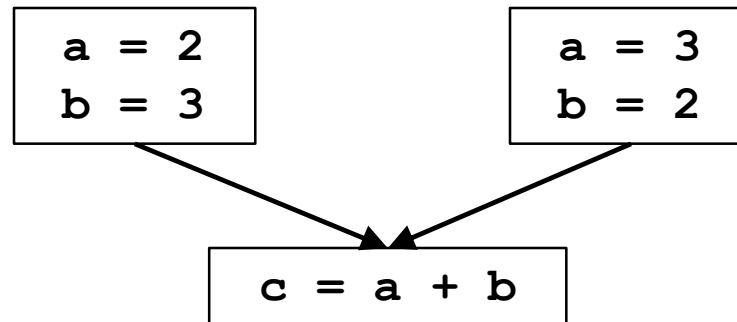


Partial Correctness of Algorithm

- If data flow framework is **monotone** (i.e., $x \leq y$ implies $f(x) \leq f(y)$) then if the algorithm converges, $IN[b] \leq MOP[b]$
- **Proof: Induction on path lengths**
 - Define $IN[entry] = OUT[entry]$
and transfer function of entry = Identity function
 - Base case: path of length 0
 - Proper initialization of $IN[entry]$
 - If true for path of length k , $p_k = (n_1, \dots, n_k)$, then true for path of length $k+1$: $p_{k+1} = (n_1, \dots, n_{k+1})$
 - Assume: $IN[n_k] \leq f_{n_{k-1}}(f_{n_{k-2}}(\dots f_{n_1}(IN[entry])))$
 - $IN[n_{k+1}] = OUT[n_k] \wedge \dots$
$$\leq OUT[n_k] = f_{n_k}(IN[n_k])$$
$$\leq f_{n_k}(f_{n_{k-1}}(\dots f_{n_1}(IN[entry]))) \quad \text{by inductive assumption \& monotonicity}$$

Precision

- If data flow framework is **distributive** (i.e., $f(x \wedge y) = f(x) \wedge f(y)$) then if the algorithm converges, $IN[b] = MOP[b]$
- Monotone but not distributive: behaves as if there are additional paths

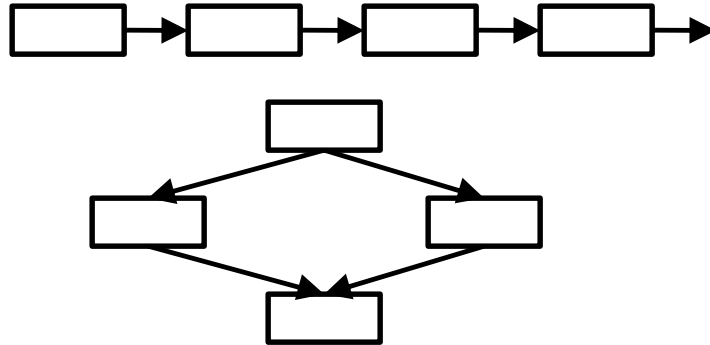


Additional Property to Guarantee Convergence

- Data flow framework (**monotone**) converges if there is a **finite descending chain**
- For each $IN[b]$, $OUT[b]$, consider the sequence of its values **across iterations**:
 - if sequence for $in[b]$ is monotonically decreasing ($in_{i+1}[b] \leq in_i[b]$)
then sequence for $out[b]$ is monotonically decreasing ($out_{i+1}[b] \leq out_i[b]$)
 - ($out[b]$ initialized to T) Why? **Transfer function is monotone**
 - if for all predecessors p of b , sequence for $out[p]$ is monotonically decreasing
then sequence for $in[b]$ is monotonically decreasing Why?
Consider predecessors p and p' of b such that $out_{i+1}[p] \leq out_i[p]$ and $out_{i+1}[p'] \leq out_i[p']$. Then $in_{i+1}[b] = out_{i+1}[p] \wedge out_{i+1}[p'] = (out_{i+1}[p] \wedge out_i[p]) \wedge (out_{i+1}[p'] \wedge out_i[p']) \leq out_i[p] \wedge out_i[p'] = in_i[b]$
- Must be at least one $out[b]$ change to warrant an additional iteration
 - Thus, guaranteed to converge after at most
(height of lattice) x (number of nodes in flow graph) iterations

IV. Speed of Convergence

- Speed of convergence depends on order of node visits



- Reverse “direction” for backward flow problems

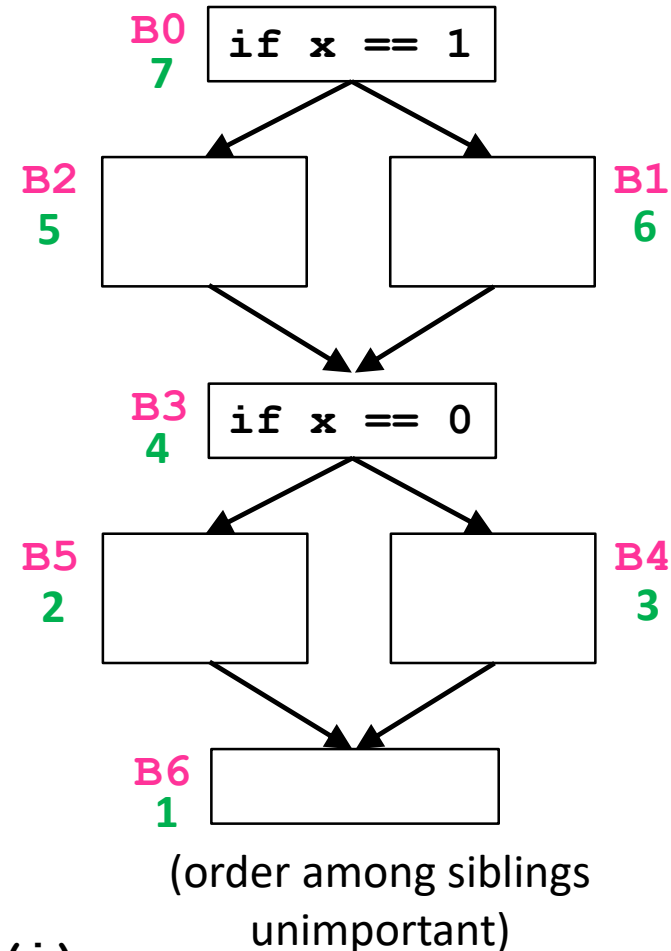
Reverse Postorder

- Step 1: depth-first post order

```
main() {  
    count = 1;  
    Visit(root);  
}  
Visit(n) {  
    for each successor s that  
        has not been visited  
        Visit(s);  
    PostOrder(n) = count;  
    count = count+1;  
}
```

- Step 2: reverse order

```
For each node i  
    rPostOrder(i) = NumNodes - PostOrder(i)
```



Depth-First Iterative Algorithm (forward)

input: control flow graph $CFG = (N, E, \text{Entry}, \text{Exit})$

/ Initialize */*

 out[entry] = init_value

 For all nodes i

 out[i] = $\perp$

 Change = True

/ iterate */*

 While Change {

 Change = False

 For each node i in rPostOrder {

 in[i] = $\wedge(\text{out}[p])$, for all predecessors p of i

 oldout = out[i]

 out[i] = $f_i(\text{in}[i])$

 if oldout $\neq$ out[i]

 Change = True

 }

 }

Speed of Convergence

- **If cycles do not add information***

- information can flow in one pass down nodes of increasing order number:

- e.g., $1 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 2 \rightarrow 6 \dots$

first pass

- passes determined by **number of back edges in the path**
 - essentially the nesting depth of the graph
- **Number of iterations = number of back edges in any acyclic path + 2**
 - (2 are necessary even for acyclic CFGs)
 - (2 not 1 since need a last pass where nothing changed)

- **What is the depth?**

- corresponds to depth of intervals for “reducible” graphs
- in real programs: average of 2.75

* E.g., if a defn d in node n_1 reaches a node n_k along a path that contains a cycle (i.e., a repeated node), then the cycle can be removed to form a shorter path from n_1 to n_k such that d reaches n_k .

Summary: A Check List for Data Flow Problems

- **Semi-lattice**
 - set of values
 - meet operator
 - top, bottom
 - finite descending chain?
- **Transfer functions**
 - function of each basic block
 - monotone
 - distributive?
- **Algorithm**
 - initialization step (entry/exit, other nodes)
 - visit order: rPostOrder
 - depth of the graph

Lecture 7

Global Common Subexpression Elimination; Constant Propagation/Folding

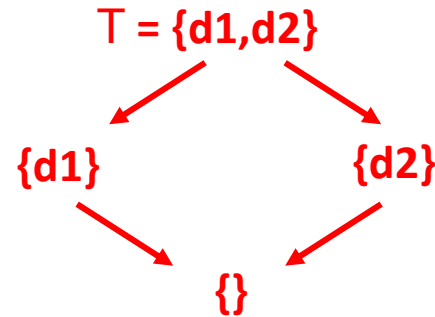
- I. Available Expressions Analysis
- II. Eliminating CSEs
- III. Constant Propagation/Folding

ALSU 9.2.6, 9.4

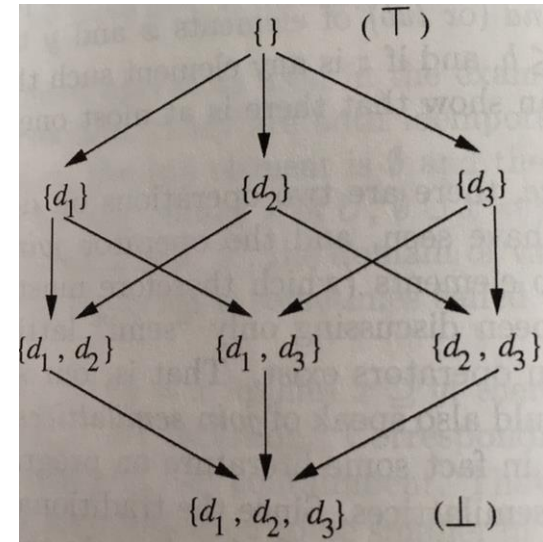
Review: A Check List for Data Flow Problems

- **Semi-lattice**

- set of values V
- meet operator $\wedge$
- Top T
- finite descending chain?



Meet Operator:
Intersection



Meet Operator:
Union

Review: A Check List for Data Flow Problems

- **Semi-lattice**

- set of values V
- meet operator $\wedge$
- Top T
- finite descending chain?

- **Transfer functions**

- function of a basic block $f: V \rightarrow V$
- closed under composition
- meet-over-paths **MOP**
- monotone
- distributive?

For each node n : $MOP(n) = \wedge f_{p_i}(T)$,
for all paths p_i in data-flow graph
reaching n .

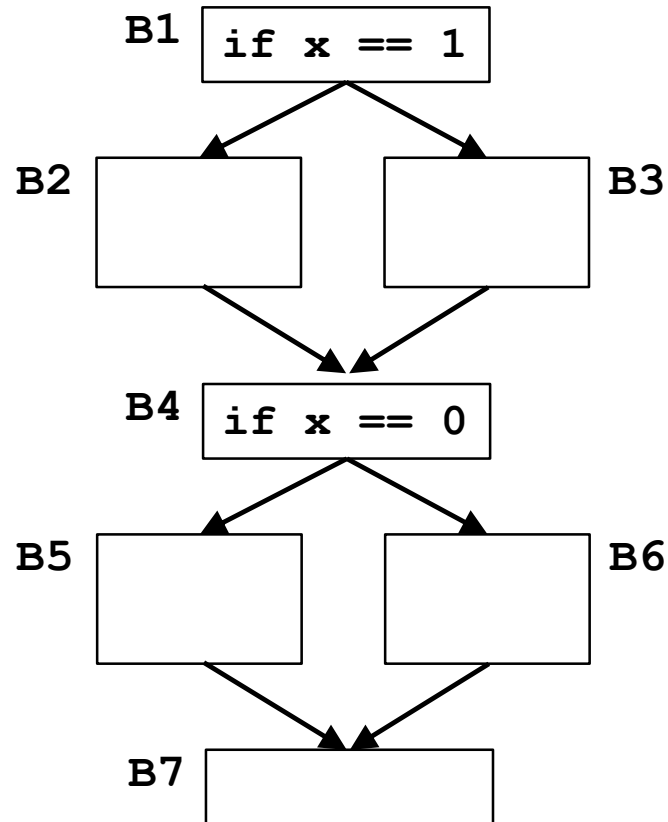
If data flow framework is **monotone**
(i.e., $x \leq y$ implies $f(x) \leq f(y)$)
then if the algorithm converges,
 $IN[b] \leq MOP[b]$ *, so analysis is ? **safe**.

Data flow framework (monotone)
converges if its lattice has ?
a finite descending chain.

If data flow framework is **distributive**
(i.e., $f(x \wedge y) = f(x) \wedge f(y)$)
then if the algorithm converges,
 $IN[b] = MOP[b]$ *, so ? **precision is high**.

* for backward analysis $OUT[b]$

Review: MOP considers more paths than Perfect



Perfect considers only:

B1-B2-B4-B6-B7 (i.e., $x=1$)

B1-B3-B4-B5-B7 (i.e., $x=0$)

MOP: Also considers unexecuted paths

B1-B2-B4-B5-B7

B1-B3-B4-B6-B7

What changes if $x \in \{0,1,2\}$?

B1-B3-B4-B6-B7 is also a Perfect path

Assume $x \in \{0,1\}$ and B2 & B3 do not update x

Review: A Check List for Data Flow Problems

- **Semi-lattice**

- set of values **V**
- meet operator **$\wedge$**
- Top **T**
- finite descending chain?

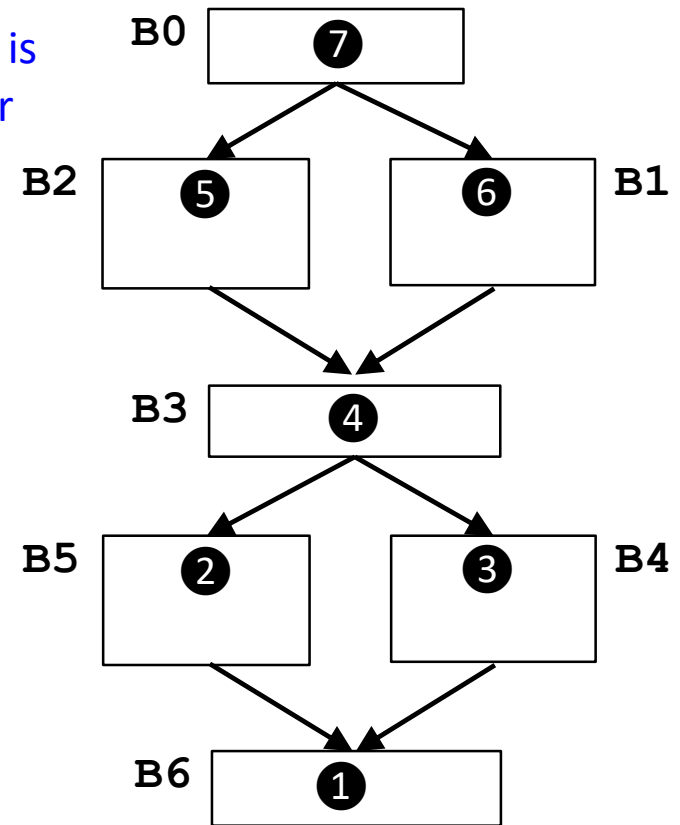
- **Transfer functions**

- function of a basic block **$f: V \rightarrow V$**
- closed under composition
- meet-over-paths **MOP**
- monotone
- distributive?

- **Algorithm**

- initialization step (entry/exit, other nodes)
- visit order: **rPostOrder**
- depth of the graph

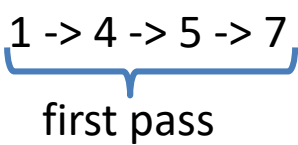
$B_0, B_1, \dots, B_6$ is
rPostOrder



Number of iterations = number of
back edges in any acyclic path + 2

Review: Speed of Convergence

- **If cycles do not add information***

- information can flow in one pass down nodes of increasing order number:
 - e.g., $1 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 2 \rightarrow 6 \dots$


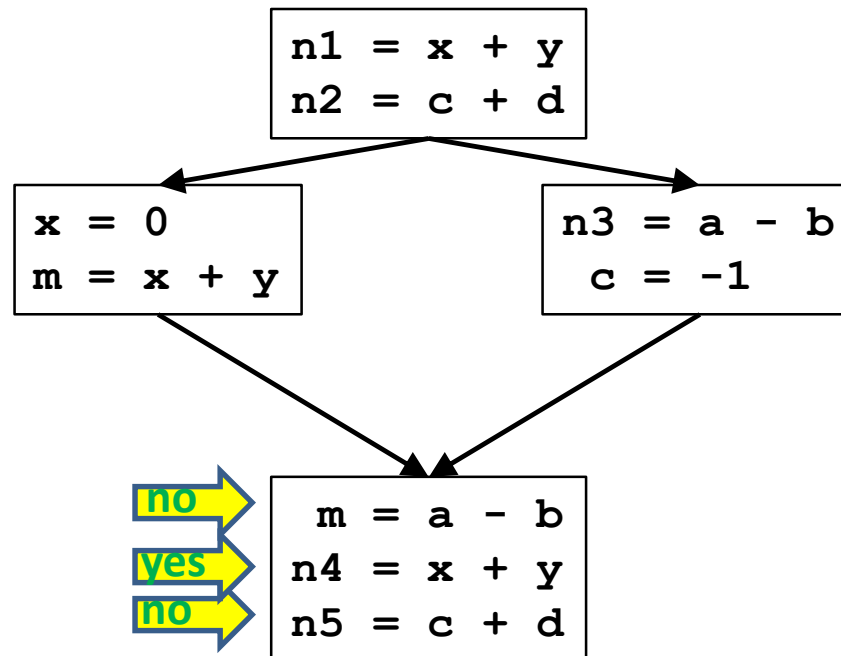
first pass
- passes determined by **number of back edges in the path**
 - essentially the nesting depth of the graph
- **Number of iterations = number of back edges in any acyclic path + 2**
 - (2 are necessary even for acyclic CFGs)
 - (2 not 1 since need a last pass where nothing changed)

* E.g., if a defn d in node n_1 reaches a node n_k along a path that contains a cycle (i.e., a repeated node), then the cycle can be removed to form a shorter path from n_1 to n_k such that d reaches n_k .

```
L: a = b
  b = c
  c = 1
  goto L
```

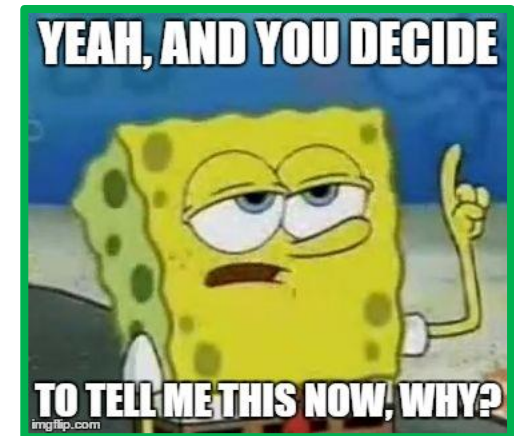
Example where cycles add information,
for constant propagation

I. Available Expressions Analysis



Is right-hand-side
expression available?

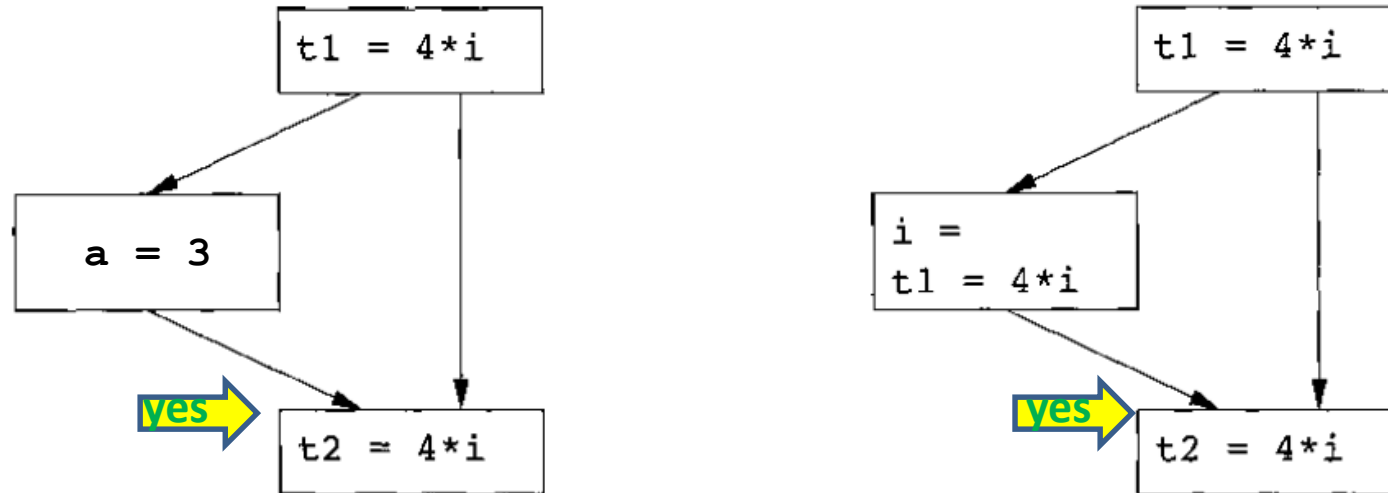
Part of Assignment #1



- **Availability** of an **expression E** at **point P**

- DEFINITION: Along **every** path to **P** in the flow graph:
 - E must be **evaluated at least once**
 - **no variables in E** redefined after the last evaluation
- Observation: E may have different values on different paths (e.g., $x+y$ above)

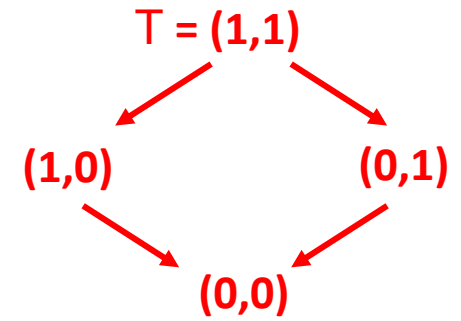
Available Expressions Example



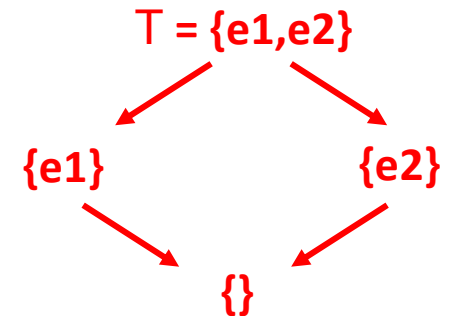
Is $4*i$ available at this point?

Formulating the Problem

- **Domain:**
 - a bit vector, with a bit for each “textually unique” expression in the program
- **Forward or Backward?** Forward
- **Lattice Elements?** All bit vectors of given length
- **Meet Operator?** Elementwise-min
 - check: commutative, idempotent, associative
- **Partial Ordering**
- **Top?** $(1,1,\dots,1)$
- **Bottom?** $(0,0,\dots,0)$
- **Boundary condition: entry/exit node?** $\text{out}[\text{entry}] = (0,\dots,0)$
- **Initialization for iterative algorithm?** Coming soon...



Meet Operator:
Elementwise-min



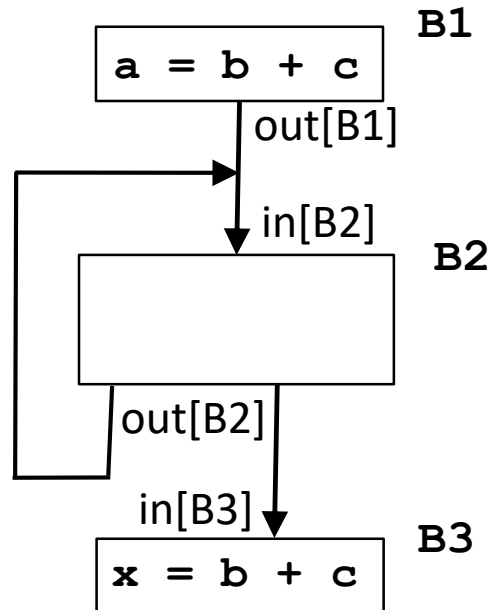
Meet Operator:
Intersection

Transfer Functions

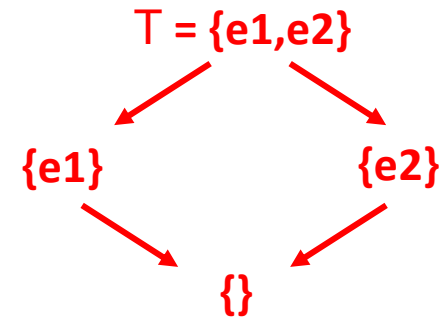
- Expression E is **available** at point P iff **along every path to P** in the flow graph:
 - E must be **evaluated at least once**
 - **no variables in E redefined** after the last evaluation
- Can use the same equation as reaching definitions
 - $out[b] = gen[b] \cup (in[b] - kill[b])$
- Start with the transfer function for a single instruction: **$x = y + z$**
 - When does the instruction kill an expression E? **It defines a variable in E.**
 - When does it generate an expression E? **It evaluates E and doesn't kill it.**
- Calculate transfer functions for complete basic blocks by composing individual instruction transfer functions

Statement	Available Expressions
	$\{\}$
$a = b + c$	$\{b+c\}$
$b = a - d$	$\{a-d\}$
$c = b + c$	$\{a-d\}$
$d = a - d$	$\{\}$

Initialization for Interior Nodes



$$\text{out}[b] = \text{Gen}[b] \cup (\text{in}(b) - \text{Kill}[b])$$



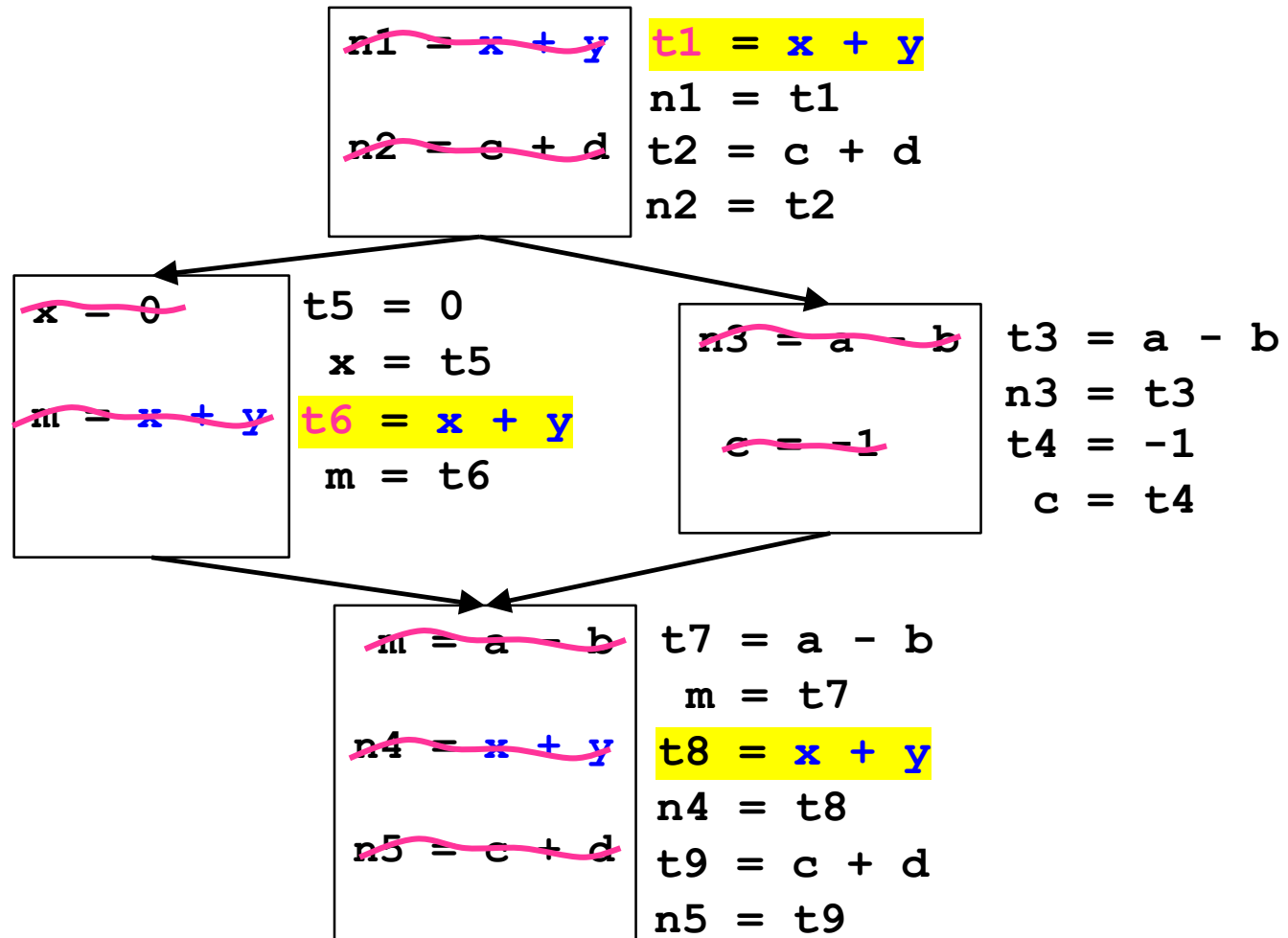
Meet Operator:
Intersection

- What if initialize $\text{out}[B2] = \{\}$? Imprecise: $\text{in}[B2] = \text{out}[B1] \wedge \text{out}[B2] = \{\}$
Thus, $\text{in}[B3] = \{\}$ each iteration, so conclude “b+c” is NOT available in B3.
- What if initialize $\text{out}[B2] = T$? Precise: $\text{in}[B2] = \text{out}[B1]$
Thus, $\text{in}[B3] = \{\text{“b+c”}\}$, so conclude “b+c” is available in B3.
- **Initialize $\text{out}[b] = T$ for all interior b**

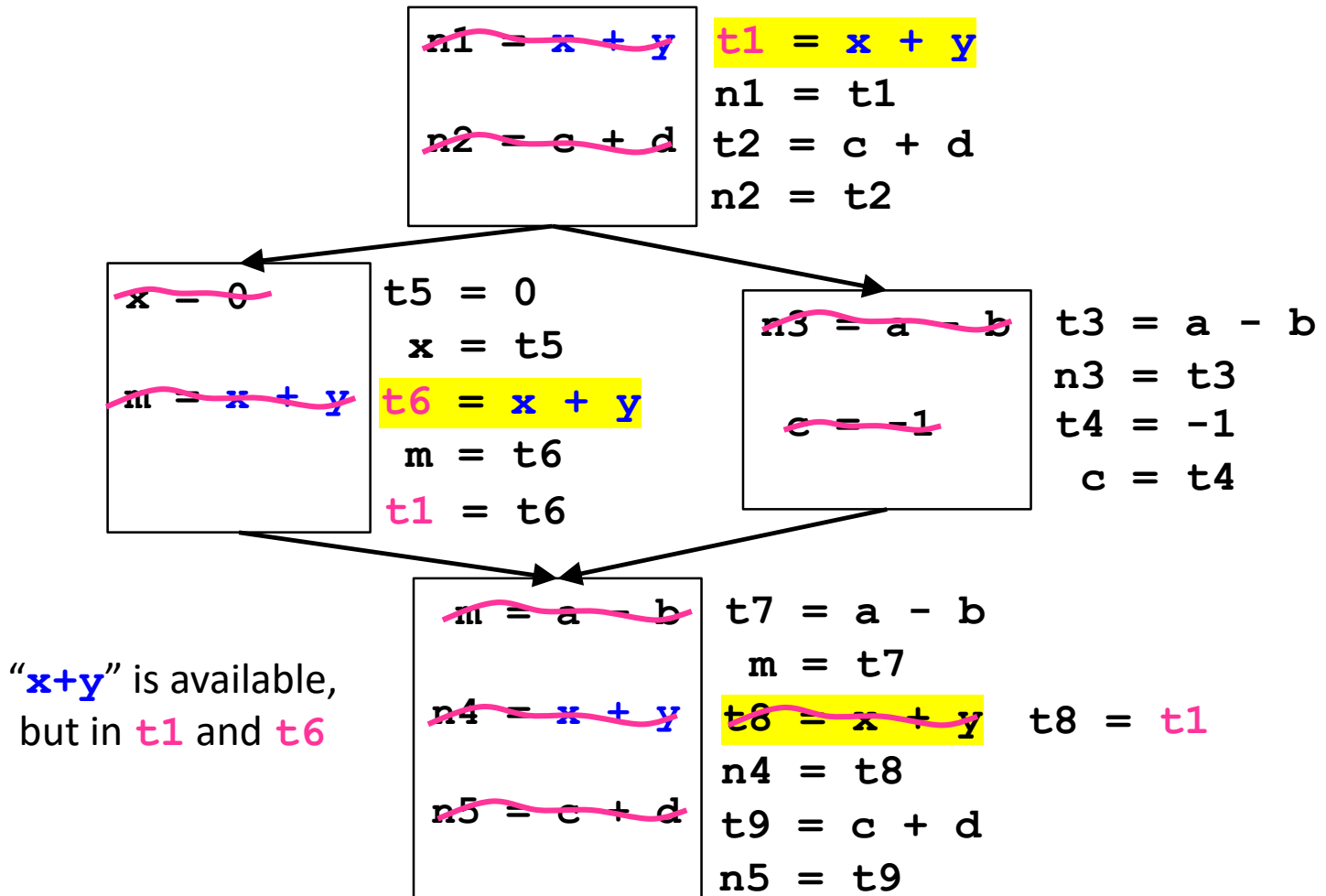
II. Eliminating CSEs

- **Value Numbering (within basic block)**
 - Eliminates local common subexpressions
- **Available expressions (across basic blocks)**
 - Provides the set of expressions available at the start of a block
- **If CSE is an “available expression”, then transform the code**
 - Original destination may be:
 - a temporary register
 - overwritten
 - different from the variables on other paths
 - One solution: Copy the expression to a new variable at each evaluation reaching the redundant use

Example Revisited: Value Numbering Only

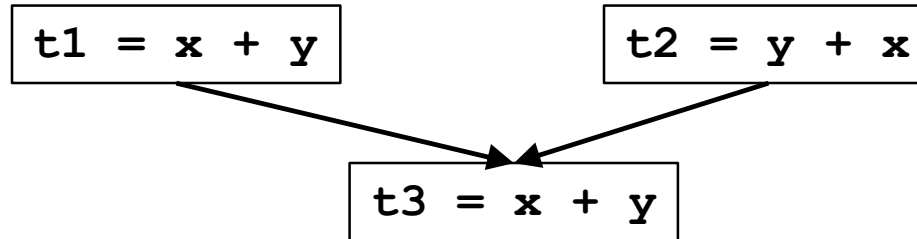


Example Revisited: Eliminating the CSE



Limitation: Textually Identical Expressions

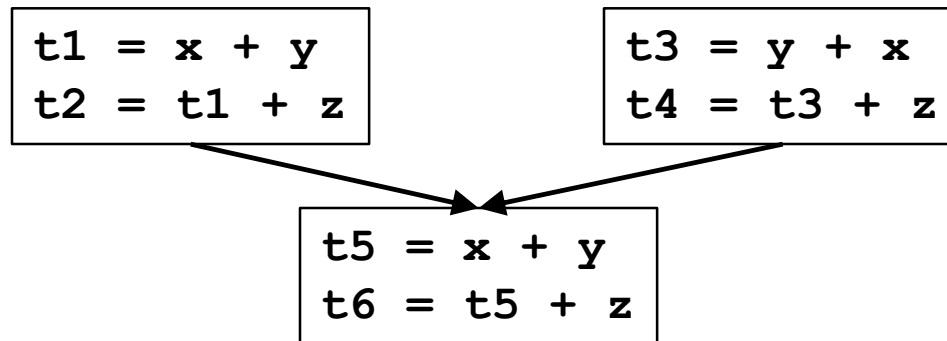
- **Commutative operations**



- Won't detect $x + y$ as an available expression
- Solution: Sort the operands

Further Improvements

- **Examples**
 - Expressions with more than two operands



- Textually different expressions may be equivalent

```
t1 = x + y
if t1 > y goto L1
z = x
t2 = z + y
```

After copy propagation:

```
t2 = x + y
```

Solution: Use multiple passes of GCSE combined with copy propagation

Summary

	Reaching Definitions	Available Expressions
Domain	Sets of definitions	Sets of expressions
Direction	forward: $\text{out}[b] = f_b(\text{in}[b])$ $\text{in}[b] = \wedge \text{out}[\text{pred}(b)]$	forward: $\text{out}[b] = f_b(\text{in}[b])$ $\text{in}[b] = \wedge \text{out}[\text{pred}(b)]$
Transfer function	$f_b(x) = \text{Gen}_b \cup (x - \text{Kill}_b)$	$f_b(x) = \text{Gen}_b \cup (x - \text{Kill}_b)$
Meet Operation ($\wedge$)	$\cup$	$\cap$
Boundary Condition	$\text{out}[\text{entry}] = \emptyset$	$\text{out}[\text{entry}] = \emptyset$
Initial interior points	$\text{out}[b] = T = \emptyset$	$\text{out}[b] = T = \text{all expressions}$

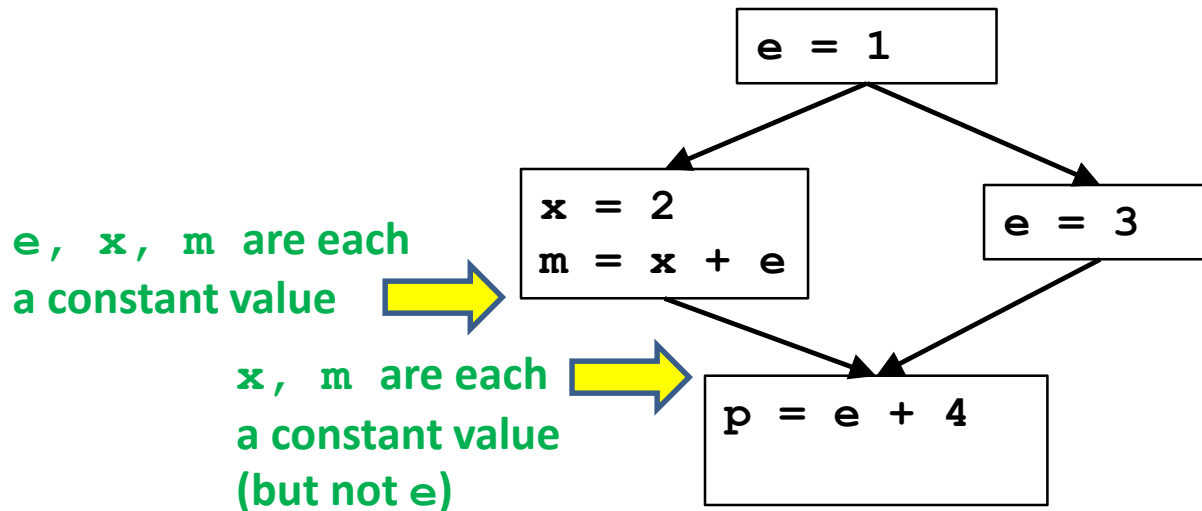
Available Expressions

Kill_b = all E such that block b defines a variable in E

Gen_b = all E such that block b evaluates E and doesn't later kill it

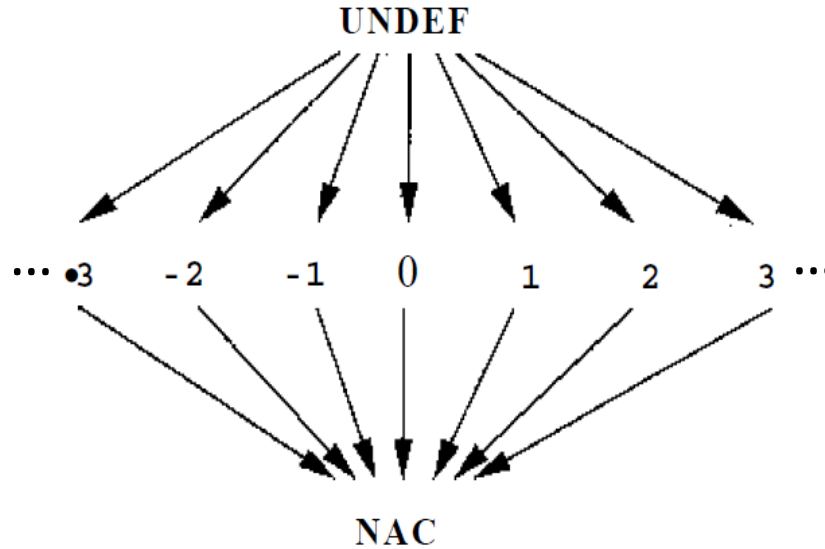
III. Constant Propagation/Folding

- At every basic block boundary, for each variable v
 - determine if v is a constant
 - if so, what is the value?



Which
variables are
constants?

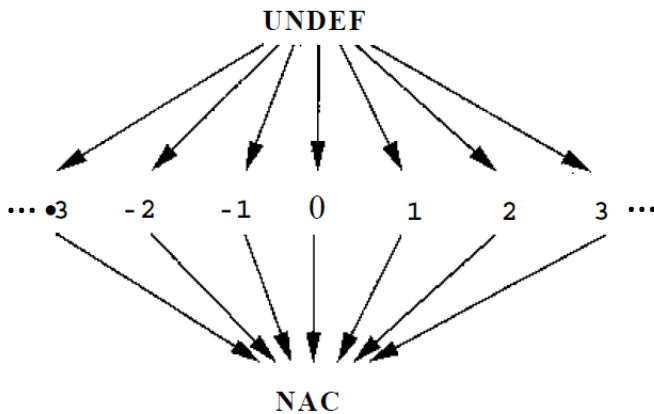
Semi-lattice Diagram



- Finite domain? No (unless bound number of bits)
- Finite height? Yes (2)
- One such lattice for each variable in the program

Meet Operation in Table Form

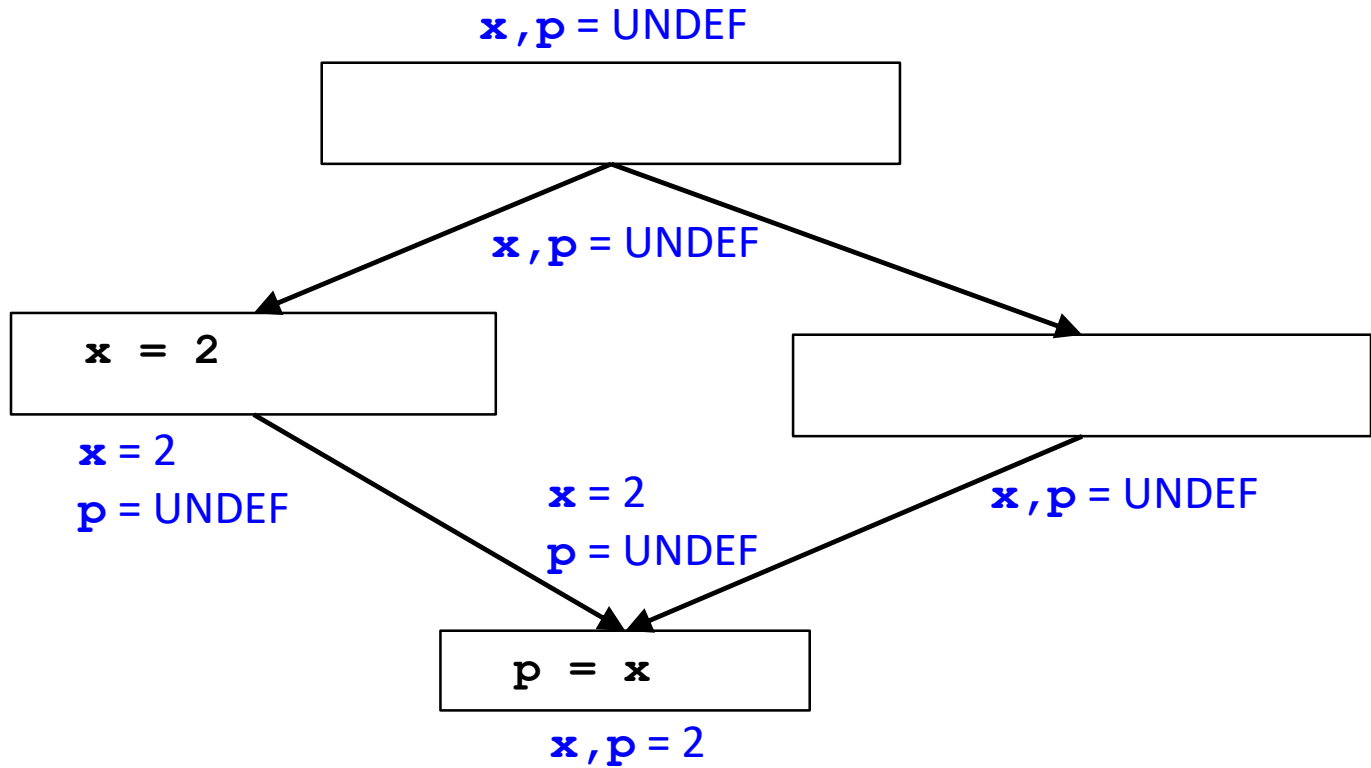
- Meet Operation:**



v_1	v_2	$v_1 \wedge v_2$
UNDEF	UNDEF	UNDEF
	c_2	c_2
	NAC	NAC
c_1	UNDEF	c_1
	c_2	c_1 , if $c_1 = c_2$ NAC otherwise
	NAC	NAC
NAC	UNDEF	NAC
	c_2	NAC
	NAC	NAC

– Note: $\text{UNDEF} \wedge c_2 = c_2$

Example



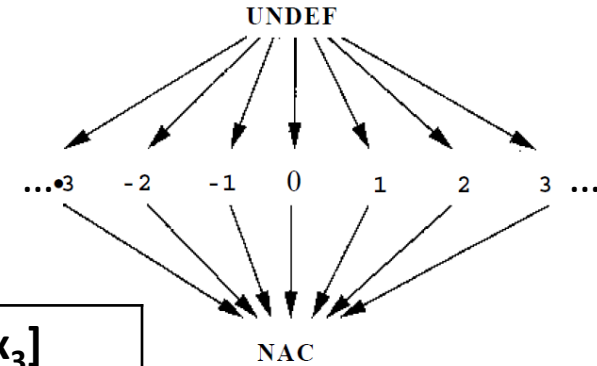
Transfer Function

- Assume a basic block has only 1 instruction
- Let $IN[b,x]$, $OUT[b,x]$
 - be the information for variable x at entry and exit of basic block b
- $OUT[entry, x] = \text{UNDEF}$, for all x .
- **Non-assignment** instructions: $OUT[b,x] = IN[b,x]$
- **Assignment** instructions: (next page)

Transfer Function (cont.)

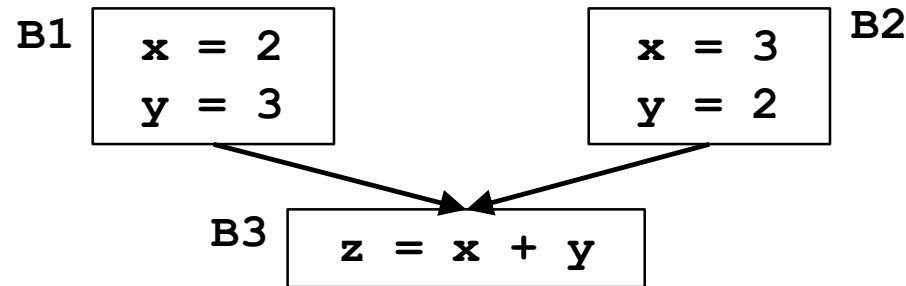
- Let an assignment be of the form $x_3 = x_1 + x_2$
 - “+” represents a generic operator
 - $OUT[b, x] = IN[b, x]$, if $x \neq x_3$

$IN[b, x_1]$	$IN[b, x_2]$	$OUT[b, x_3]$
UNDEF	UNDEF	UNDEF
	c_2	UNDEF
	NAC	NAC
c_1	UNDEF	UNDEF
	c_2	$c_1 + c_2$
	NAC	NAC
NAC	UNDEF	NAC
	c_2	NAC
	NAC	NAC



- Use:** $x \leq y$ implies $f(x) \leq f(y)$ to check if framework is monotone
 - $[v_1 \ v_2 \ \dots] \leq [v'_1 \ v'_2 \ \dots]$, $f([v_1 \ v_2 \ \dots]) \leq f([v'_1 \ v'_2 \ \dots])$

Not Distributive



	x	y	z
$f_1(T)$	2	3	UNDEF
$f_2(T)$	3	2	UNDEF
$f_1(T) \wedge f_2(T)$	NAC	NAC	UNDEF
$f_3(f_1(T) \wedge f_2(T))$	NAC	NAC	NAC
$f_3(f_1(T))$	2	3	5
$f_3(f_2(T))$	3	2	5
$f_3(f_1(T)) \wedge f_3(f_2(T))$	NAC	NAC	5

- Not Distributive: $f_3(f_1(T) \wedge f_2(T)) < f_3(f_1(T)) \wedge f_3(f_2(T))$
- Iterative solution is not precise. It is not wrong. It is conservative.

Summary of Constant Propagation

- **A useful optimization**
- **Illustrates:**
 - abstract execution
 - an infinite semi-lattice
 - a non-distributive problem
 - a problem where cycles can add information

Lecture 8:

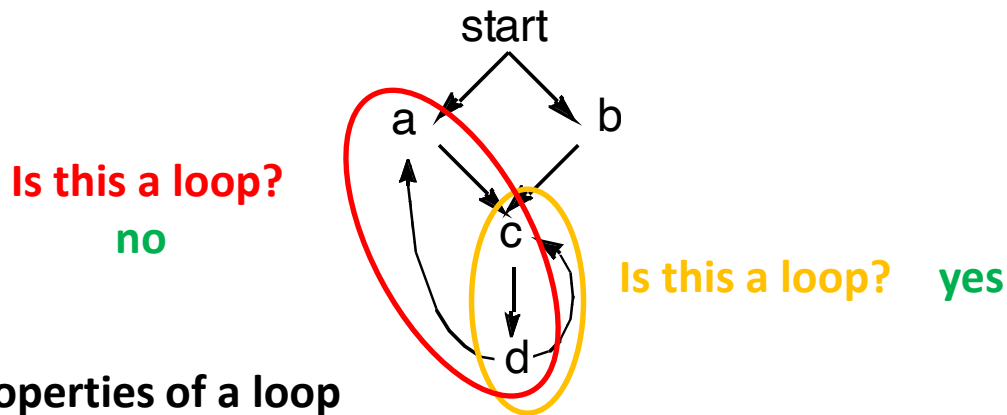
Induction Variable Optimizations

- I. Finding loops
- II. Overview of Induction Variable Optimizations
- III. Further details

ALSU 9.1.8, 9.6, 9.8.1

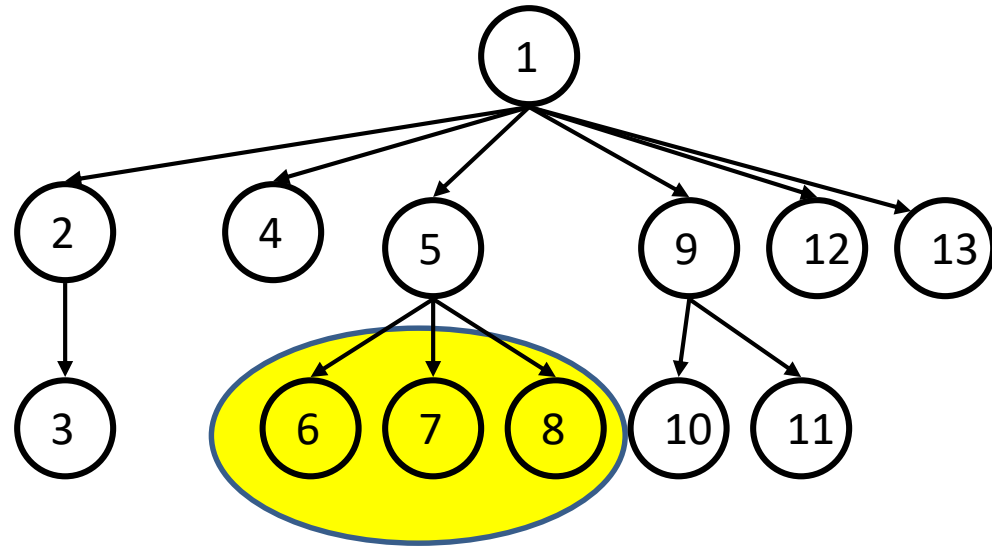
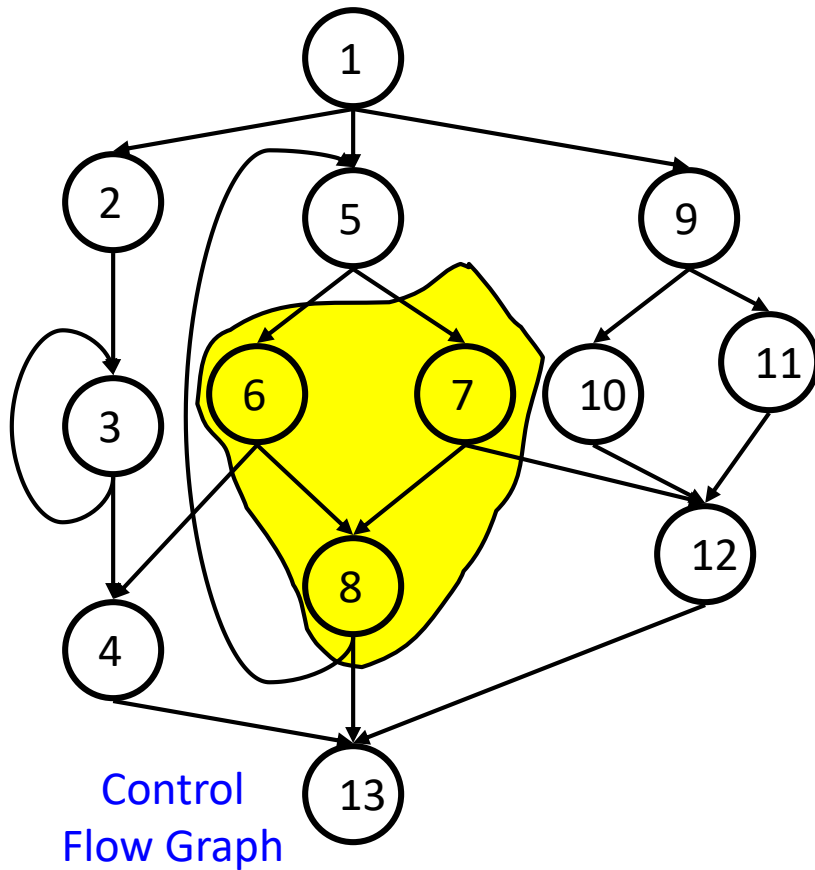
What is a Loop?

- **Goals:**
 - Define a loop in graph-theoretic terms (control flow graph)
 - Independent of specific programming language constructs used
 - A uniform treatment for all loops: DO, while, for, goto's
- **Not every cycle is a “loop” from an optimization perspective**



- **Intuitive properties of a loop**
 - single entry point
 - edges must form at least a cycle
- **Loops can nest**

Important Concept: Dominance



Dominance Tree (D-Tree)
 $x \text{ sdom } w$ iff x is a proper ancestor of w

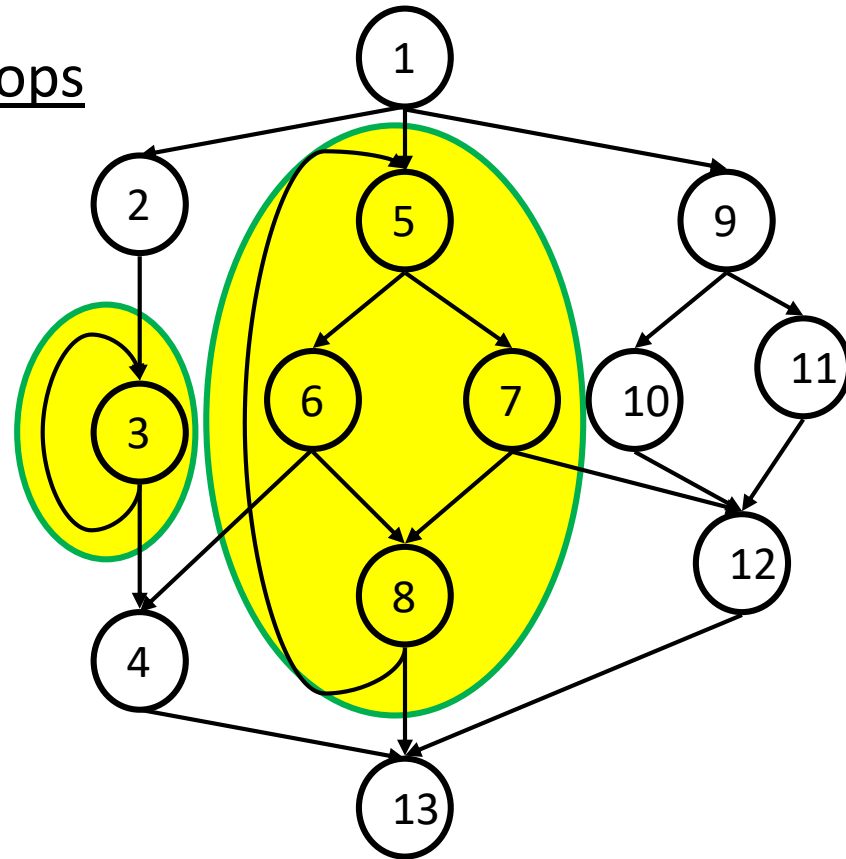
All paths to 6, 7, or 8 must visit 5 first

x strictly dominates w ($x \text{ sdom } w$) iff impossible to reach w without passing through x first

x dominates w ($x \text{ dom } w$) iff $x \text{ sdom } w$ OR $x = w$

Natural Loops

- Single entry-point: **header**
 - a header **dominates** all nodes in the loop
- A **back edge** is an arc $t \rightarrow h$ whose **head h dominates its tail t**
 - a back edge **must be a part of at least one loop**
- The **natural loop of a back edge** $t \rightarrow h$ is the **smallest set** of nodes that **includes t and h** , and has **no predecessors outside the set**, except for the predecessors of the header h .



What are the back edges?

3→3 and 8→5

What are the natural loops?

highlighted in yellow above

I. Algorithm to Find Natural Loops

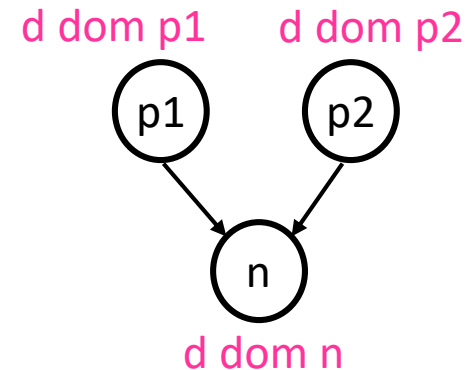
Step 1. Find the dominator relations in a flow graph

Step 2. Identify the back edges

Step 3. Find the natural loop associated with the back edge

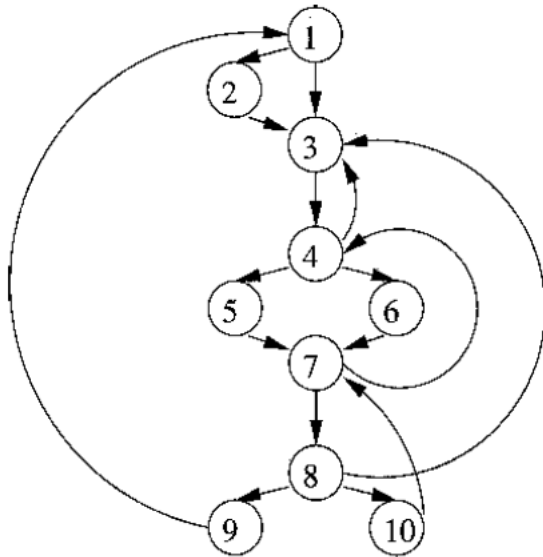
Step 1. Finding Dominators

- Node d dominates node n in a graph ($d \text{ dom } n$) if every path from the start node to n goes through d
- **Formulated as Data Flow Analysis problem:**
 - node d lies on all possible paths reaching node n iff $d \text{ dom } p$ for all pred p of n
 - Direction: forward
 - Values: basic blocks
 - Meet operator: $\cap$
 - Top (T): all basic blocks
 - Bottom: $\{\}$
 - Boundary condition for entry node: $\text{OUT}[\text{entry}] = \{\text{entry}\}$
 - Initialization for internal nodes $\text{OUT}[b] = T$
 - Finite descending chain? Yes (depth=number of basic blocks)
 - Transfer function: $f_b(x) = \{b\} \cup x$
 - Monotone & Distributive? Yes and yes: $(\{b\} \cup x) \cap (\{b\} \cup y) = \{b\} \cup (x \cap y)$
- **Speed:**
 - With rPostorder, most flow graphs (reducible flow graphs) converge in 1 pass

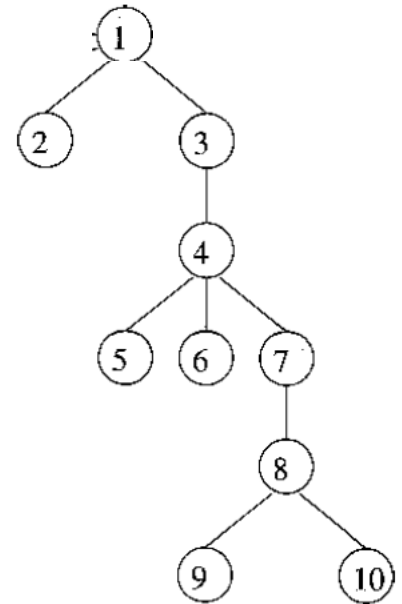


Example: Finding Dominators

$$\text{OUT}[b] = \{b\} \cup \left(\bigcap_{p=\text{pred}(b)} \text{OUT}[p] \right)$$



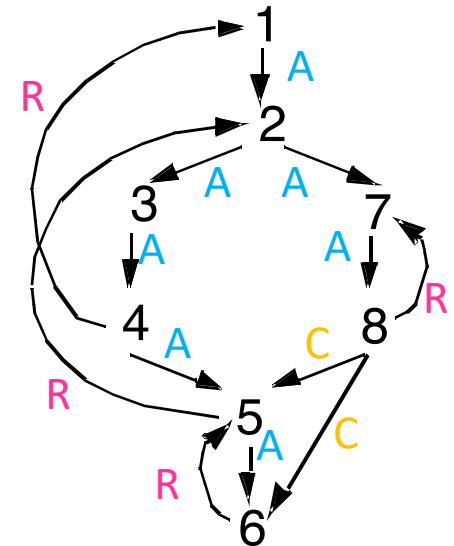
OUT[1] = {1}
OUT[2] = {1,2}
OUT[3] = {1,3}
OUT[4] = {1,3,4}
OUT[5] = {1,3,4,5}
OUT[6] = {1,3,4,6}
OUT[7] = {1,3,4,7}
OUT[8] = {1,3,4,7,8}
OUT[9] = {1,3,4,7,8,9}
OUT[10] = {1,3,4,7,8,10}
(No change in second iteration)



Step 2. Finding Back Edges

- **Depth-first spanning tree**

- Edges traversed in a depth-first search of the flow graph form a depth-first spanning tree



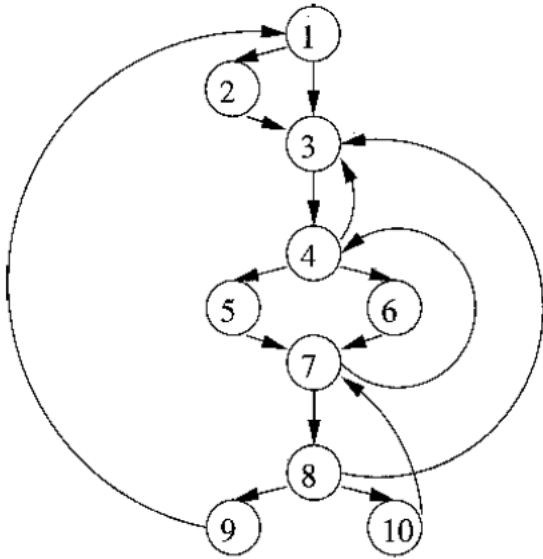
- **Categorizing edges in graph**

- **Advancing** edges (A): from ancestor to proper descendant
- **Cross** edges (C): from right to left
- **Retreating** edges (R): from descendant to ancestor (not necessarily proper)

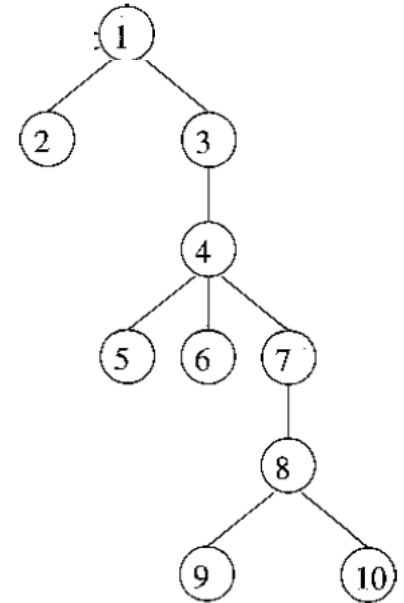
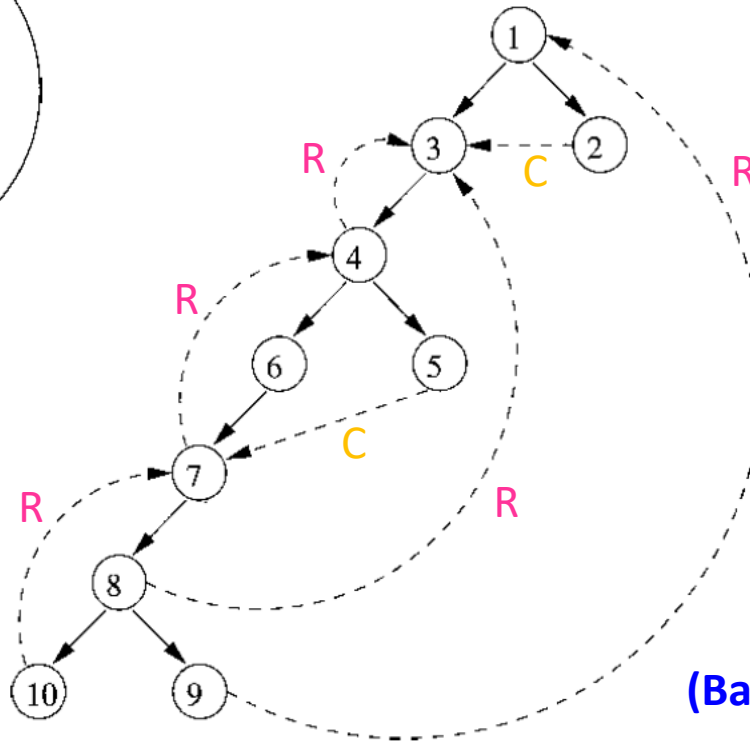
Back Edges

- **Definition**
 - **Back edge**: $t \rightarrow h$, h dominates t
- **Relationships between graph edges and back edges**
- **Algorithm**
 - Perform a depth first search
 - For each retreating edge $t \rightarrow h$, check if h is in t 's dominator list
- **Most programs (all structured code, and most GOTO programs) have **reducible** flow graphs**
 - retreating edges = back edges

Example: Cross Edges, Retreating Edges, Back Edges



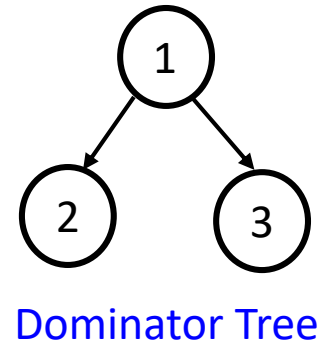
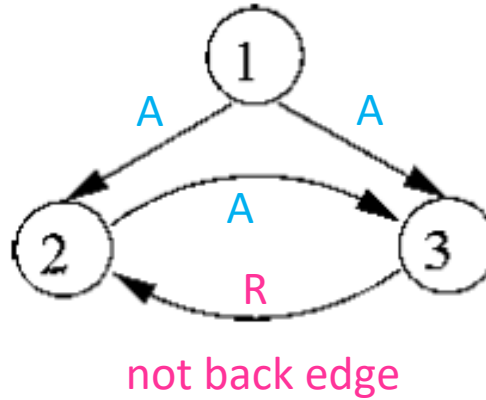
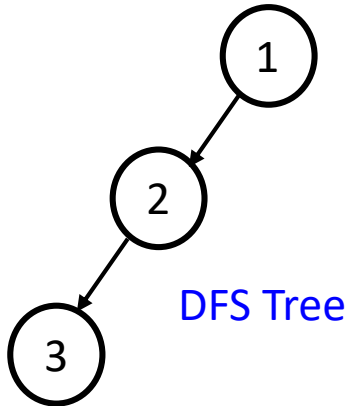
Which edges are:
cross edges?
retreating edges?
back edges?



All the retreating edges
are back edges

(Back edge: $t \rightarrow h$, h dominates t)

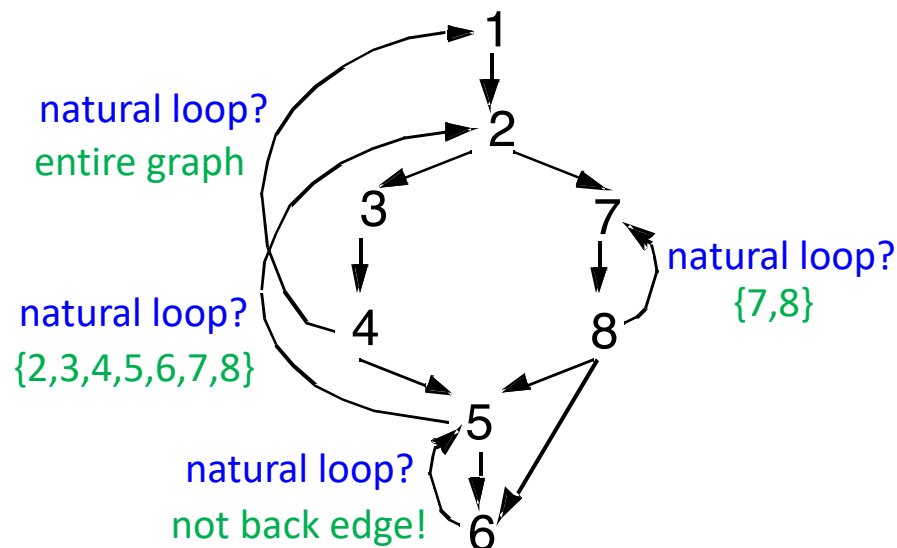
A Nonreducible Flow Graph



- **Categorizing edges in graph (relative to a DFS tree)**
 - Advancing edges (A): from ancestor to proper descendant
 - Cross edges (C): from right to left
 - Retreating edges (R): from descendant to ancestor (not necessarily proper)
 - Back edges: $t \rightarrow h$, h dominates t

Step 3. Constructing Natural Loops

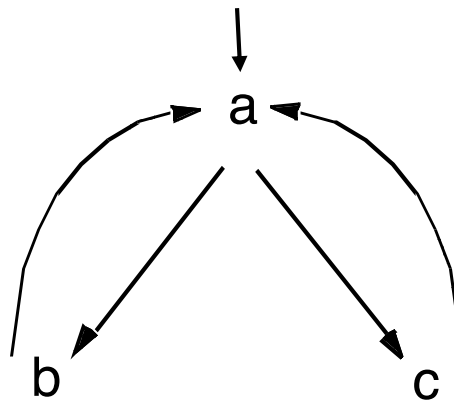
- The **natural loop of a back edge $t \rightarrow h$** is the smallest set of nodes that includes t and h , and has no predecessors outside the set, except for the predecessors of the header h .



- Algorithm: For each back edge $t \rightarrow h$:**
 - delete h from the flow graph
 - find those nodes that can reach t
(those nodes plus h form the natural loop of $t \rightarrow h$)

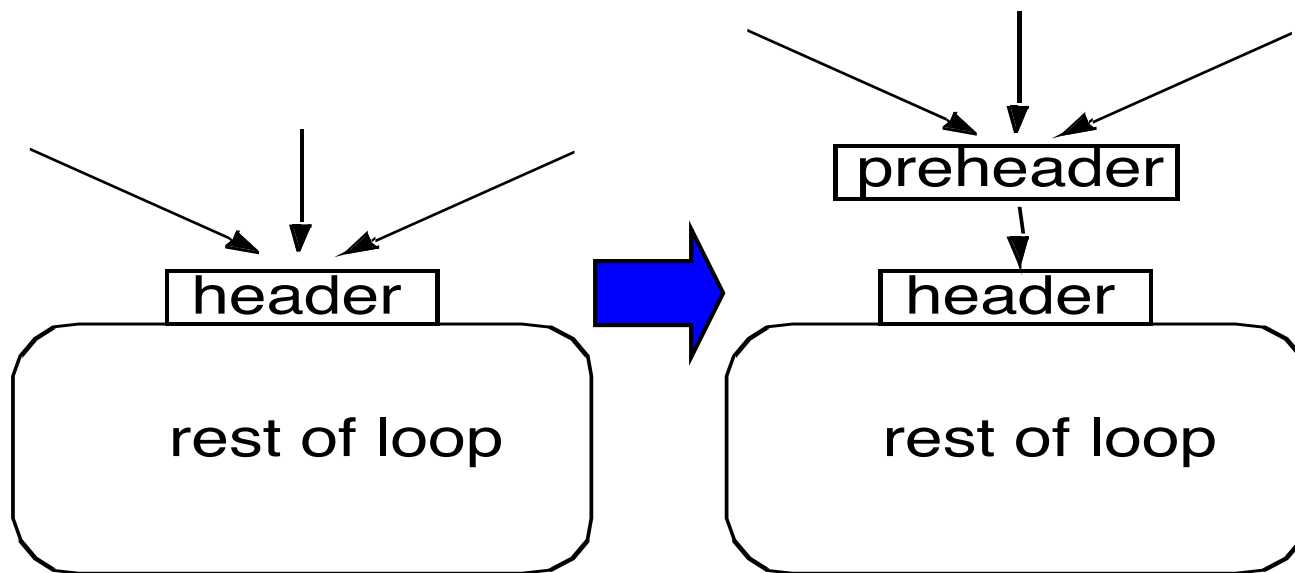
Inner Loops

- **If two loops do not have the same header:**
 - they are either disjoint, or
 - one is entirely contained (nested within) the other
 - inner loop: one that contains no other loop.
- **If two loops share the same header:**
 - Hard to tell which is the inner loop
 - Solution: Combine and treat as one loop



Preheader

- Optimizations often emit code that is to be executed once before the loop
- **Solution:** Create a preheader basic block for every loop



Finding Loops: Summary

- **Define loops in graph theoretic terms**
- **Definitions and algorithms for:**
 - Dominators
 - Back edges
 - Natural loops

II. Overview of Induction Variable Elimination

Example

```
for(i=0; i<100; i++)  
    A[i] = 0;
```

Induction variables:

$t1 = 4i$

$t2 = 4i + \&A$

```
i = 0  
L2: IF i >= 100 GOTO L1  
t1 = 4 * i  
t2 = &A + t1  
*t2 = 0  
i = i + 1  
  
GOTO L2  
L1:
```

original code
(A[i] is 4 bytes)

```
t1' = 0  
t2' = &A  
IF t1' >= 400  
t1 = t1'  
t2 = t2'  
  
t1' = t1' + 4  
t2' = t2' + 4
```

after induction
variable substitution

```
t2' = &A  
t3' = &A + 400  
L2: IF t2' >= t3' GOTO L1  
*t2' = 0  
t2' = t2' + 4  
GOTO L2  
L1:
```

final code

Definitions

```
i = 0
L2: IF i>=100...
    t1 = 4 * i
    t2 = &A + t1
    *t2 = 0
    i = i+1
    GOTO L2
```

- A **basic induction variable** is
 - a variable X whose only definitions within the loop are assignments of the form:
$$X = X + c \text{ or } X = X - c,$$
where c is either a **constant** or a **loop-invariant variable**. (e.g., i)
- An **induction variable** is
 - a **basic induction variable** B , or
 - a variable **defined once** within the loop, whose value is a **linear function of some basic induction variable** at the time of the definition:
$$A = c_1 * B + c_2 \quad (\text{e.g., } t1, t2)$$
- The **FAMILY of a basic induction variable** B is
 - the set of induction variables A such that each time A is assigned in the loop, the value of A is a linear function of B . (e.g., $t1, t2$ is in family of i)

Optimizations

1. Strength reduction:

– A is an induction variable in family of basic induction variable B (i.e., $A = c_1 * B + c_2$)

- Create new variable:
- Initialize in preheader:
- Track value of B :
- Replace assignment to A :

A'

$A' = c_1 * B + c_2$

add after $B = B + x$: $A' = A' + x * c_1$

replace lone $A = \dots$ with $A = A'$

```
i = 0
L2: IF i >= 100 GOTO L1
t1 = 4 * i
t2 = &A + t1
*t2 = 0
i = i + 1

GOTO L2
```

$t1' = 0$

$t2' = \&A$

$t1 = t1'$

$t2 = t2'$

$t1' = t1' + 4$

$t2' = t2' + 4$

Induction variables:

$t1 = 4 * i$

$t2 = 4 * i + \&A$

Optimizations (continued)

2. Optimizing **non-basic** induction variables

- copy propagation
- dead code elimination

3. Optimizing **basic** induction variables

- Eliminate basic induction variables used only for
 - calculating other induction variables and loop tests
- Algorithm:
 - Select an **induction variable A in the family of B**, preferably with simple constants ($A = c_1 * B + c_2$).
 - Replace a comparison such as

```
if B > X goto L1
```

with

```
if (A' > c1 * X + c2) goto L1
```

 (assuming c_1 is positive)
 - if B is live at any exit from the loop, **recompute it from A'**
 - After the exit, $B = (A' - c_2) / c_1$

Example Continued

```
for(i=0; i<100; i++)  
    A[i] = 0;
```

Induction variables:

$t1 = 4i$

$t2 = 4i + \&A$

```
i = 0  
L2: IF i >= 100 GOTO L1  
t1 = 4 * i  
t2 = &A + t1  
*t2 = 0  
i = i + 1  
  
GOTO L2  
L1:
```

```
t1' = 0  
t2' = &A  
IF t2' >= &A + 400  
t1 = t1'  
t2 = t2'  
*t2' = 0  
t1' = t1' + 4  
t2' = t2' + 4
```

```
t2' = &A  
t3' = &A + 400  
L2: IF t2' >= t3' GOTO L1  
*t2' = 0  
t2' = t2' + 4  
GOTO L2  
L1:
```

$$B \geq X \Rightarrow A' \geq c_1 * X + c_2$$

III. Further Details

- **A BASIC induction variable in a loop L**
 - a variable X whose **only definitions within L** are assignments of the form:
 $X = X + c$ or $X = X - c$, where c is either a constant or a loop-invariant variable.

- **Algorithm: can be detected by scanning L**

- **Example:**

```
k = 0;
for (i = 0; i < n; i++) {
    k = k + 3;
    ... = m;
    if (x < y)
        k = k + 4;
    if (a < b)
        m = 2 * k;
    k = k - 2;
    ... = m;
}
```

Basic induction variable(s)? i, k

Additional induction variable(s)?
 $m = 2k + 0$ (in family of k)

Each iteration may execute a different number of increments/decrements!!

Strength Reduction Algorithm

- **Key idea:**
 - For each induction variable A , ($A = c_1 * B + c_2$ at time of definition)
 - variable A' holds expression $c_1 * B + c_2$ at all times
 - replace definition of A with $A = A'$ only when executed
(m is only updated when appropriate)
- **Result:**
 - Program is correct
 - Definition of A does not need to refer to B

Finding Induction Variable Families

- **Let B be a basic induction variable**
 - Find all induction variables A in family of B:
 - $A = c_1 * B + c_2$
(where B refers to the value of B at time of definition)
- **Conditions:**
 - C1: If A has a single assignment in the loop L, and assignment is one of:

$$A = B * c$$

$$A = c * B \quad (\text{e.g., } m)$$

$$A = B / c \quad (\text{assuming } A \text{ is real})$$

$$A = B + c$$

$$A = c + B$$

$$A = B - c$$

$$A = c - B$$

- OR, ... (next page)

Finding Induction Variable Families (continued)

- C2: Let D be an induction variable in the family of B ($D = c_1 * B + c_2$)

Rule 1: If A has a single assignment in the loop L, and assignment is one of:

$$A = D * c$$

$$A = c * D$$

$$A = D / c \quad (\text{assuming } A \text{ is real})$$

$$A = D + c$$

$$A = c + D$$

$$A = D - c$$

$$A = c - D$$

Rule 2: No definition of D outside L reaches the assignment to A

Rule 3: Every path between the lone point of assignment to D in L and the assignment to A has the same sequence (possibly empty) of definitions of B

Induction Variable Family Example 1

```
L2: IF i >= 100 GOTO L1
    t2 = t1 + 10
    t1 = 4 * i
    t3 = t1 * 8
    i = i + 1
    goto L2
L1:
```

Is **i** a basic induction variable? yes

Is **t2** in family of **i**? no (fails Rule 2)

Is **t1** in family of **i**? yes (by C1)

Is **t3** in family of **i**? yes (by C2 with
A:t3, D:t1, B:i)

Condition C1

A has a single assignment in the loop L of the form $A = B * c$, $c * B$, $B + c$, etc

Condition C2

A is in family of B if $D = c_1 * B + c_2$ for basic induction variable B and:

- Rule 1: A has a single assignment in the loop L of the form $A = D * c$, $D + c$, etc
- Rule 2: No definition of D outside L reaches the assignment to A
- Rule 3: Every path between the lone point of assignment to D in L and the assignment to A has the same sequence (possibly empty) of definitions of B

Induction Variable Family Example 2

```
L3: IF i >= 100 GOTO L1
    t1 = 4 * i
    IF t1 < 50 GOTO L2
    i = i + 2
L2: t2 = t1 + 10
    i = i + 1
    goto L3
```

L1:

Is **i** a basic induction variable? yes (all are $i = i + c$)

Is **t1** in family of **i**? yes (by C1)

Is **t2** in family of **i**? no (fails Rule 3)

Condition C1

A has a single assignment in the loop L of the form $A = B * c$, $c * B$, $B + c$, etc

Condition C2

A is in family of B if $D = c_1 * B + c_2$ for basic induction variable B and:

- Rule 1: A has a single assignment in the loop L of the form $A = D * c$, $D + c$, etc
- Rule 2: No definition of D outside L reaches the assignment to A
- Rule 3: Every path between the lone point of assignment to D in L and the assignment to A has the same sequence (possibly empty) of definitions of B

Induction Variables Summary

- **Precise definitions of induction variables**
- **Systematic identification of induction variables**
- **Strength reduction**
- **Clean up:**
 - eliminating basic induction variables
 - used in other induction variable calculations
 - replacement of loop tests
 - eliminating other induction variables
 - standard optimizations

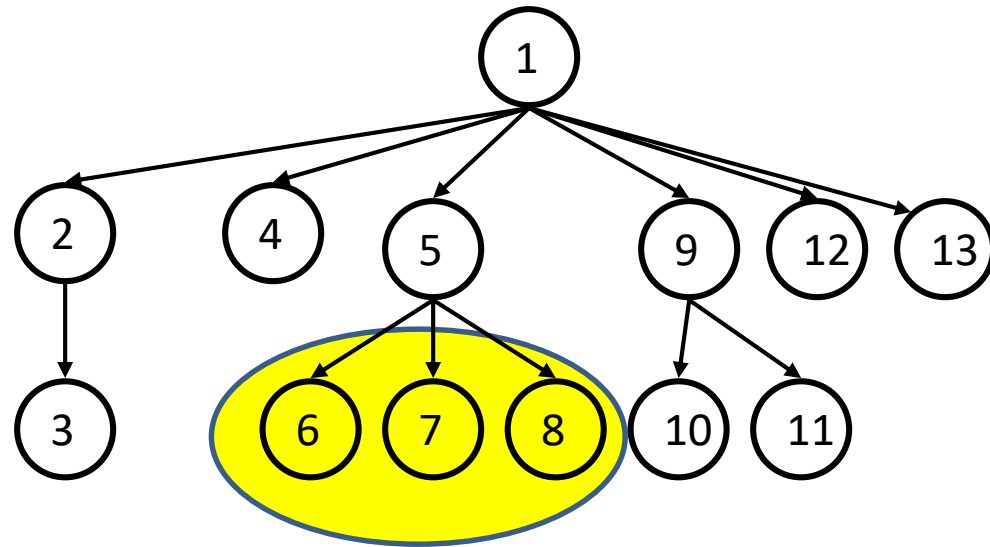
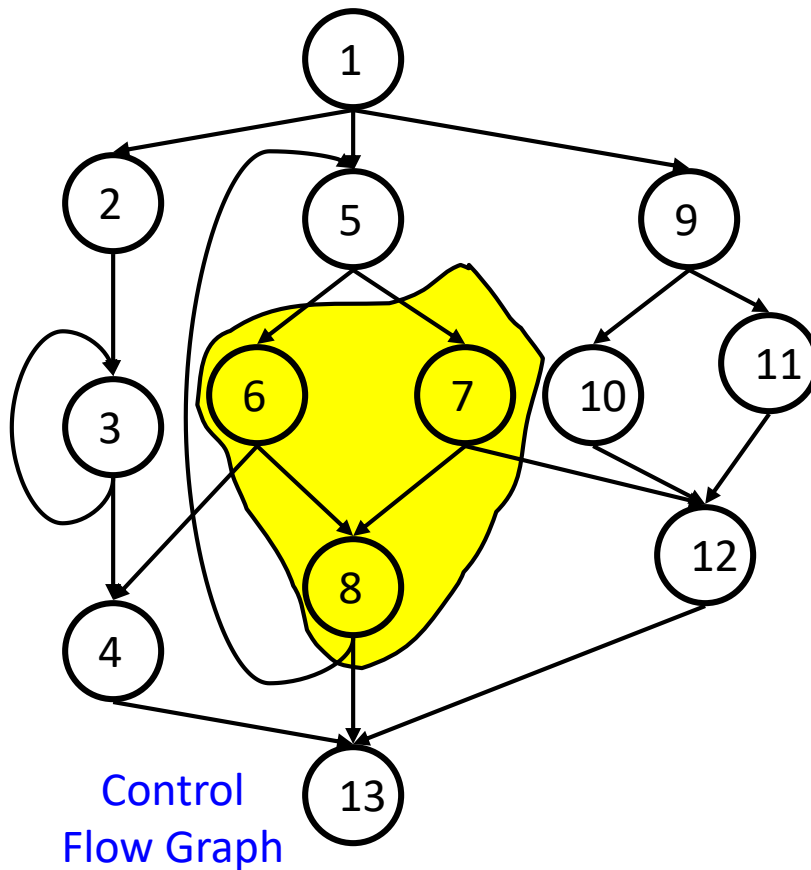
Lecture 9:

Loop Invariant Computation and Code Motion

- I. Loop-invariant computation
- II. Algorithm for code motion
- III. Partial redundancy elimination

ALSU 9.5-9.5.2

Review: Dominators



x **sdom** w iff x is a proper ancestor of w

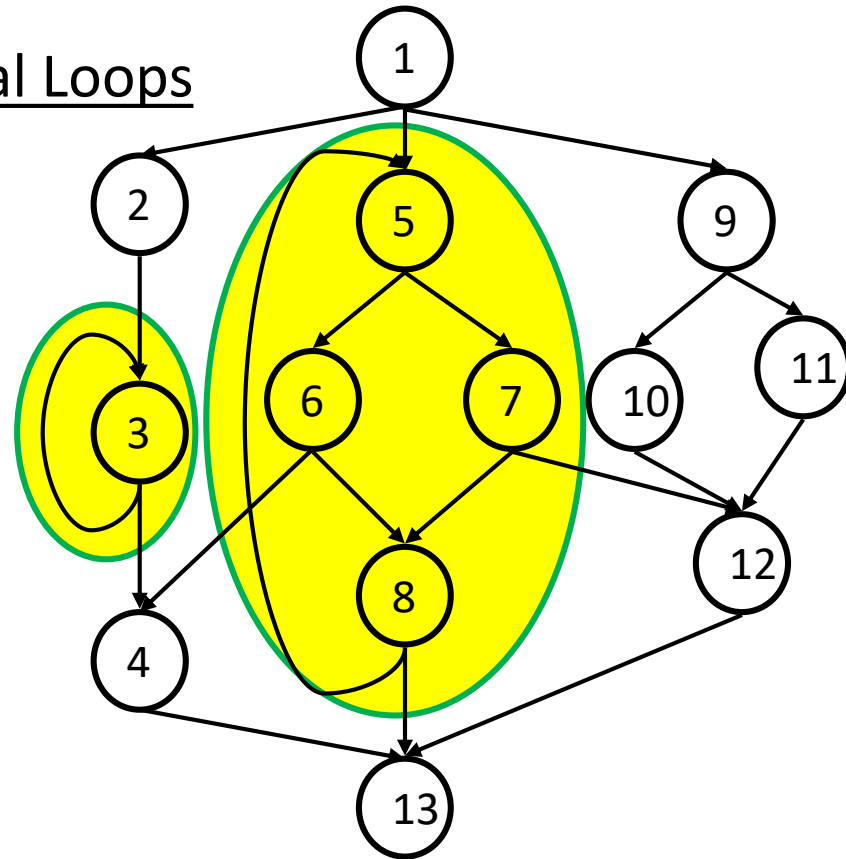
All paths to 6, 7, or 8 must visit 5 first

x **strictly dominates** w (x **sdom** w) iff impossible to reach w without passing through x first

x **dominates** w (x **dom** w) iff x **sdom** w OR $x = w$

Review: Natural Loops

- Single entry-point: **header**
 - a header **dominates all nodes in the loop**
- A **back edge** is an arc $t \rightarrow h$ whose **head h dominates its tail t**
 - a back edge **must be a part of at least one loop**
- The **natural loop of a back edge** $t \rightarrow h$ is the **smallest set** of nodes that **includes t and h** , and has **no predecessors outside the set**, except for the predecessors of the header h .



What are the back edges?

3-→3 and 8-→5

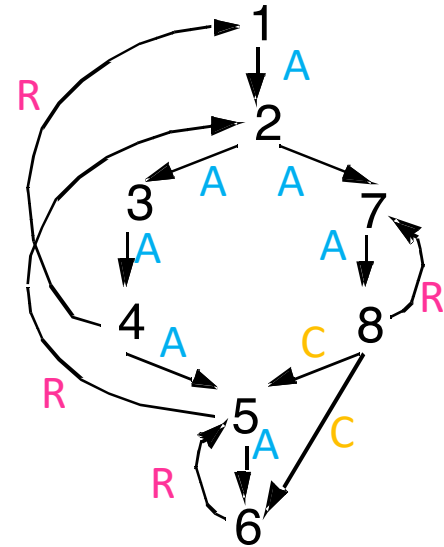
What are the natural loops?

highlighted in yellow above

Recall: Finding Back Edges

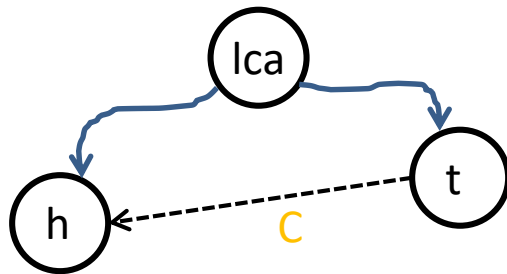
1. Construct a depth-first spanning tree of the CFG

- Edges traversed in a depth-first search of the CFG form a depth-first spanning tree
- Advancing edges (A): from ancestor to proper descendant
- Cross edges (C): from right to left
- Retreating edges (R): from descendant to ancestor



2. Determine which Retreating edges are Back edges ($t \rightarrow h$, h dominates t)

- Note: h can never dominate t for an advancing or cross edge $t \rightarrow h$

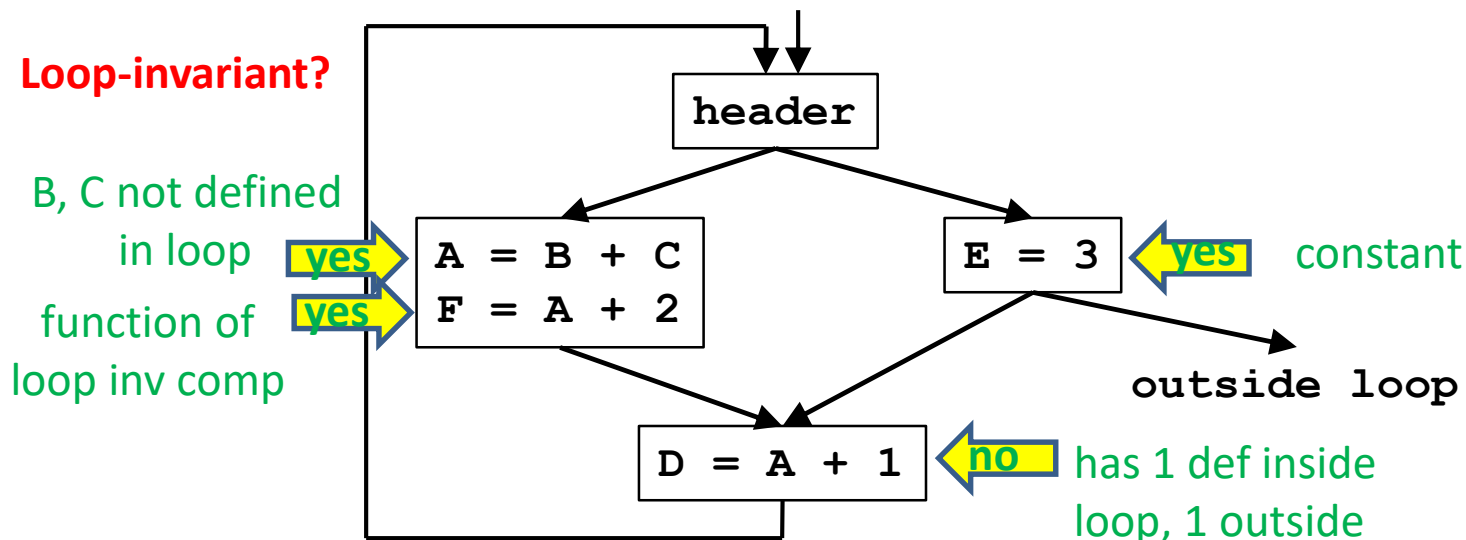


- Cross edge: t is not ancestor/descendant of h
- Thus, there is a least common ancestor, lca , of h and t in the tree
- Thus, $entry \rightarrow lca \rightarrow t$ is a path without h

- Could apply step 2 to all edges, skipping step 1---but `rPostOrder` uses step 1

I. Loop-Invariant Computation and Code Motion

- **A loop-invariant computation:**
 - a computation whose value does not change as long as control stays within the loop
- **Code motion:**
 - to move a statement within a loop to the preheader of the loop



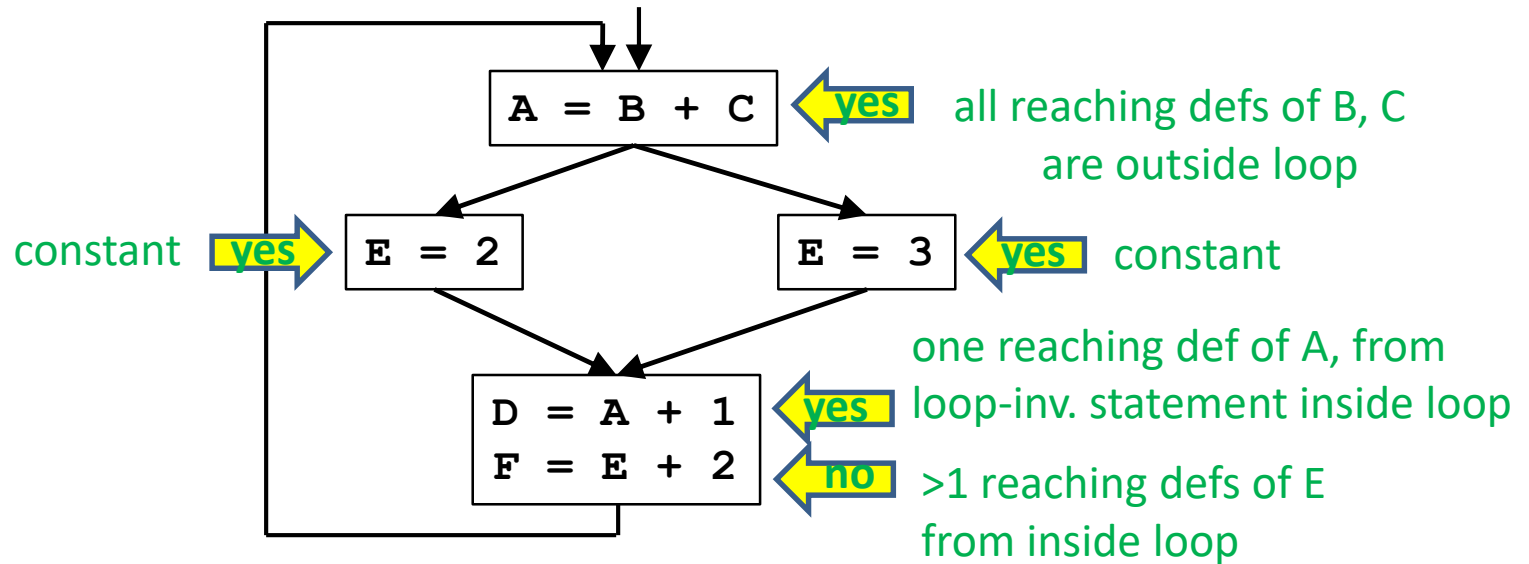
Algorithm

- **Observations**
 - Loop invariant
 - operands are defined outside loop or invariant themselves
 - Code motion
 - not all loop invariant instructions can be moved to preheader
- **Algorithm**
 - Find invariant expressions
 - Conditions for code motion
 - Code transformation

Algorithm: Detecting Loop Invariant Computation

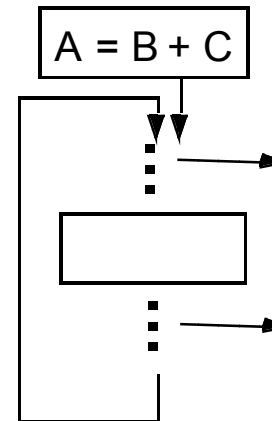
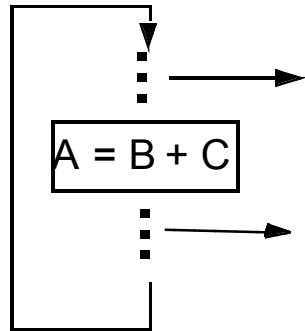
- Compute reaching definitions
- Mark INVARIANT if
all the definitions of B and C that reach a statement $A=B+C$
are outside the loop
 - What about a constant B, C? **invariant**
- Repeat: Mark INVARIANT if
 - (all reaching definitions of B are outside the loop OR
there is exactly one reaching definition for B and it is from a loop-invariant
statement inside the loop)
 - AND (similarly for C)until no changes to the set of loop-invariant statements occur.

Which Statements are Loop Invariant?



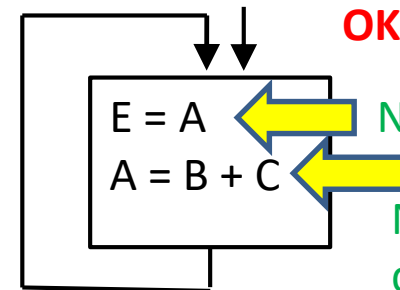
II. Conditions for Code Motion

- **Correctness:** Movement does not change semantics of program
- **Performance:** Code is not slowed down



OK to move?
No (moved past exit)

- **Basic idea:** defines once and for all
 - control flow: once?
 - Code dominates all exits
 - other definitions: for all?
 - No other definition
 - uses of the definition: for all?
 - Dominates use or no other reaching defs to use



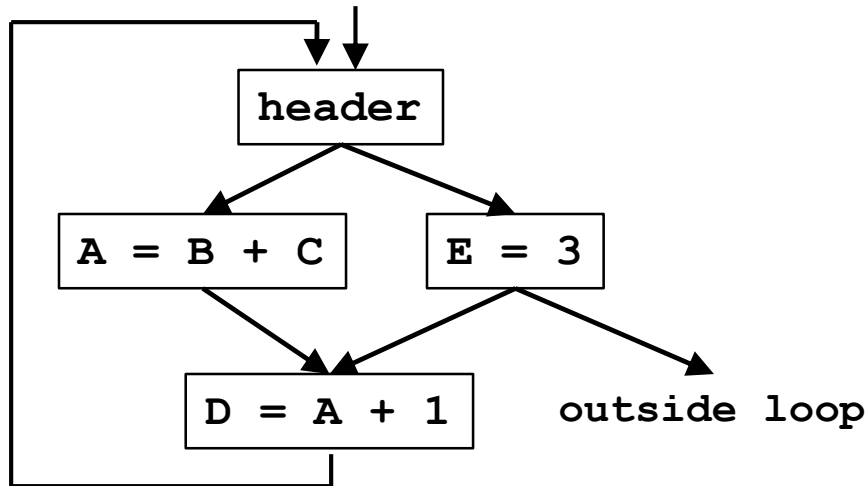
OK to move?
Not loop inv.
No (doesn't dominate use)

Code Motion Algorithm

Given: a set of nodes in a loop

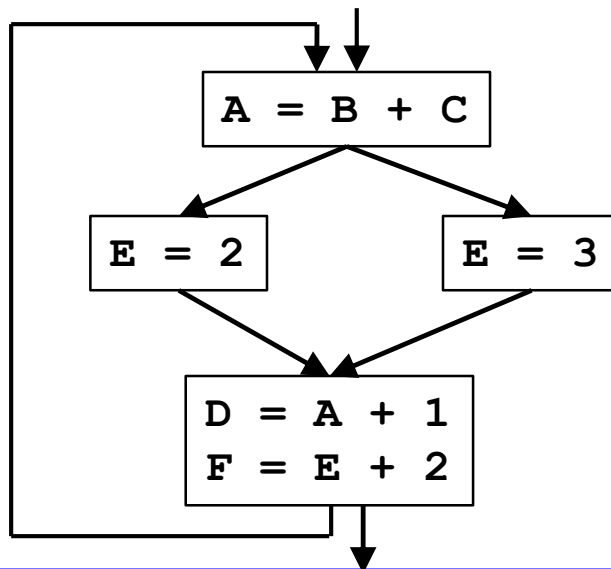
- **Compute reaching definitions**
- **Compute loop invariant computation**
- **Compute dominators**
- **Find the exits of the loop (i.e. nodes with successor outside loop)**
- **Candidate statement for code motion:**
 - loop invariant
 - in blocks that dominate all the exits of the loop
 - assign to variable not assigned to elsewhere in the loop
 - in blocks that dominate all blocks in the loop that use the variable assigned
- **Perform a depth-first search of the blocks**
 - Move the candidate to the preheader if
all the invariant operations it depends upon have been moved

Code Motion Examples



Which statements can be moved to loop preheader?

Only E=3: only statement dominating all exits



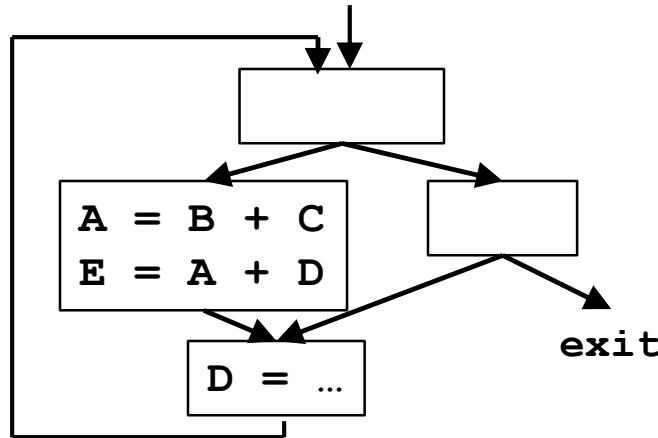
A=B+C
D=A+1

(Although E=2, E=3 are invariant, neither is only def of E)

defines once and for all

More Aggressive Optimizations

- **Gamble on: most loops get executed**
 - When can we relax constraint of dominating all exits?



Can relax if destination not live after loop
& can compute in preheader
w/o causing an exception

- **Landing pads**

```
While p do loop-body      ➔      if p {
                                preheader
                                repeat
                                    loop-body
                                until not p;
                                }
```

Ensures preheader
executes only
if enter loop

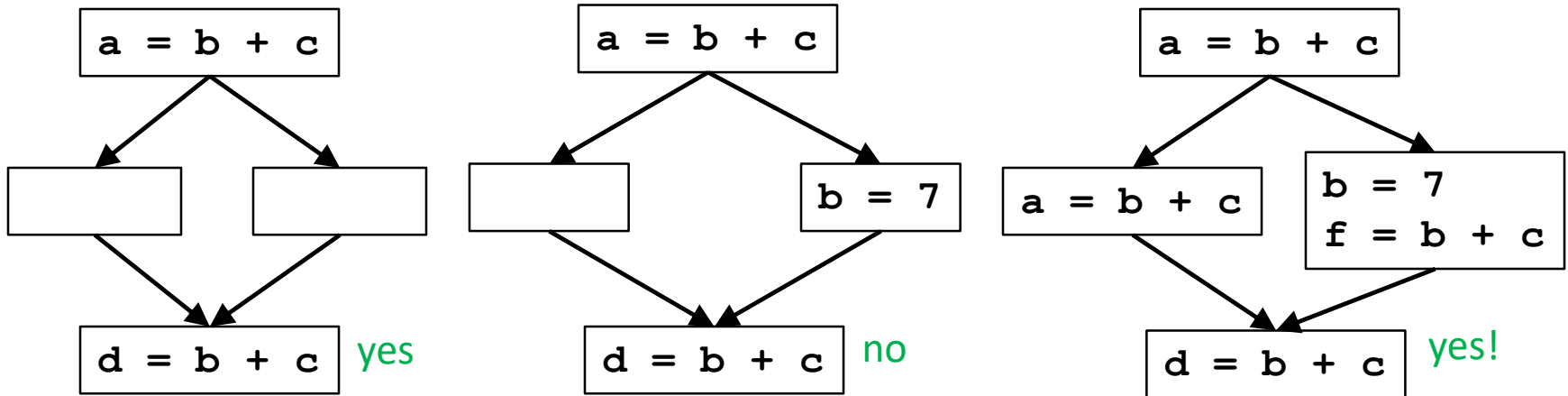
LICM Summary

- **Precise definition and algorithm for loop invariant computation**
- **Precise algorithm for code motion**
- **Use of reaching definitions and dominators in optimizations**

III. Partial Redundancy Elimination

- Sources of Redundancy
 - Global common subexpressions
 - Loop-invariant expressions
 - Partially redundant expressions

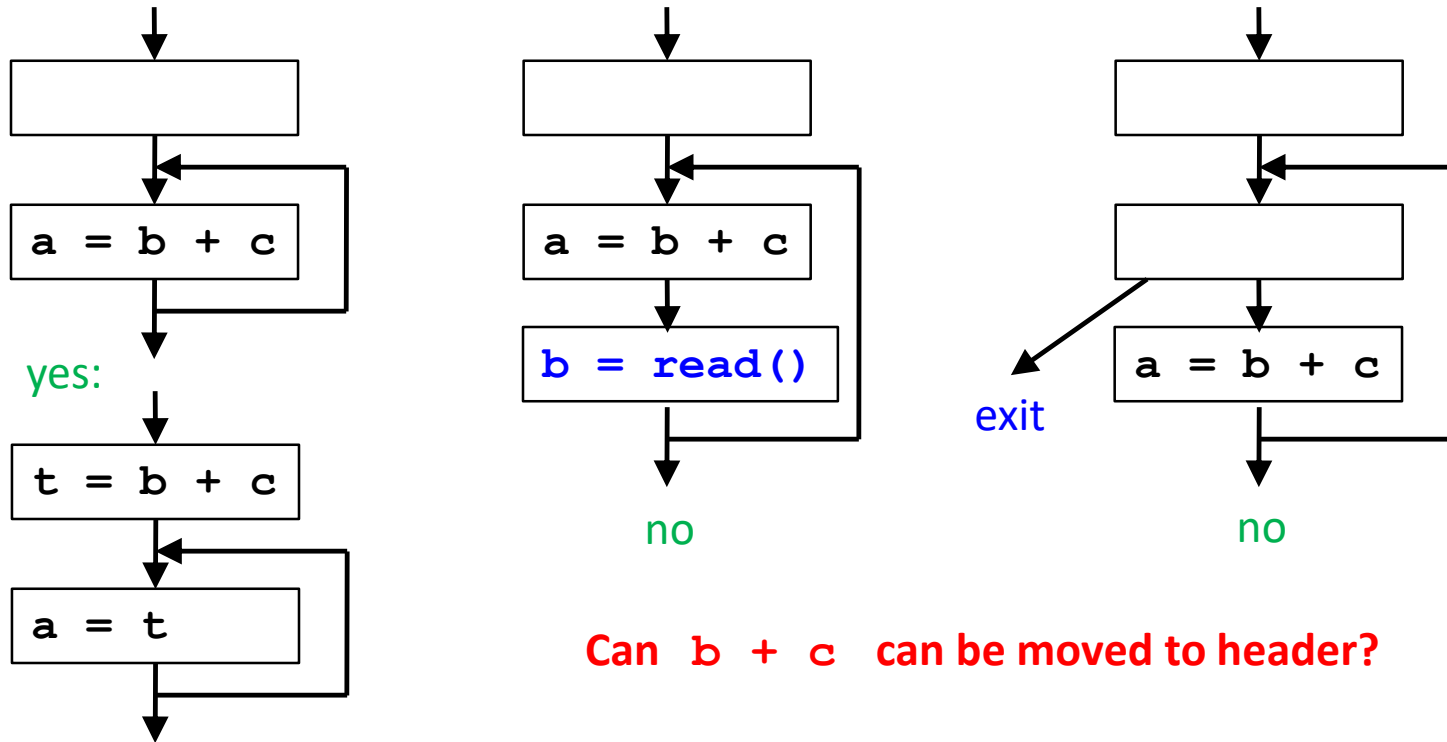
Recall: Global Common Subexpression Elimination



Which $b + c$ in bottom row is a common subexpression?

- On every path reaching p ,
 - expression $b+c$ has been computed
 - b, c not overwritten after the expression
- A common expression may have different values on different paths!

Loop Invariant Code Motion

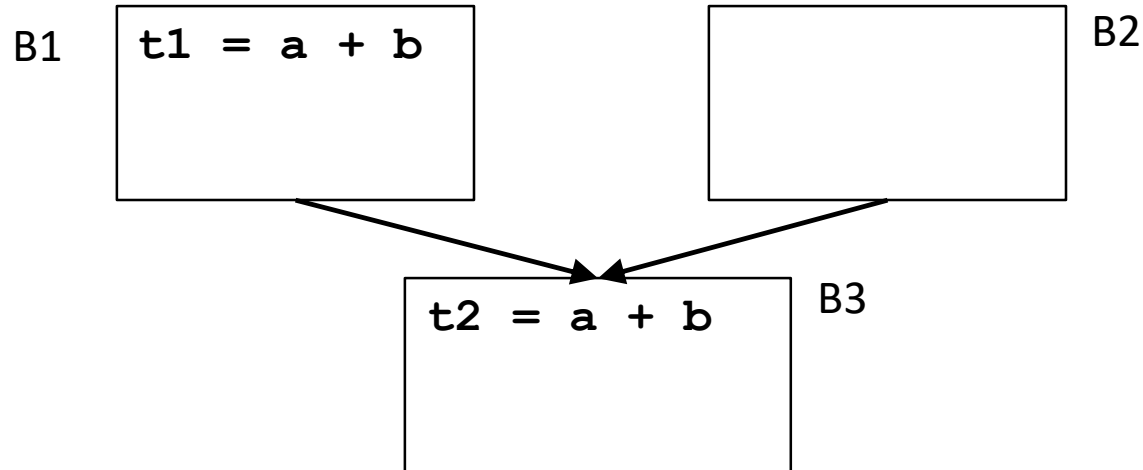


Can `b + c` can be moved to header?

- Given an expression $(b+c)$ inside a loop,
 - does the value of $b+c$ change inside the loop?
 - is the code executed at least once?

Partial Redundancy

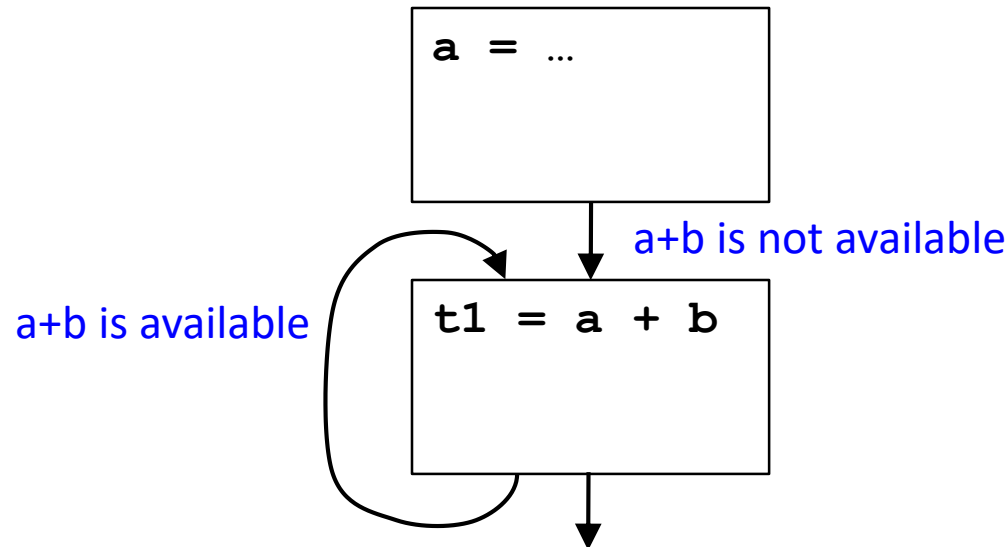
- Partially Redundant Computation



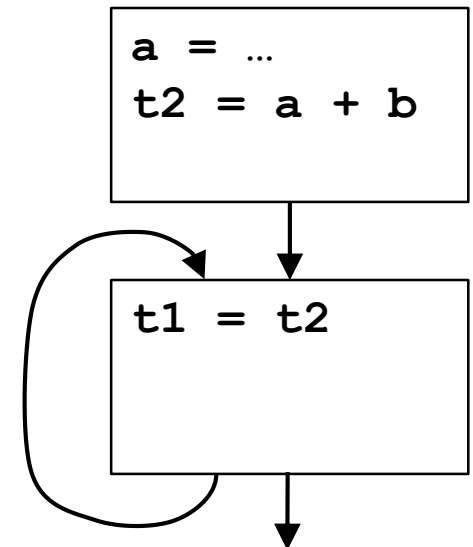
- Occurrence of expression E at P is **partially redundant** if E is **partially available** there:
 - E is evaluated along **at least one path** to P, with no operands redefined since.
- Partially redundant expression **can be eliminated** if we can **insert computations** to make it **fully redundant**.
 - E.g., insert `t1 = a + b` in B2

Loop Invariants are Partial Redundancies

- Loop invariant expression is partially redundant

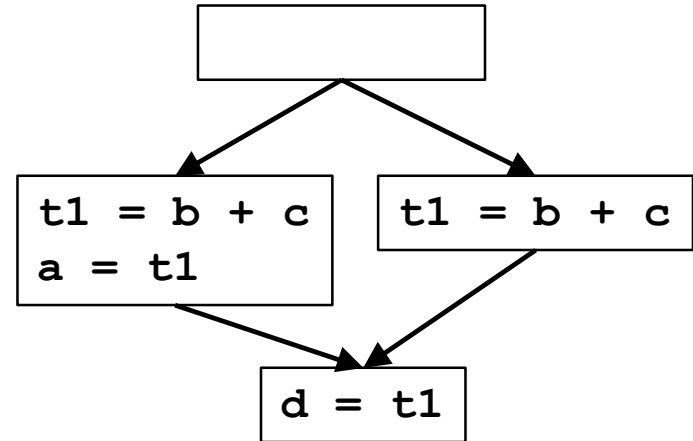
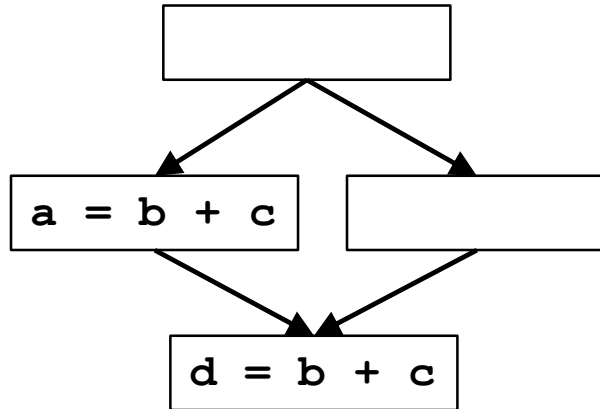


After:



- As before, partially redundant computation can be eliminated if we insert computations to make it fully redundant.
- Remaining copies can be eliminated through copy propagation or more complex analysis of partially redundant assignments.

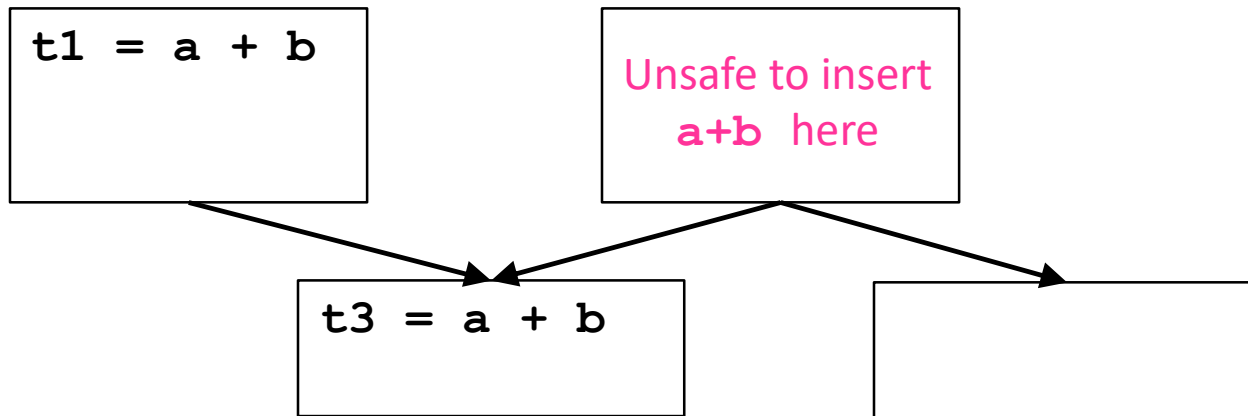
Partial Redundancy Elimination



- Can we place calculations of `b+c` such that no path re-executes the same expression?
- **Partial Redundancy Elimination (PRE)**
 - subsumes:
 - global common subexpression (full redundancy)
 - loop invariant code motion (partial redundancy for loops)

Where Can We Insert Computations?

- **Safety:** never introduce a new expression along any path.



- Insertion could introduce exception, change program behavior.
 - If we can add a new basic block, can insert safely in most cases.
 - Solution: insert expression only where it is **anticipated**, i.e., **its value computed at point p will be used along ALL subsequent paths** (more in next lecture)
- **Performance:** never increase the # of computations on any path.
 - Under simple model, guarantees program won't get worse.
 - Reality: might increase register lifetimes, add copies, lose.

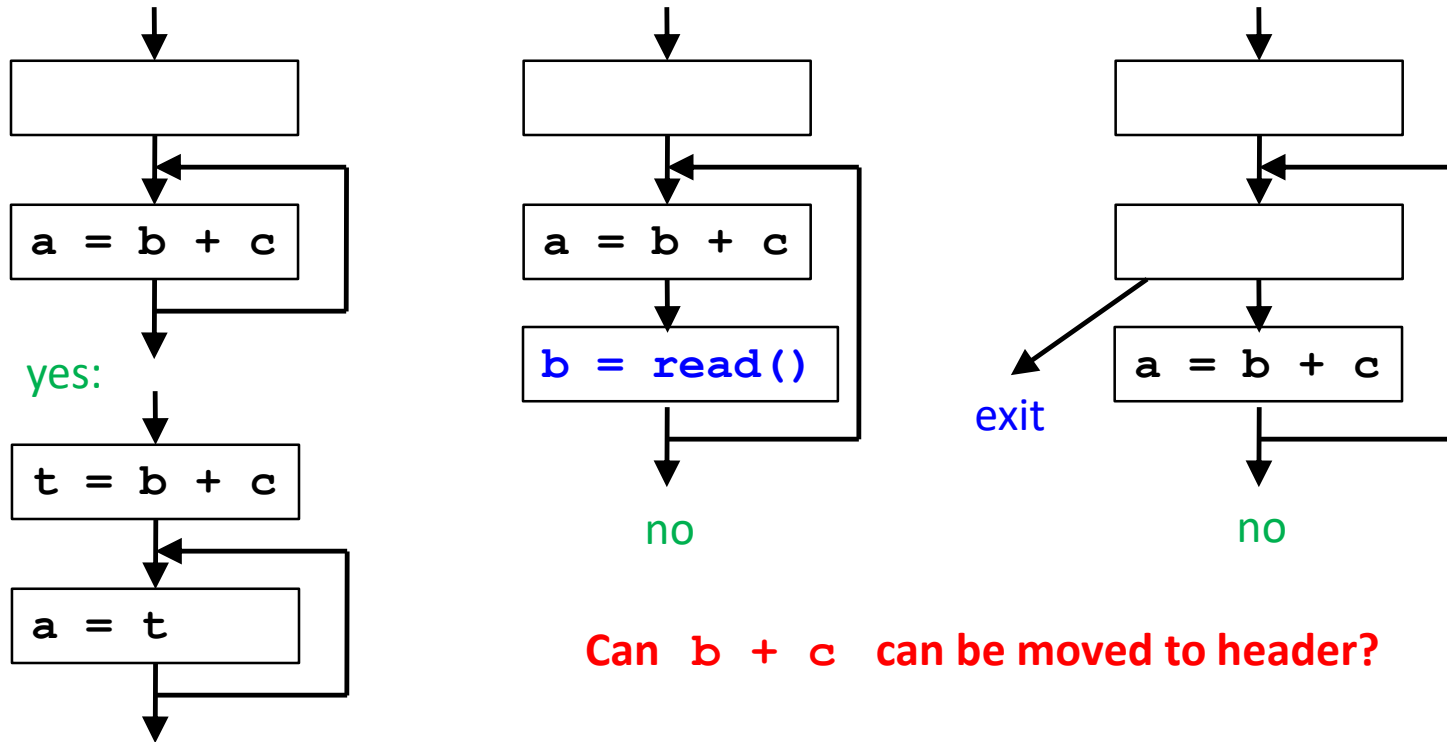
Lecture 10:

Lazy Code Motion

- I. Mathematical concept: a cut set
- II. Lazy Code Motion Algorithm
 - Pass 1: Anticipated Expressions
 - Pass 2: (Will be) Available Expressions
 - Pass 3: Postponable Expressions
 - Pass 4: Used Expressions

ALSU 9.5.3-9.5.5

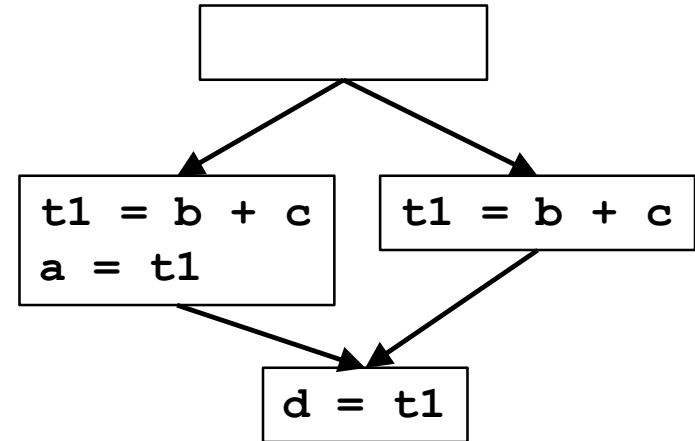
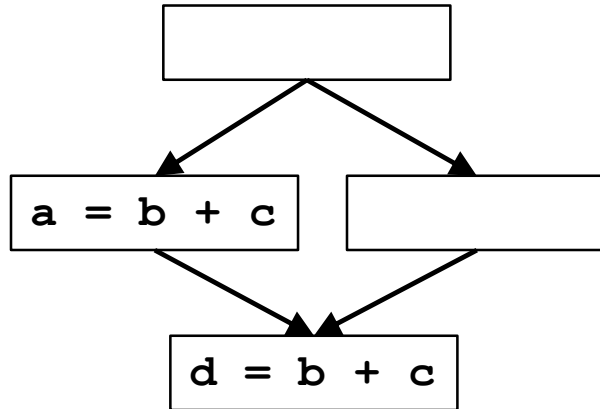
Review: Loop Invariant Code Motion



Can `b + c` can be moved to header?

- Given an expression $(b+c)$ inside a loop,
 - does the value of $b+c$ change inside the loop?
 - is the code executed at least once?

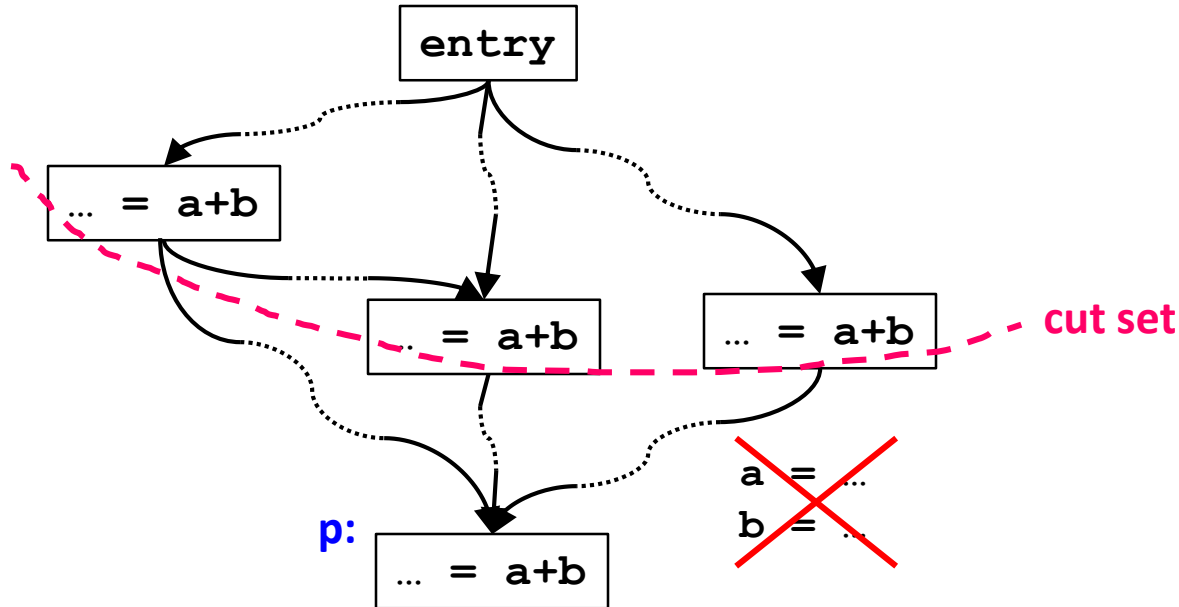
Review: Partial Redundancy Elimination



- Can we place calculations of `b+c` such that no path re-executes the same expression?
- **Partial Redundancy Elimination (PRE)**
 - subsumes:
 - global common subexpression (full redundancy)
 - loop invariant code motion (partial redundancy for loops)

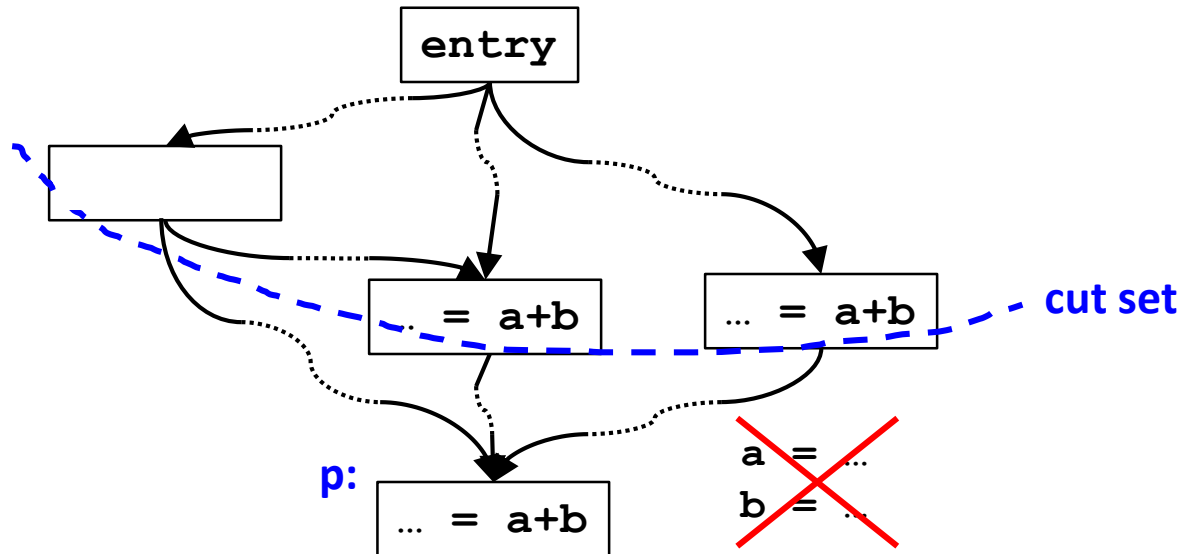
I. Full Redundancy: A Cut Set in a Graph

Key mathematical concept



- **Full redundancy at p:** expression **$a+b$** redundant on **all paths**
 - a **cut set**: nodes that separate **entry** from **p** (there can be many cut sets)
 - each node in a cut set contains a calculation of **$a+b$**
 - **a, b** not redefined

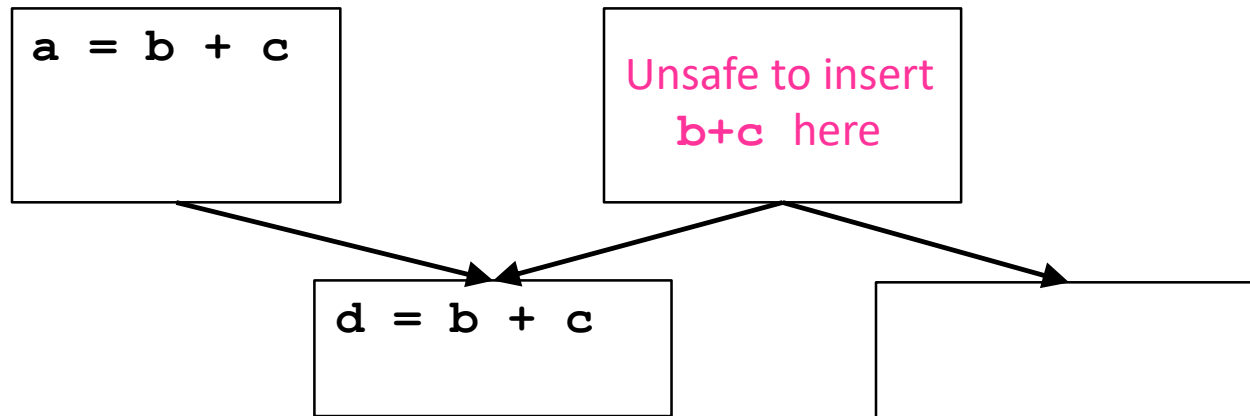
Partial Redundancy: Completing a Cut Set



- **Partial redundancy at p**: redundant on **some but not all paths**
 - Add operations to create a cut set containing $a+b$
 - Note: Moving operations up can eliminate redundancy
- **Constraint on placement: no wasted operation**
 - $a+b$ is “**anticipated**” at B if its value computed at B will be used along ALL subsequent paths
 - a, b not redefined, no branches that lead to exit without use
- **Range where $a+b$ is anticipated $\rightarrow$ Choice**

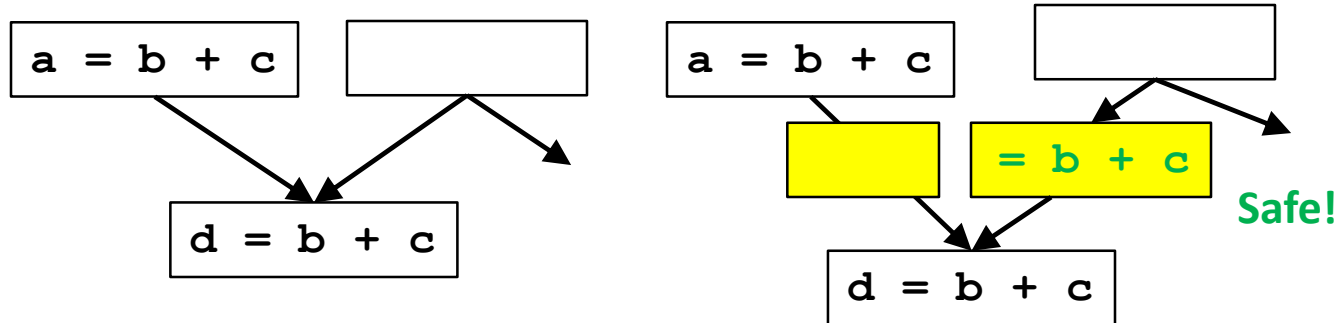
Review: Where Can We Insert Computations?

- **Safety:** never introduce a new expression along any path.



- Insertion could introduce exception, change program behavior.
 - Solution: insert expression only where it is **anticipated**, i.e., its value computed at point *p* will be used along ALL subsequent paths
- **Performance:** never increase the # of computations on any path.
 - Under simple model, guarantees program won't get worse.
 - Reality: might increase register lifetimes, add copies, lose.

Preparing the Flow Graph



- **Definition: Critical edges**
 - source basic block has multiple successors
 - destination basic block has multiple predecessors
- **Modify the flow graph:**
 - Add a basic block for every edge that leads to a basic block with multiple predecessors (not just on critical edges)
 - How does this help the example?
 - To keep algorithm simple: consider each statement as its own basic block and restrict placement of instructions to the beginning of a basic block

II. Lazy Code Motion Algorithm

- Pass 1: Anticipated Expressions
- Pass 2: (Will be) Available Expressions
- Pass 3: Postponable Expressions
- Pass 4: Used Expressions

Big picture:

- First calculates the “**earliest**” set of blocks for insertion
 - this maximizes redundancy elimination
 - but may also result in long register lifetimes
- Then it calculates the “**latest**” set of blocks for insertion
 - achieving the same amount of redundancy elimination as “earliest”
 - but hopefully reducing the **lifetime of the register** holding the value of the expression

Pass 1: Anticipated Expressions

This pass does most of the heavy lifting in eliminating redundancy

- **Backward pass: Anticipated expressions**

Anticipated[b].in: Set of expressions anticipated at the entry of b

- An expression is anticipated if its value computed at point p will be used along ALL subsequent paths

	Anticipated Expressions
Domain	Sets of expressions
Direction	
Transfer Function	
$\wedge$	
Boundary	in[exit] =
Initialization	in[b] =

Pass 1: Anticipated Expressions

This pass does most of the heavy lifting in eliminating redundancy

- **Backward pass: Anticipated expressions**

Anticipated[b].in: Set of expressions anticipated at the entry of b

- An expression is anticipated if its value computed at point p will be used along ALL subsequent paths

	Anticipated Expressions
Domain	Sets of expressions
Direction	backward
Transfer Function	$f_b(x) = EUse_b \cup (x - EKill_b)$ EUse: used exp, EKill: exp killed
$\wedge$	$\cap$
Boundary	$in[exit] = \emptyset$
Initialization	$in[b] = \{\text{all expressions}\}$

- **First approximation:**
 - place operations at the frontier of anticipation
(boundary between not anticipated and anticipated)

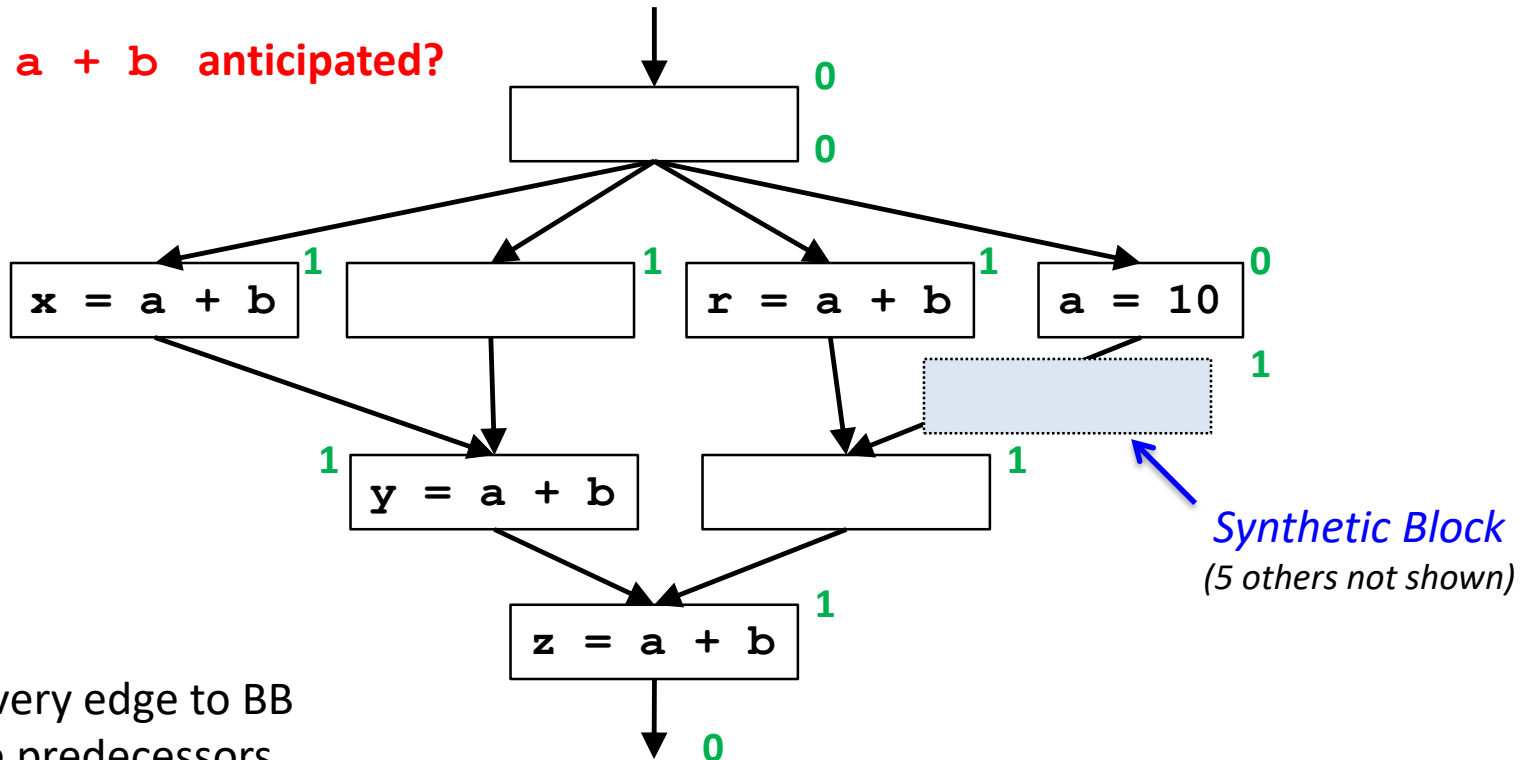
Example 1

See the algorithm in action

$$IN[i] = EUse[i] \cup (OUT[i] - EKill[i])$$

$$Meet = \cap$$

Where is $a + b$ anticipated?



Add BB for every edge to BB
with multiple predecessors

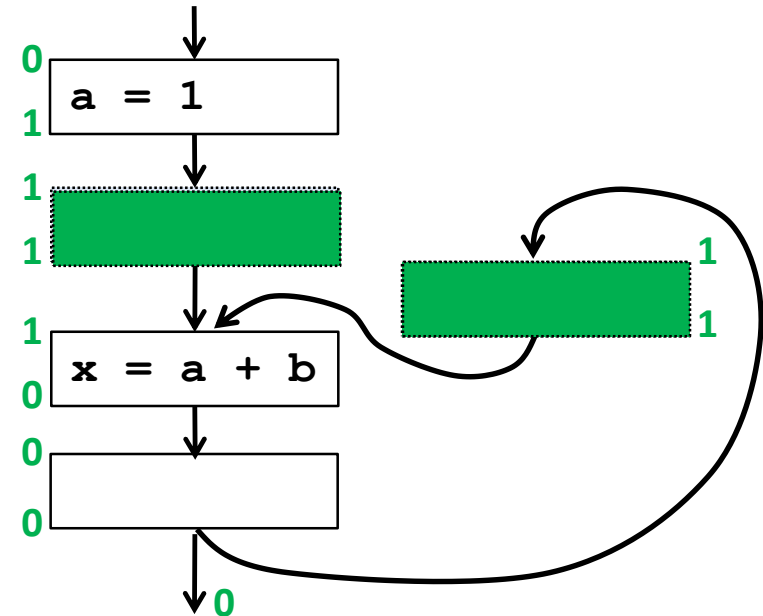
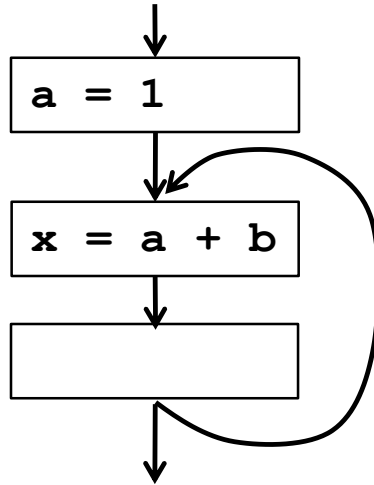
- What is the result if we insert $t = a + b$ at the frontier of anticipation?

Example 2

(Loop Invariant Code)

$$IN[i] = EUse[i] \cup (OUT[i] - EKill[i])$$

$$Meet = \cap$$



Add BB for every edge to BB
with multiple predecessors

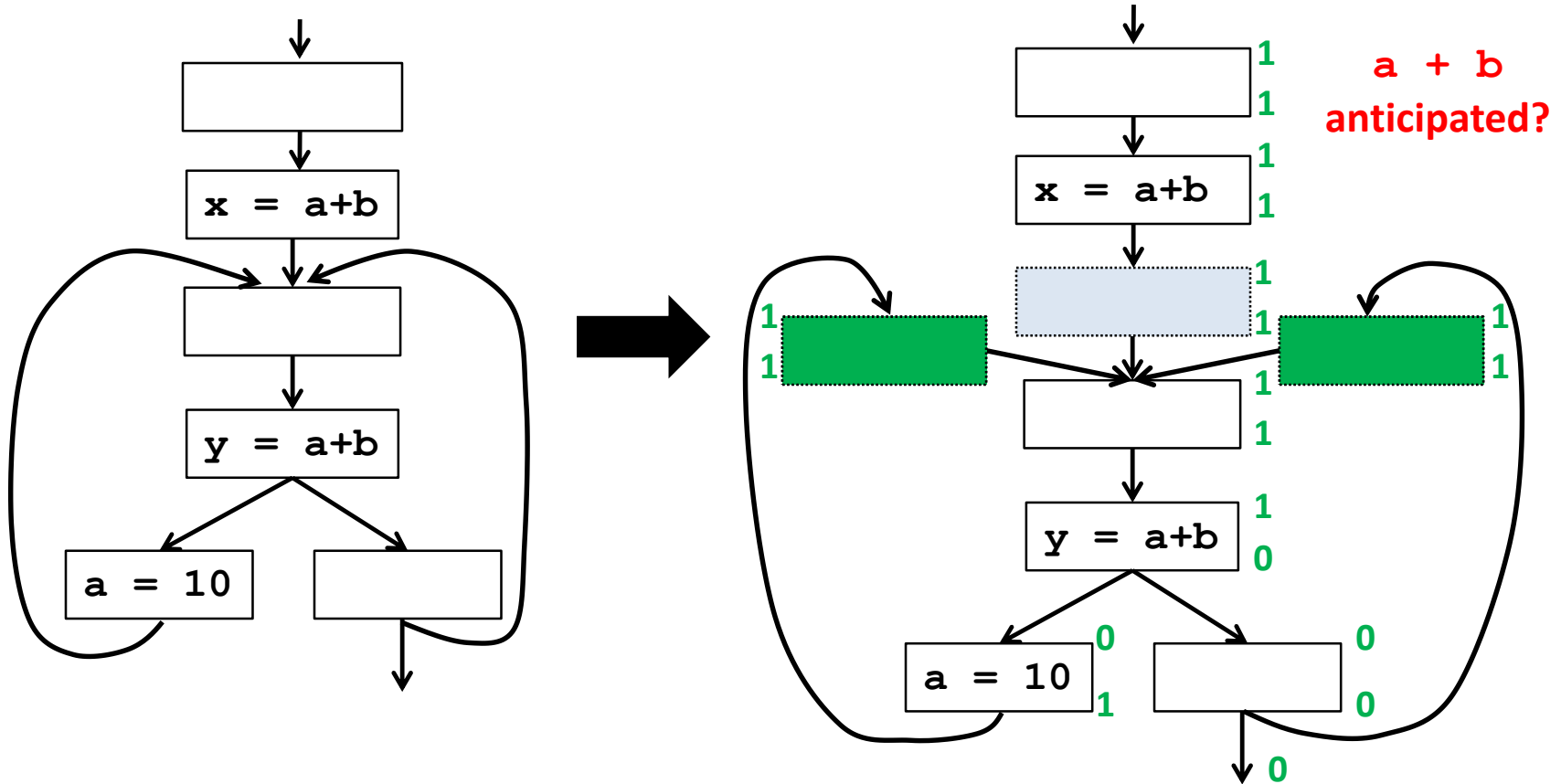
Where is $a + b$ anticipated?

- Which blocks comprise the **frontier of anticipation**?
- Was inserting $a + b$ at the **frontier of anticipation** the right thing to do in this case?
 - **doesn't eliminate redundancy within loop (why not?)**

Example 3 (More Complex Loop)

$$IN[i] = EUse[i] \cup (OUT[i] - EKill[i])$$

Meet = $\cap$



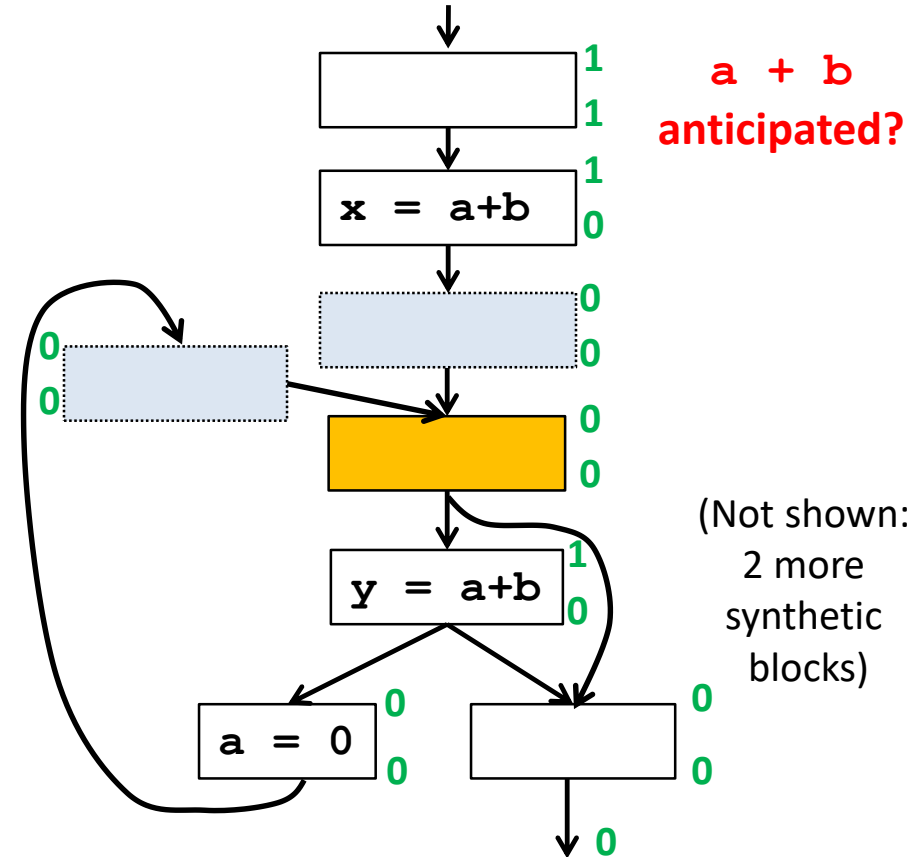
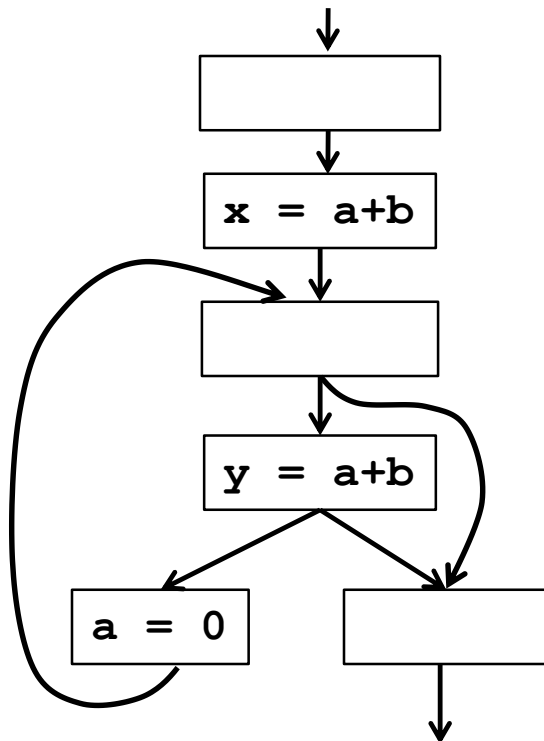
- Where would we **ideally** like to insert “`a+b`” in this case? **only in added block on left**
- What happens if we insert at the **frontier of anticipation**? **insert in both green blocks**

Example 4

(Variation on Previous Loop)

$$IN[i] = EUse[i] \cup (OUT[i] - EKill[i])$$

Meet = $\cap$

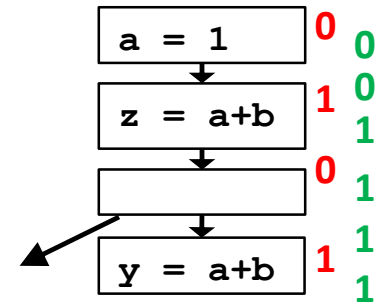


- Is there any opportunity to eliminate redundancy here?
 no: unsafe to insert in left added block ($a+b$ not anticipated there)
 (e.g. " $a+b$ " could be " b/a " & orange block could be "if $a > 0$ ")

Pass 2: Place As Early As Possible

There is still some redundancy left!

- **First approximation:** frontier between “not anticipated” & “anticipated”
- **Complication:** anticipation may **oscillate**
- Pretend we calculate expression **e** whenever it is anticipated
- **e will be available at p** if **e** has been “anticipated but not subsequently killed” on all paths reaching p



	(will be) Available Expressions
Domain	Sets of expressions
Direction	forward
Transfer Function	$f_b(x) = (\text{Anticipated}[b].\text{in} \cup x) - \text{EKill}_b$
$\wedge$	$\cap$
Boundary	$\text{out}[\text{entry}] = \emptyset$
Initialization	$\text{out}[b] = \{\text{all expressions}\}$

Early Placement

- **earliest(b)**
 - set of expressions added to block b under early placement
 - calculated from results of first 2 passes
- **Place expression at the earliest point anticipated and not already available**
 - $\text{earliest}(b) = \text{anticipated}[b].\text{in} - \text{available}[b].\text{in}$
- **Algorithm**
 - For all basic block b, if $x+y \in \text{earliest}[b]$
 - at beginning of b:
create a new variable t
 $t = x+y$,
replace every original $x+y$ by t

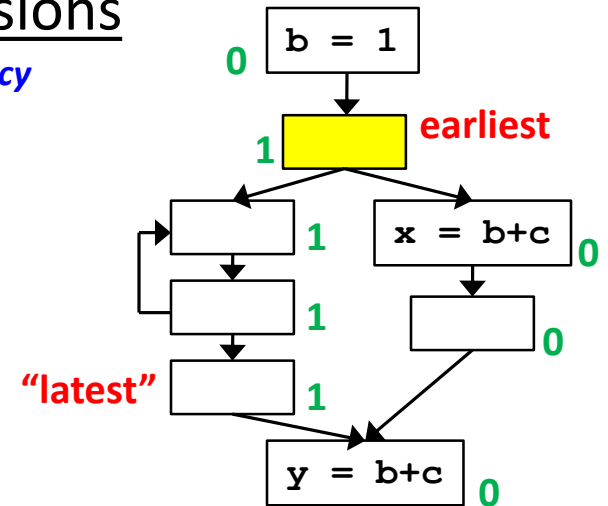
Result:

- Maximized redundancy elimination
- Placed as early as possible
- But: register lifetimes?

Pass 3: Postponable Expressions

Let's be lazy without introducing redundancy

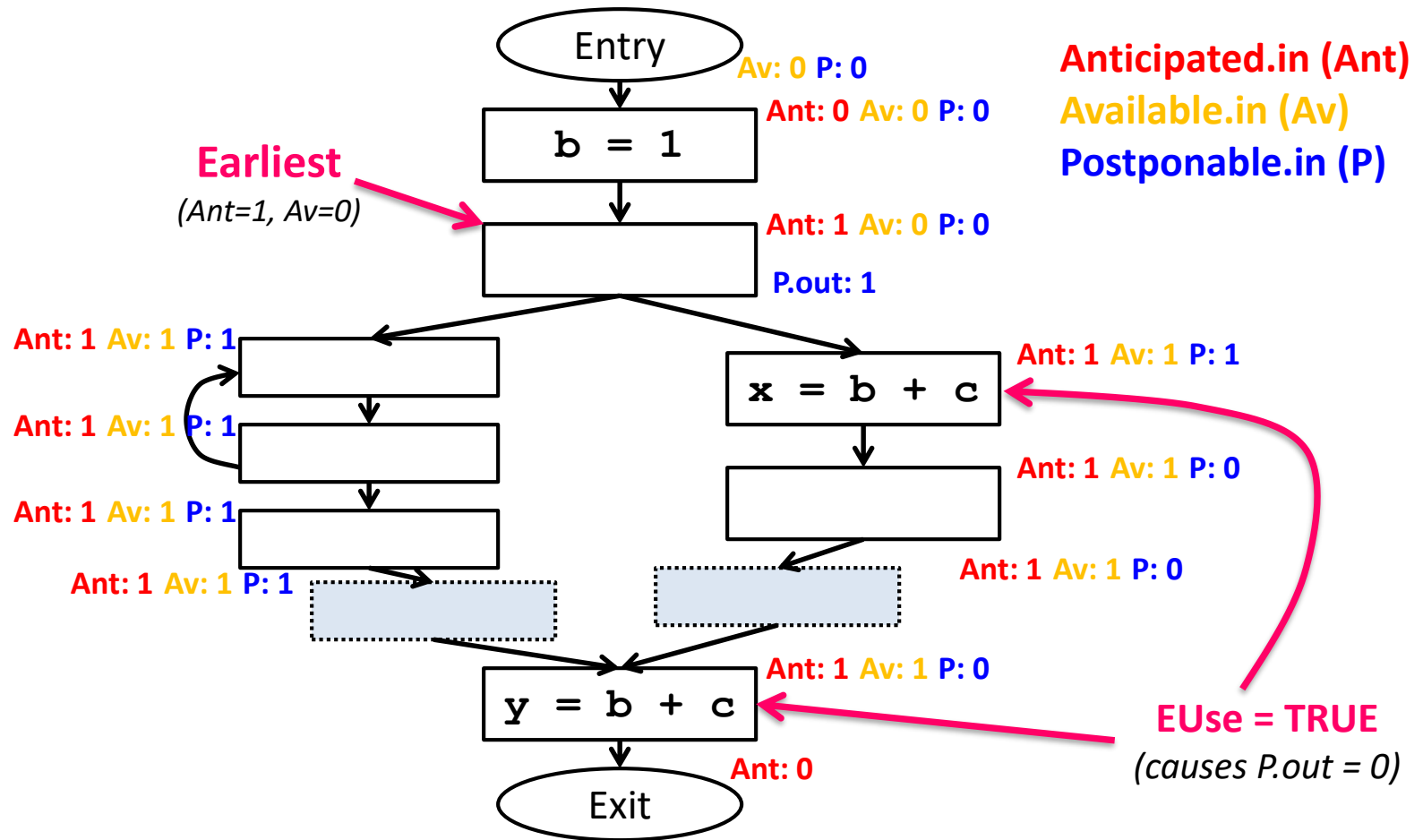
- Delay creating redundancy to reduce register pressure



- An expression **e** is **postponable** at a program point **p** if
 - all paths leading to **p** have seen earliest placement of **e** but not a subsequent use

	Postponable Expressions
Domain	Sets of expressions
Direction	forward
Transfer Function	$f_b(x) = (\text{earliest}[b] \cup x) - \text{EUse}_b$
$\wedge$	$\cap$
Boundary	$\text{out}[\text{entry}] = \emptyset$
Initialization	$\text{out}[b] = \{\text{all expressions}\}$

Example Illustrating “Postponable”



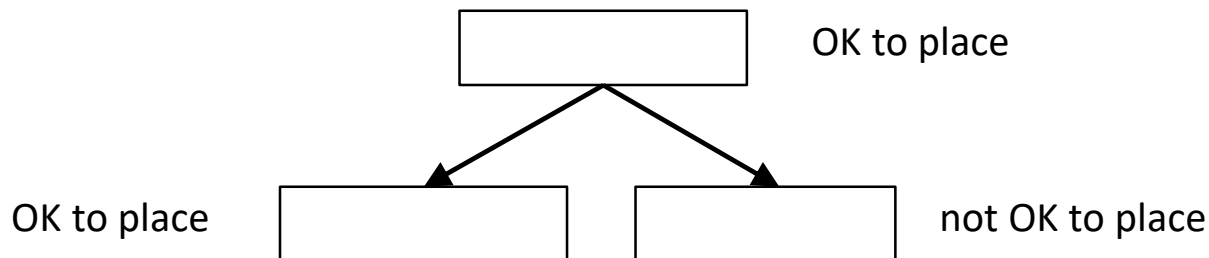
$$\text{Ant.IN}[i] = \text{EUse}[i] \cup (\text{Ant.OUT}[i] - \text{EKill}[i])$$

$$\text{Avail.OUT}[i] = (\text{Ant.IN}[i] \cup \text{Avail.IN}[i]) - \text{EKill}[i]$$

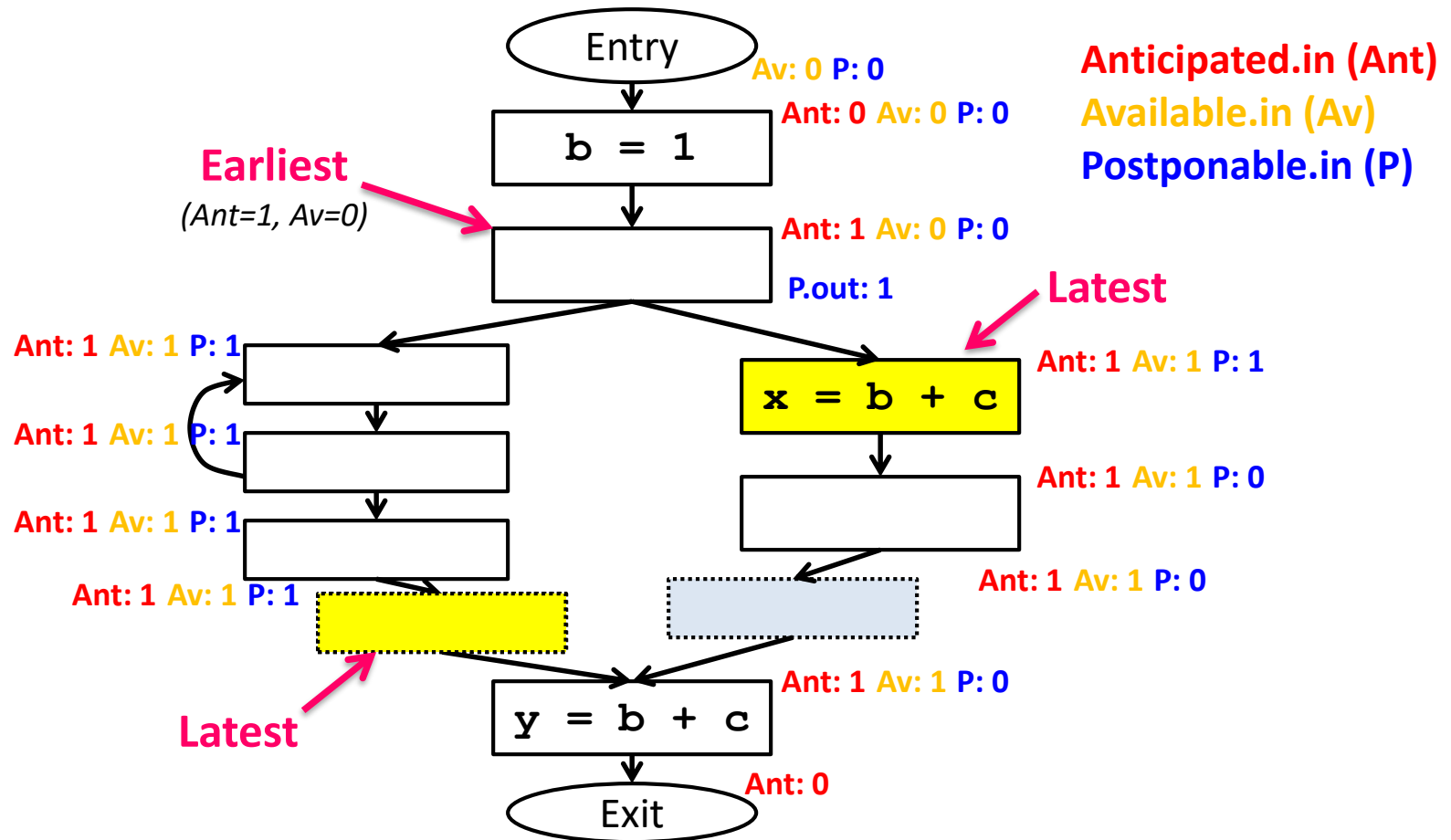
$$\text{Post.OUT}[i] = (\text{Earliest}[i] \cup \text{Post.IN}[i]) - \text{EUse}[i]$$

Latest: frontier at the end of “postponable” cut set

- $\text{latest}[b] = (\text{earliest}[b] \cup \text{postponable.in}[b]) \cap$
 $(\text{EUse}_b \cup \neg(\bigcap_{s \in \text{succ}[b]} (\text{earliest}[s] \cup \text{postponable.in}[s])))$
 - OK to place expression: **earliest** or **postponable**
 - Need to place at b if either
 - **used in b**, or
 - **not OK to place in one of its successors**
- Works because of **pre-processing step** (an empty block was introduced to an edge if the destination has multiple predecessors)
 - if b has a successor that cannot accept postponement, b has only one successor
 - The following does not exist:



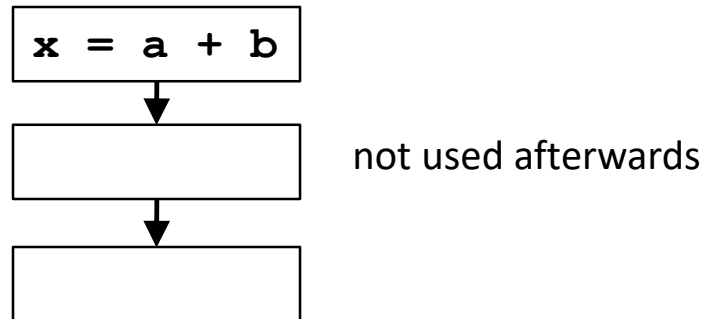
Example Illustrating “Latest”



- $\text{latest}[b] = (\text{earliest}[b] \cup \text{postponable.in}[b]) \cap$
 $(\text{EUse}_b \cup \neg(\bigcap_{s \in \text{succ}[b]} (\text{earliest}[s] \cup \text{postponable.in}[s])))$

Pass 4: Used Expressions

Finally... this is easy, it is like liveness (for expressions)

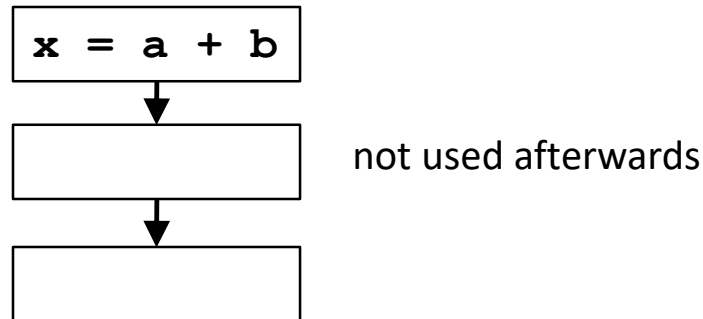


- Eliminate temporary variable assignments unused beyond current block
- **Compute:** **Used**.out[b]: sets of **used (live) expressions** at exit of b.

	Used Expressions
Domain	Sets of expressions
Direction	
Transfer Function	
$\wedge$	
Boundary	in[exit] =
Initialization	in[b] =

Pass 4: Used Expressions

Finally... this is easy, it is like liveness (for expressions)



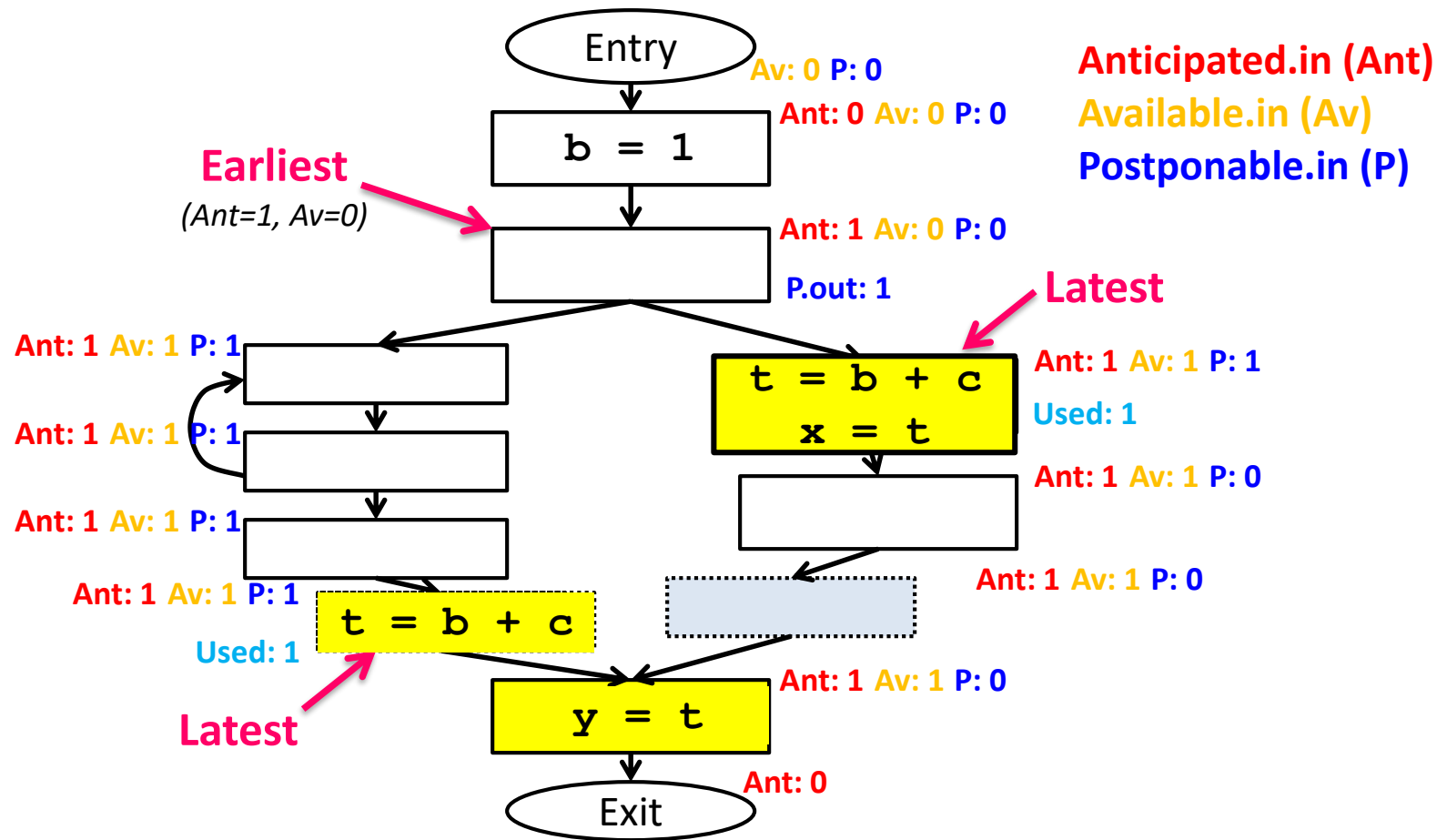
- Eliminate temporary variable assignments unused beyond current block
- **Compute:** **Used.out[b]**: sets of **used (live) expressions** at exit of b.

	Used Expressions
Domain	Sets of expressions
Direction	backward
Transfer Function	$f_b(x) = (\text{EUse}[b] \cup x) - \text{latest}[b]$
$\wedge$	$\cup$
Boundary	$\text{in}[\text{exit}] = \emptyset$
Initialization	$\text{in}[b] = \emptyset$

Code Transformation

- For all basic blocks b ,
if $(x+y) \in (\text{latest}[b] \cap \text{used.out}[b])$
at beginning of b :
add $\text{new } t = x+y$
replace every original $x+y$ by t

Transformed Code



If $(x+y) \in (\text{latest}[b] \cap \text{used.out}[b])$ then add $t = x+y$. Replace every original $x+y$ by t

4 Passes for Partial Redundancy Elimination

1. *Safety*: Cannot introduce operations not executed originally

- Pass 1 (backward): **Anticipation**: range of code motion
- Placing operations at the frontier of anticipation gets most of the redundancy

2. *Squeezing the last drop of redundancy*:

An anticipation frontier may cover a subsequent frontier

- Pass 2 (forward): **Availability**
- **Earliest**: anticipated, but not yet available

3. *Push the cut set out -- as late as possible*

To minimize register lifetimes

- Pass 3 (forward): **Postponability**: move it down provided it does not create redundancy
- **Latest**: where it is used or the frontier of postponability

4. *Cleaning up*

- Pass 4 (backward): **Remove unneeded temporary assignments**

Remarks

- **Powerful algorithm**
 - Finds many forms of redundancy in one unified framework
- **Illustrates the power of data flow**
 - Multiple data flow problems

Lecture 11

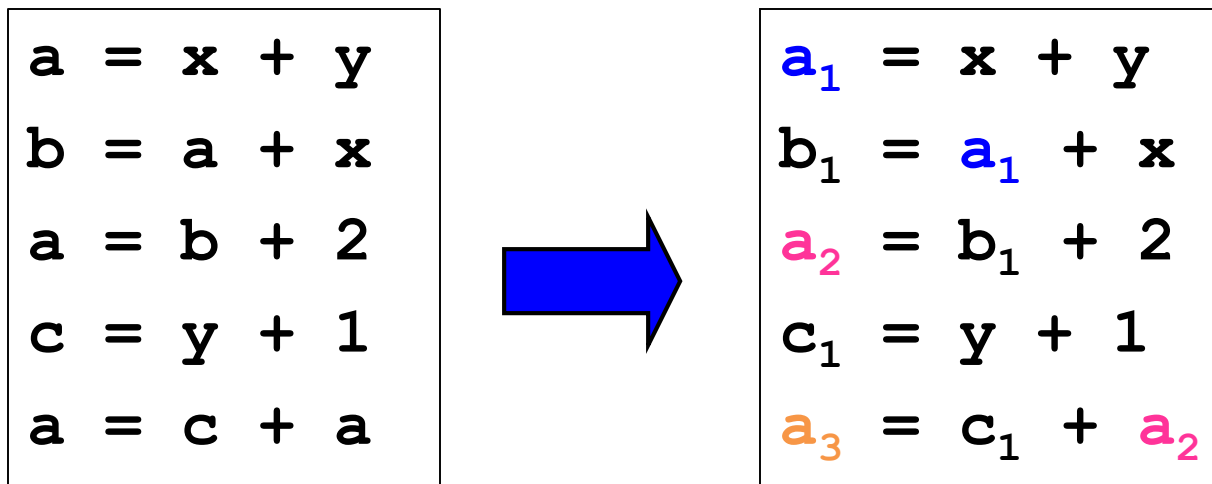
Static Single Assignment (SSA)

- I. Review: Intro to SSA
- II. When/Where to Insert Φ
- III. Example
- IV. Constant Propagation with SSA

ALSU 6.2.4

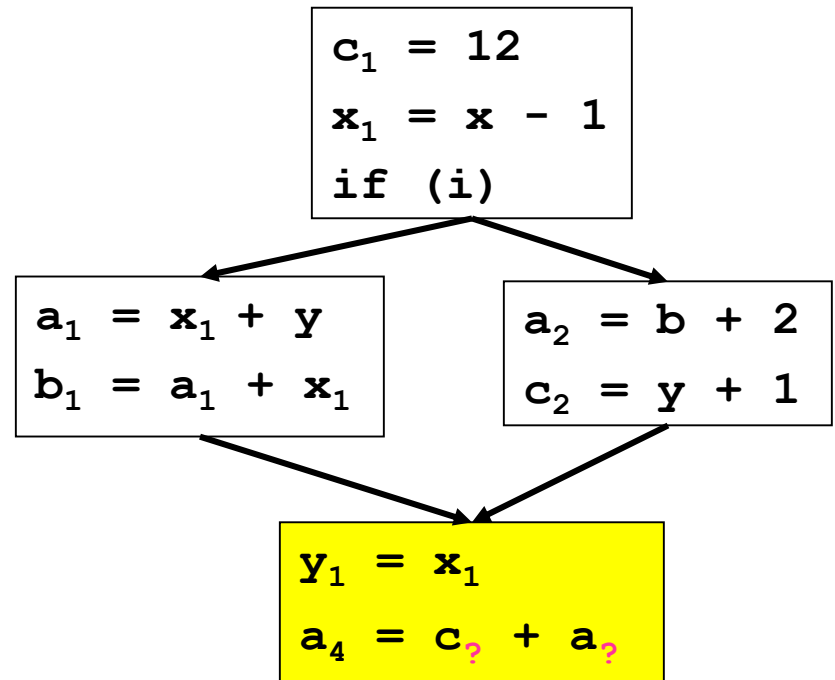
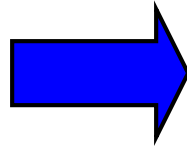
I. Review: Static Single Assignment (SSA)

- Static single assignment is an IR where every variable is assigned a value at most once in the program text
- Easy for a basic block (reminiscent of Value Numbering):
 - Visit each instruction in program order:
 - LHS: assign to a *fresh version* of the variable
 - RHS: use the *most recent version* of each variable



Review: What about Joins in the CFG?

```
c = 12
x = x - 1
if (i) {
    a = x + y
    b = a + x
} else {
    a = b + 2
    c = y + 1
}
y = x
a = c + a
```

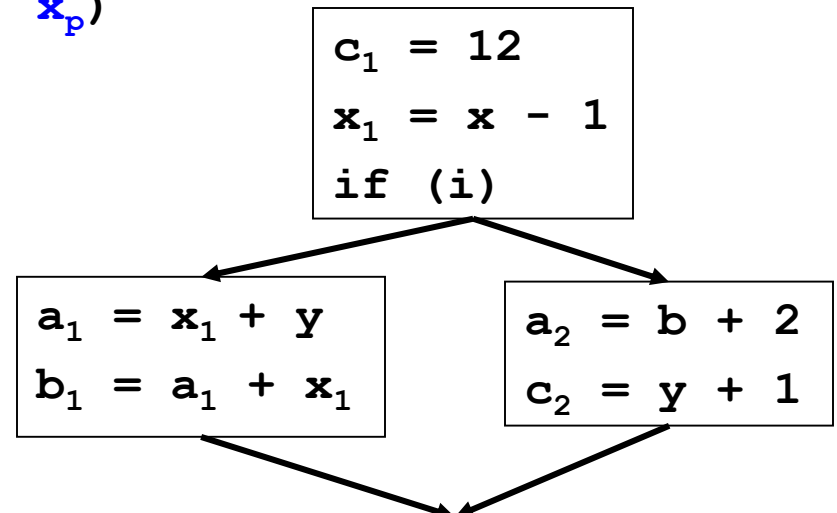


→ Use a notational convention (fiction): a Φ function

Review: The Φ function

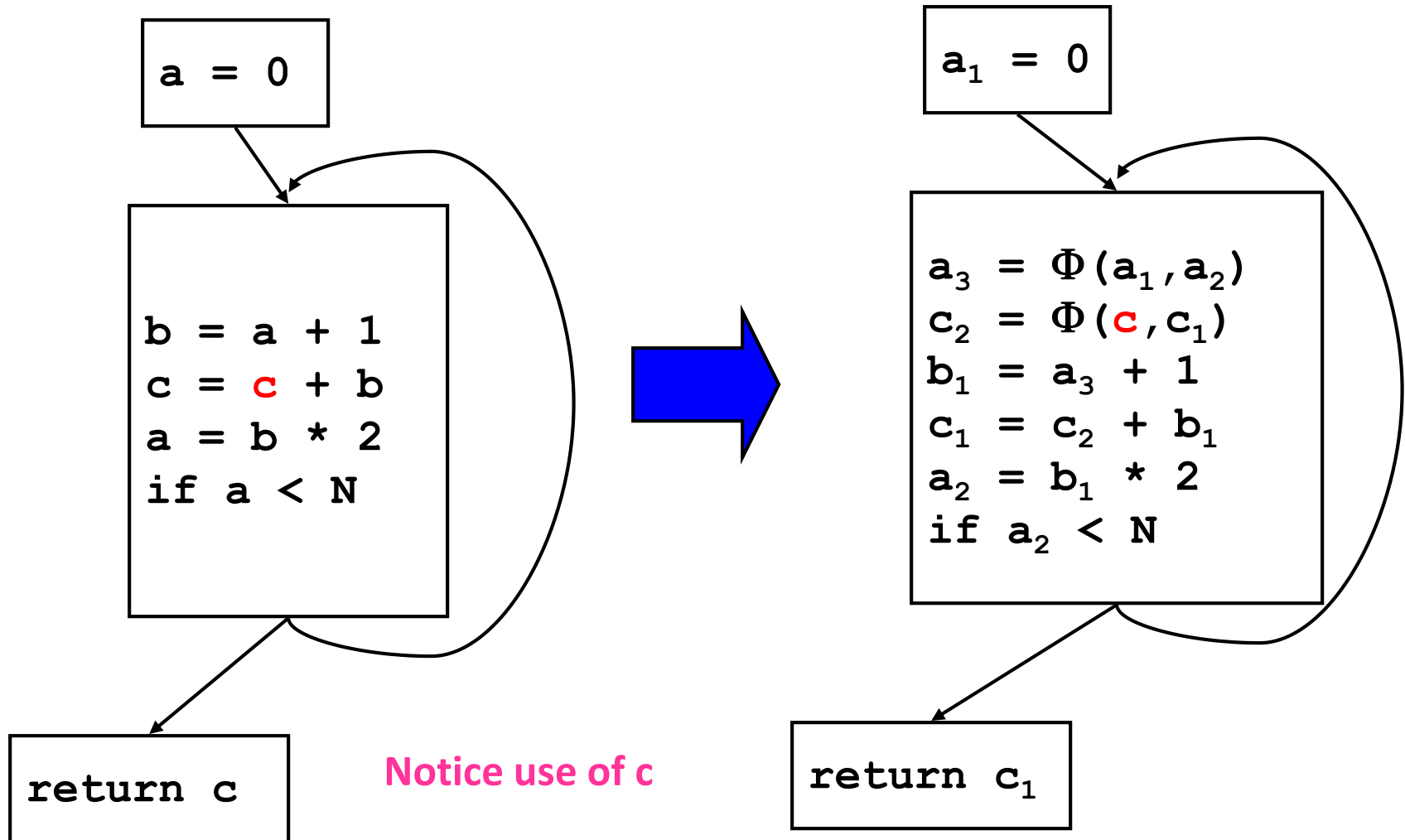
- Φ merges multiple definitions along multiple control paths into a single definition.
- At a basic block with p predecessors, there are p arguments to the Φ function.

$$x_{\text{new}} = \Phi(x_1, x_2, x_3, \dots, x_p)$$

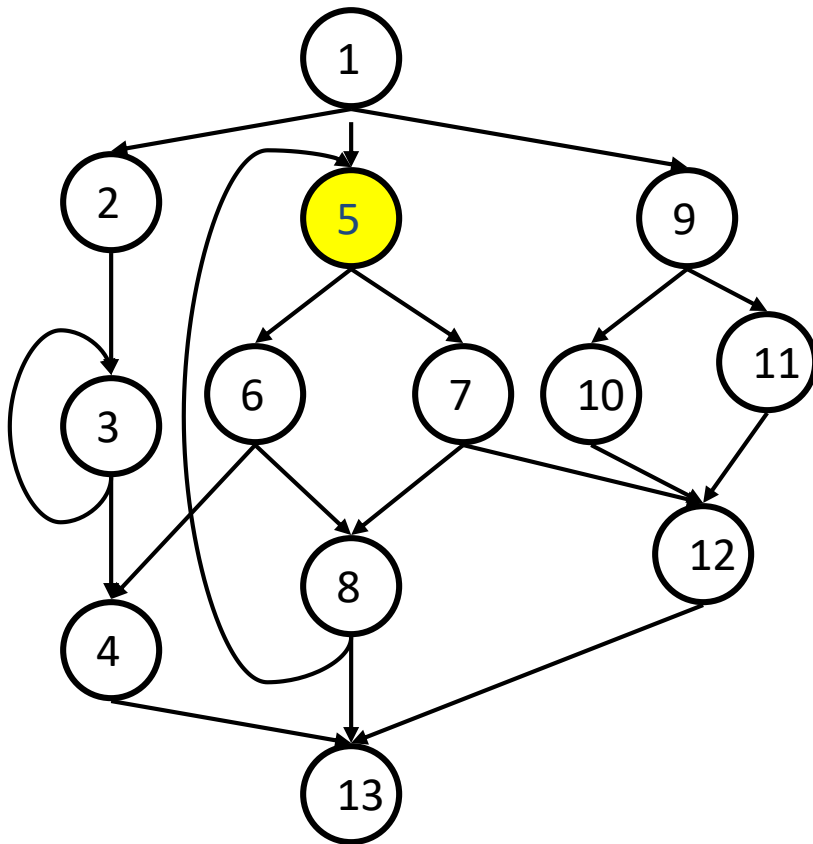


- **Minimal SSA:** At each join point, insert Φ functions for **all live variables** with **multiple outstanding defs**

Another Example



II. When/Where to Insert Φ ?

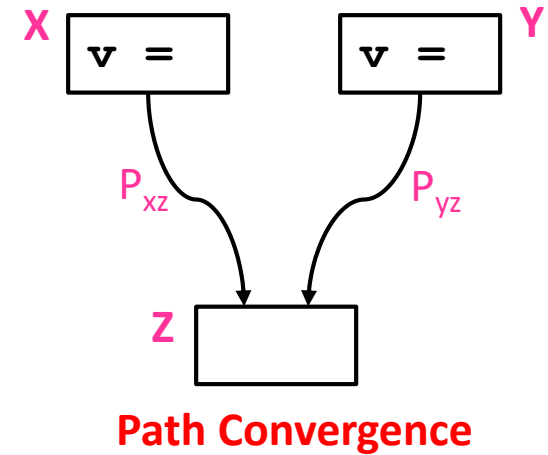


If there is a def of **a** in block **5**,
which nodes need a $\Phi()$?

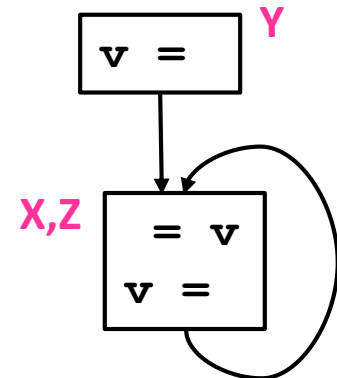
Control Flow Graph (CFG)

When/Where to insert Φ ?

- We insert a Φ function for variable v in block Z iff:
 - v was defined more than once before
 - (i.e., v defined in X and Y AND $X \neq Y$)
 - There exists nonempty paths P_{xz} from X to Z and P_{yz} from Y to Z s.t. Z is the first node common to the two paths
 - Nonempty = at least one edge
 - Note: one of X or Y can be Z

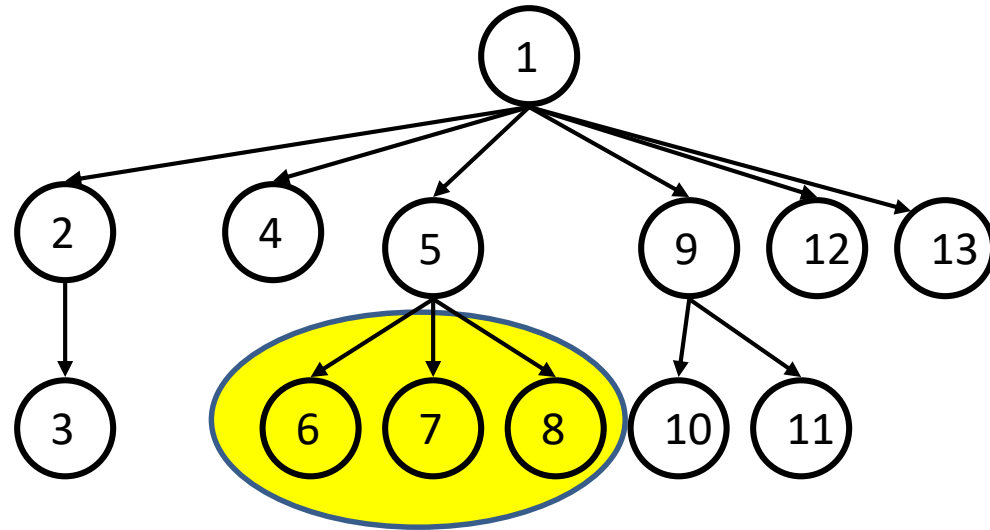
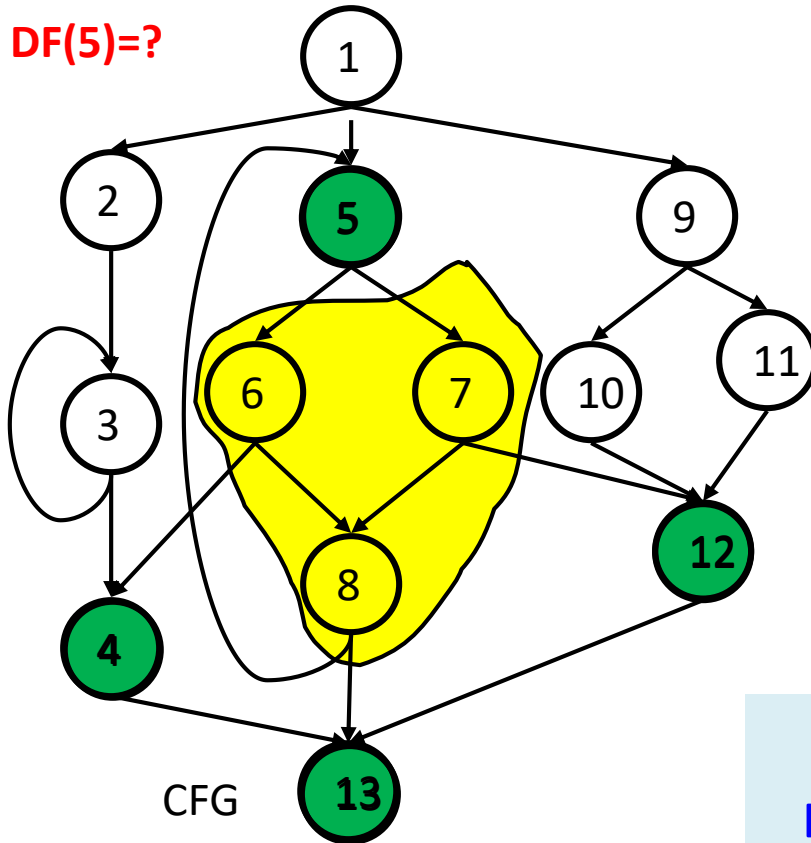


- **Entry block** contains an implicit def of all vars
- Note: $v = \Phi(\dots)$ is a def of v



Dominance Frontier

DF(5)=?



Dominance Tree

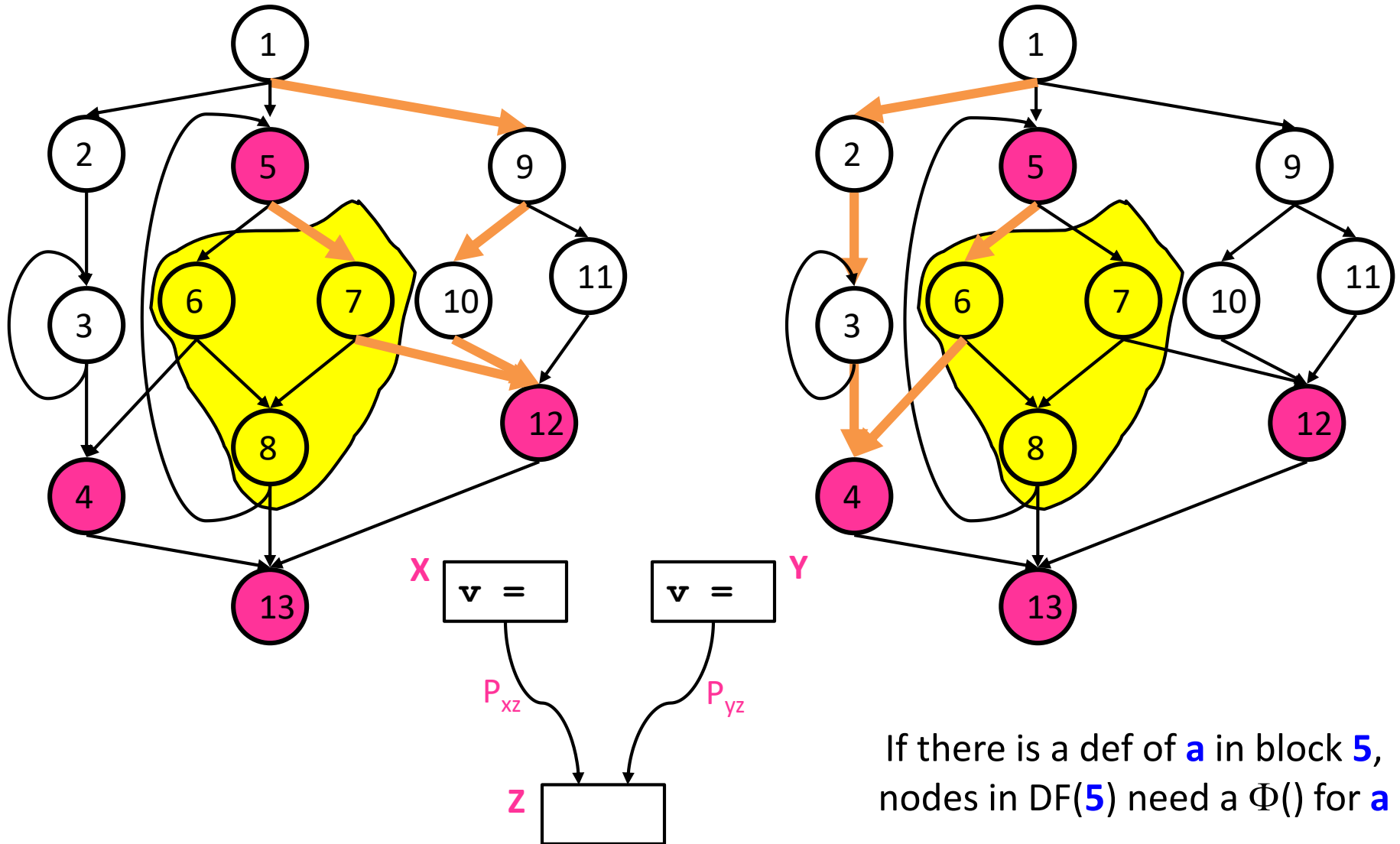
x sdom w iff x is a proper ancestor of w

The **Dominance Frontier** of a node x
 $DF(x) = \{ w \mid x \text{ dom pred}(w) \text{ AND } !(x \text{ sdom } w) \}$

x strictly dominates w (x sdom w) iff impossible to reach w without passing through x first

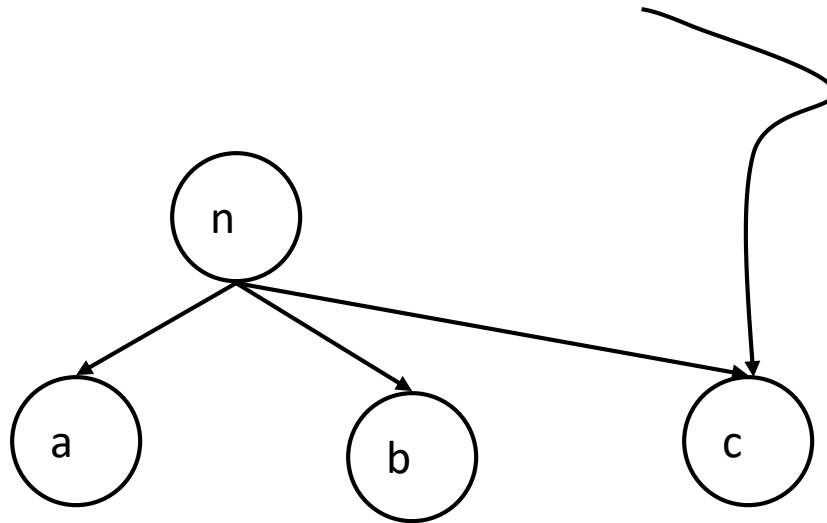
x dominates w (x dom w) iff x sdom w OR $x = w$

Dominance Frontier (DF) and Path Convergence



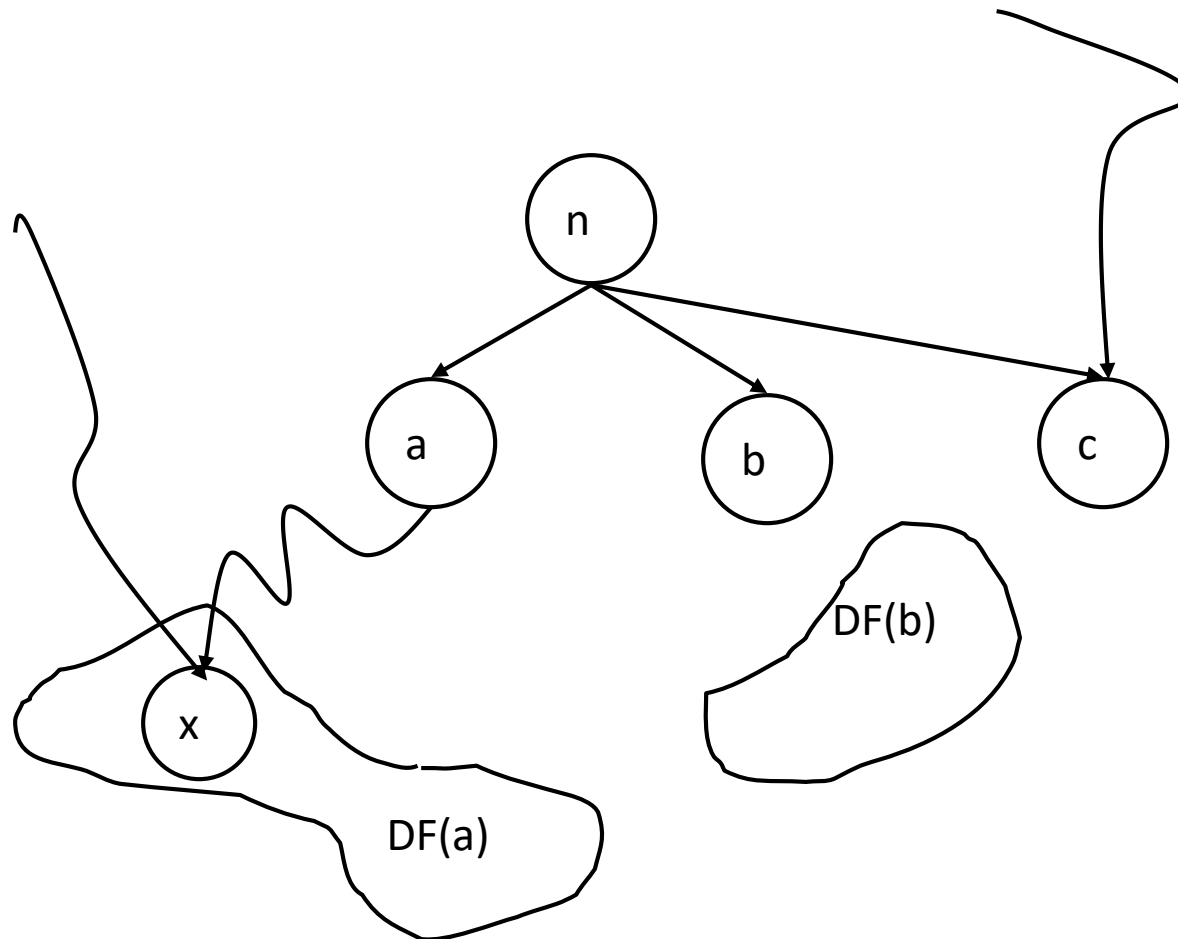
Computing DF(n)

$$DF(x) = \{ w \mid x \text{ dom pred}(w) \text{ AND } !(x \text{ sdom } w) \}$$



n dom n
n dom a
n dom b
!(n dom c)

Computing DF(n)



$n \text{ dom } n$
 $n \text{ dom } a$
 $n \text{ dom } b$
 $!(n \text{ dom } c)$

Computing the Dominance Frontier

$$DF(n) = \{ w \mid n \text{ dom pred}(w) \text{ AND } !(n \text{ sdom } w) \}$$

compute-DF(n)

$S = \{\}$

foreach node c in succ[n]

if $!(n \text{ sdom } c)$

$S = S \cup \{ c \}$ // e.g., node c on previous slide

foreach child a of n in the Dominance Tree

compute-DF(a)

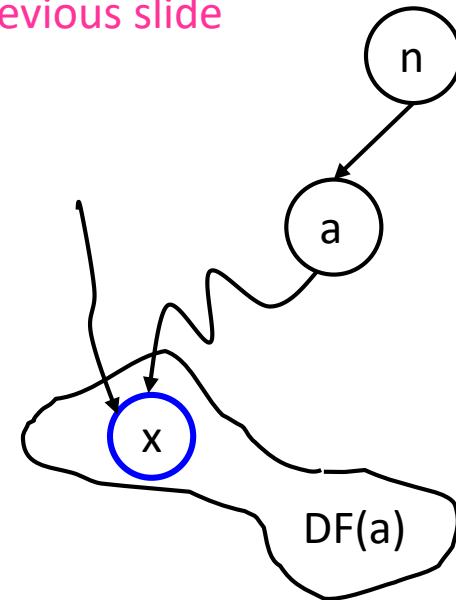
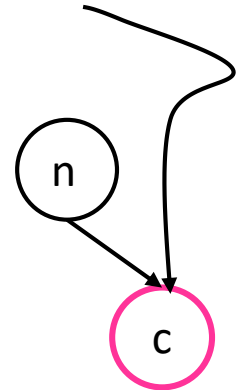
foreach x in DF[a]

if $!(n \text{ dom } x)$

$S = S \cup \{ x \}$

// e.g., node x on previous slide

$DF[n] = S$



Using Dominance Frontier to Compute SSA: Overview

1. Place all $\Phi()$
2. Rename all variables

Using Dominance Frontier to Place $\Phi()$

- Gather all the defsites of every variable
- Then, for every variable
 - foreach defsitem
 - foreach node in DominanceFrontier(defsite)
 - if we haven't put $\Phi()$ in node, then put one in
 - if this node didn't define the variable before, then add this node to the defsites (because Φ counts as def)
- This essentially computes the Iterated Dominance Frontier on the fly, inserting the minimal number of $\Phi()$ necessary

Using Dominance Frontier to Place $\Phi()$: Algorithm

```

 $\forall n, v$ : initialize orig[n], defsites[v], PHI[v] to empty set
foreach node n {
    foreach variable v defined in n {
        orig[n]  $\cup$ = {v}           // variables defined in basic block n
        defsites[v]  $\cup$ = {n}       // basic blocks that define variable v
    }
}
foreach variable v {
    W = defsites[v]               // work list of basic blocks
    while W not empty {
        n = remove node from W
        foreach y in DF[n]
            if  $y \notin \text{PHI}[v]$  {
                insert " $v \leftarrow \Phi(v, v, \dots)$ " at top of y
                 $\text{PHI}[v] = \text{PHI}[v] \cup \{y\}$            // BBs containing a  $\Phi$  for v
                if  $v \notin \text{orig}[y]$ :  $W = W \cup \{y\}$  // add BB to work list
            }
    }
}
}
```

Renaming Variables

- Algorithm:
 - Walk the Dominance Tree, renaming variables as you go
 - Replace uses with more recent renamed def
- For straight-line code this is easy
- What if there are branches and joins?
 - use the **closest def such that the def is above the use in the Dominance Tree**

- Easy implementation:
 - Call **rename**(entry)

rename(B):

for each statement in B:

 replace (non- Φ) use of v with top of stack(v)

 replace def of v with v_{new} , push v_{new} onto stack(v)

for each successor S of B in CFG:

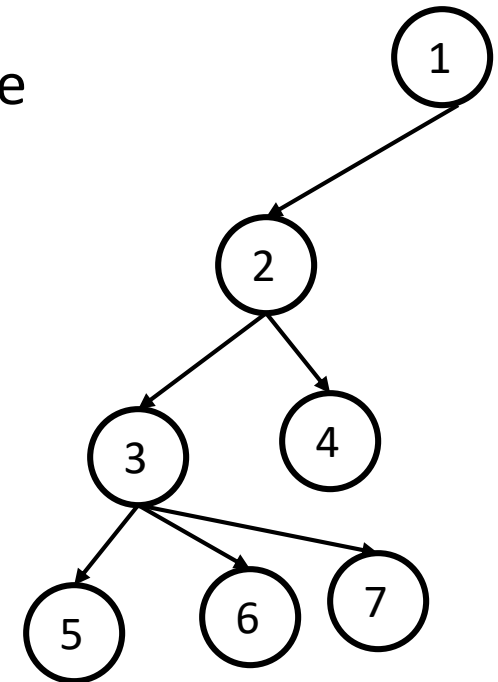
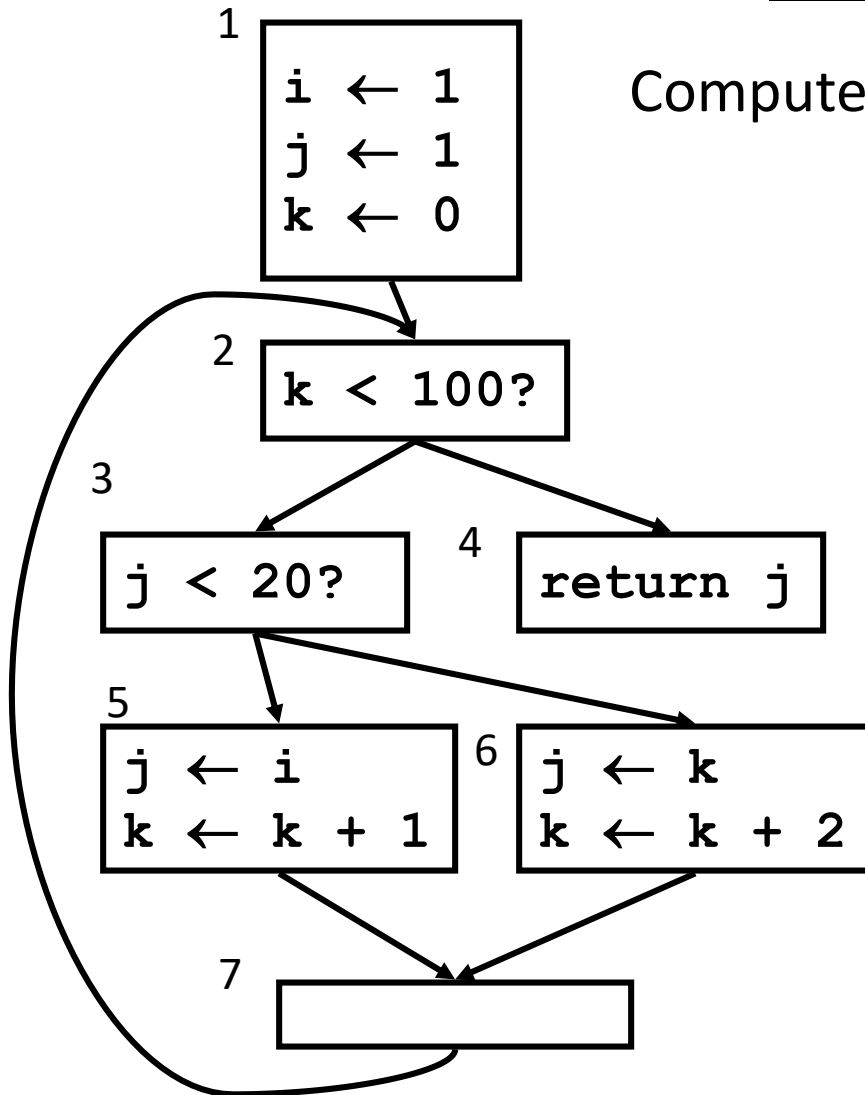
 replace k 'th arg. of $\Phi(\mathbf{v}, \dots \mathbf{v})$ with top of stack(v),
 where B is k 'th predecessor of S

call **rename**(C) on all children C of B in Dominance Tree

pop all defs in B from stacks

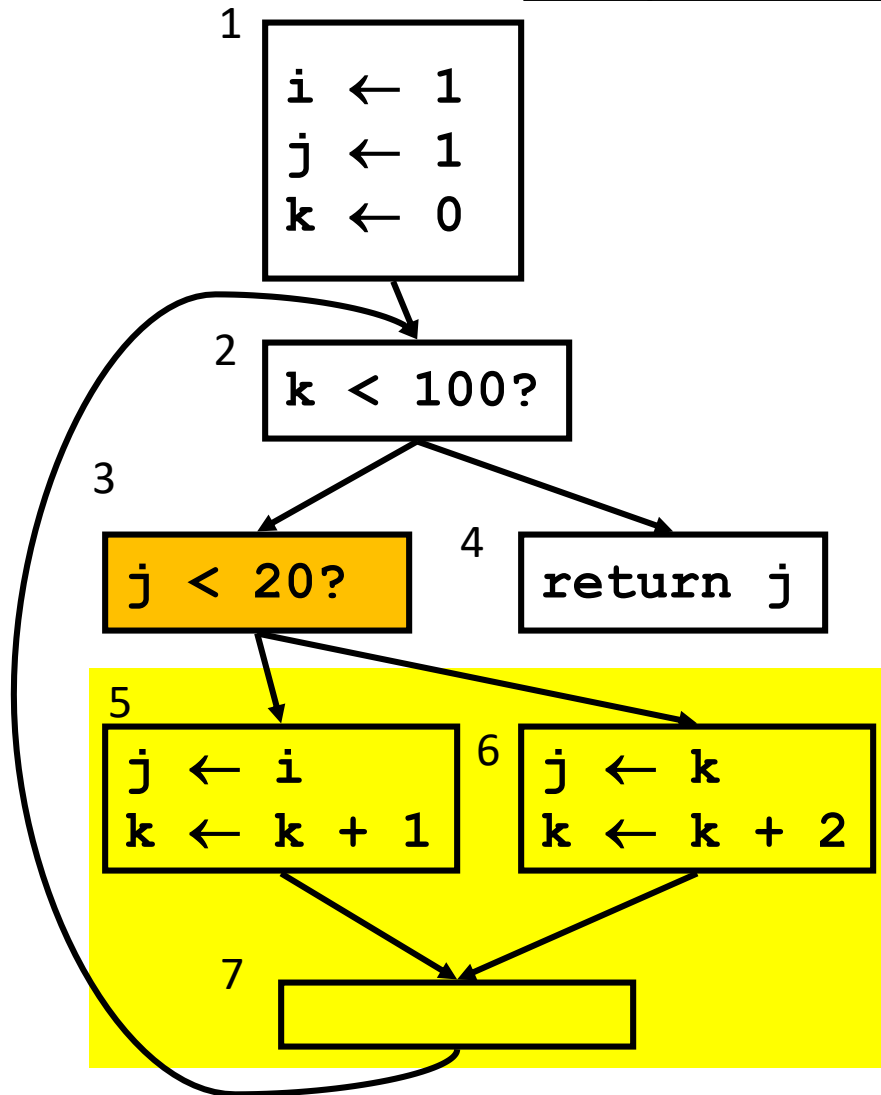
III. Example

Compute Dominance Tree



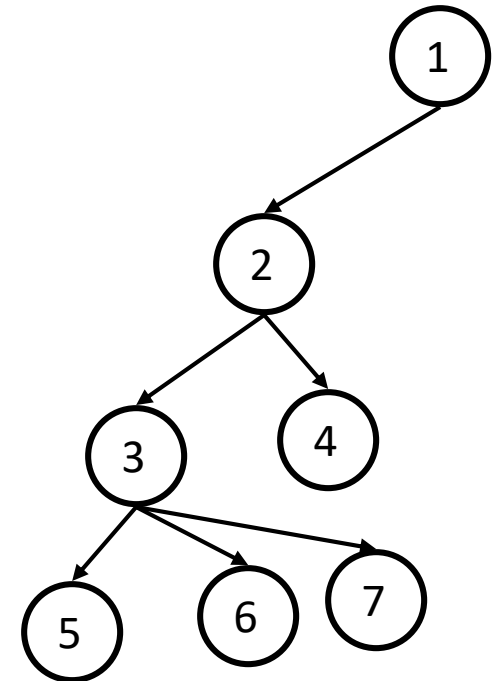
Dominance Tree

Compute Dominance Frontiers



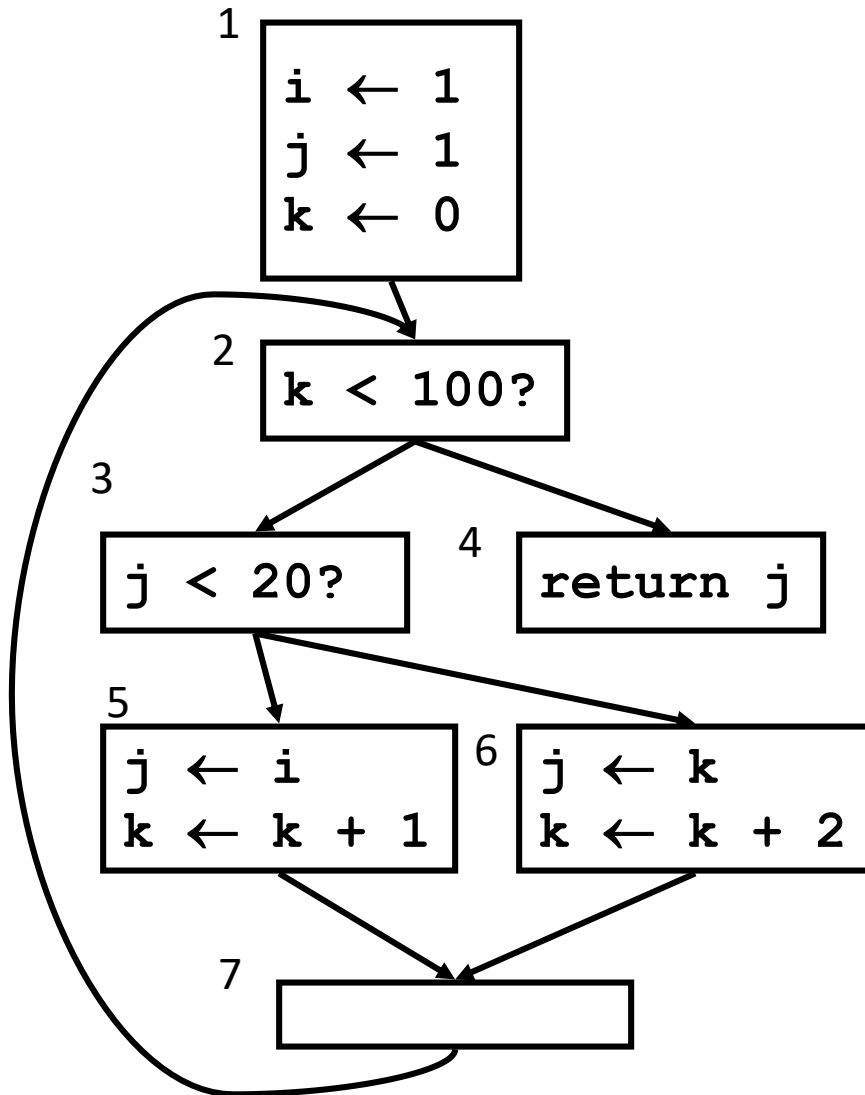
DFs

1	{}
2	{2}
3	?? {2}
4	{}
5	{7}
6	{7}
7	{2}



$$DF(x) = \{ w \mid x \text{ dom pred}(w) \text{ AND } !(x \text{ sdom } w) \}$$

Insert $\Phi()$



DFs

1	{}
2	{2}
3	{2}
4	{}
5	{7}
6	{7}
7	{2}

$\text{orig}[n]$ = variables defined in n

1	{i,j,k}
2	{}
3	{}
4	{}
5	{j,k}
6	{j,k}
7	{}

$\text{defsites}[v]$

i	{1}
j	{1,5,6}
k	{1,5,6}

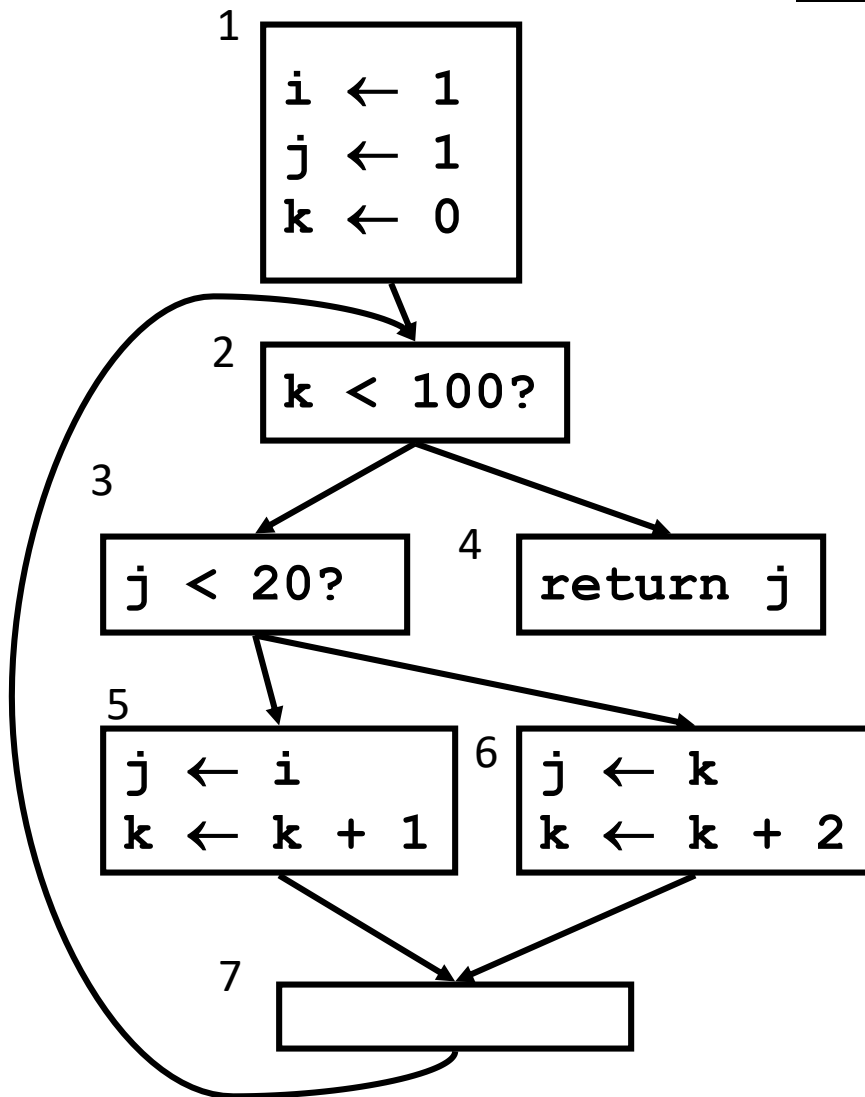
```

foreach variable v {
  W = defsites[v]
  while W not empty {
    n = remove node from W
    ...
  }
}

```

var i: $W=\{1\}$

Insert $\Phi()$



DFs

1	{}
2	{2}
3	{2}
4	{}
5	{7}
6	{7}
7	{2}

orig[n]

1	{i,j,k}
2	{}
3	{}
4	{}
5	{j,k}
6	{j,k}
7	{}

PHI[v]

j	{}
k	{}
defsites[v]	
i	{1}
j	{1,5,6}
k	{1,5,6}

```

foreach y in DF[n]
  if y ∉ PHI[v] {
    insert "v ← Φ(v,v,...)" in y
    PHI[v] = PHI[v] ∪ {y}
    if v ∉ orig[y]: W = W ∪ {y}
  }
  
```

var i: W={1}

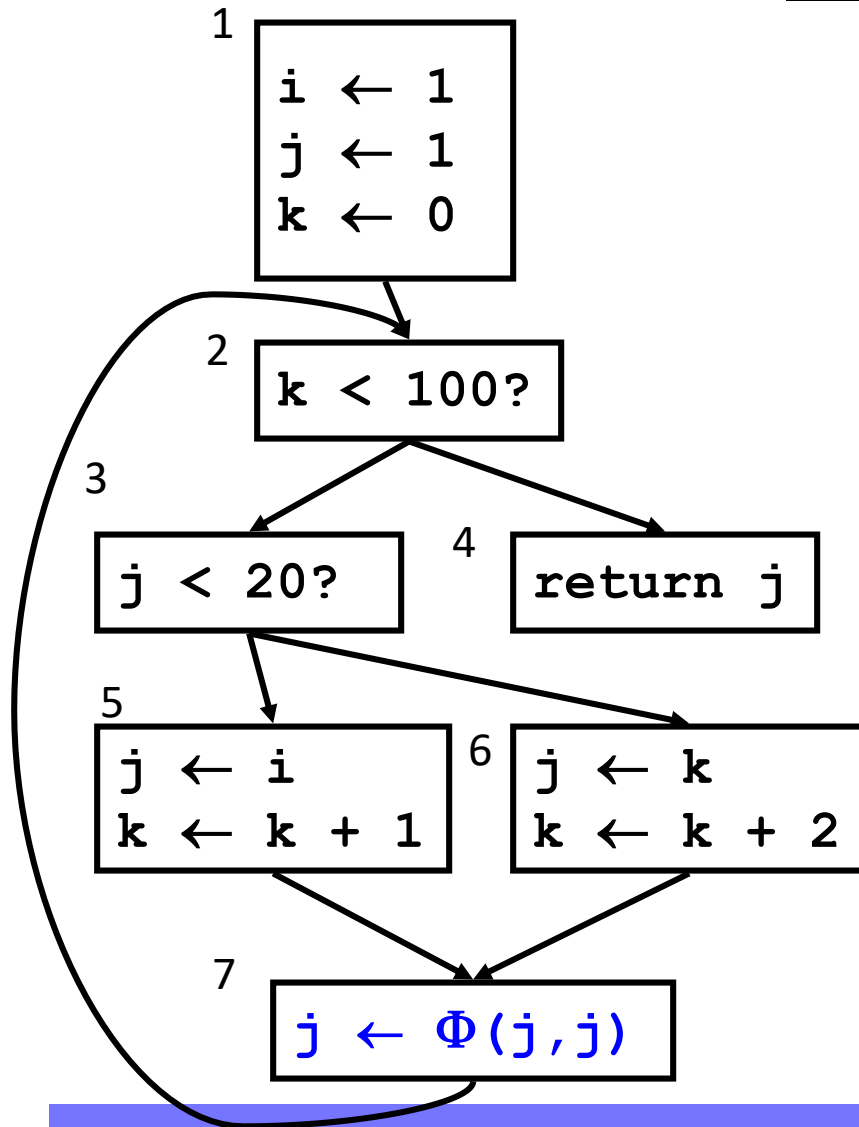
~~DF{1}~~

var j: W={1,5,6}

~~DF{1}~~

DF{5}

Insert $\Phi()$



DFs

1	{}
2	{2}
3	{2}
4	{}
5	{7}
6	{7}
7	{2}

orig[n]

1	{i,j,k}
2	{}
3	{}
4	{}
5	{j,k}
6	{j,k}
7	{}

PHI[v]

j	{7}
k	{}

defsites[v]

i	{1}
j	{1,5,6}
k	{1,5,6}

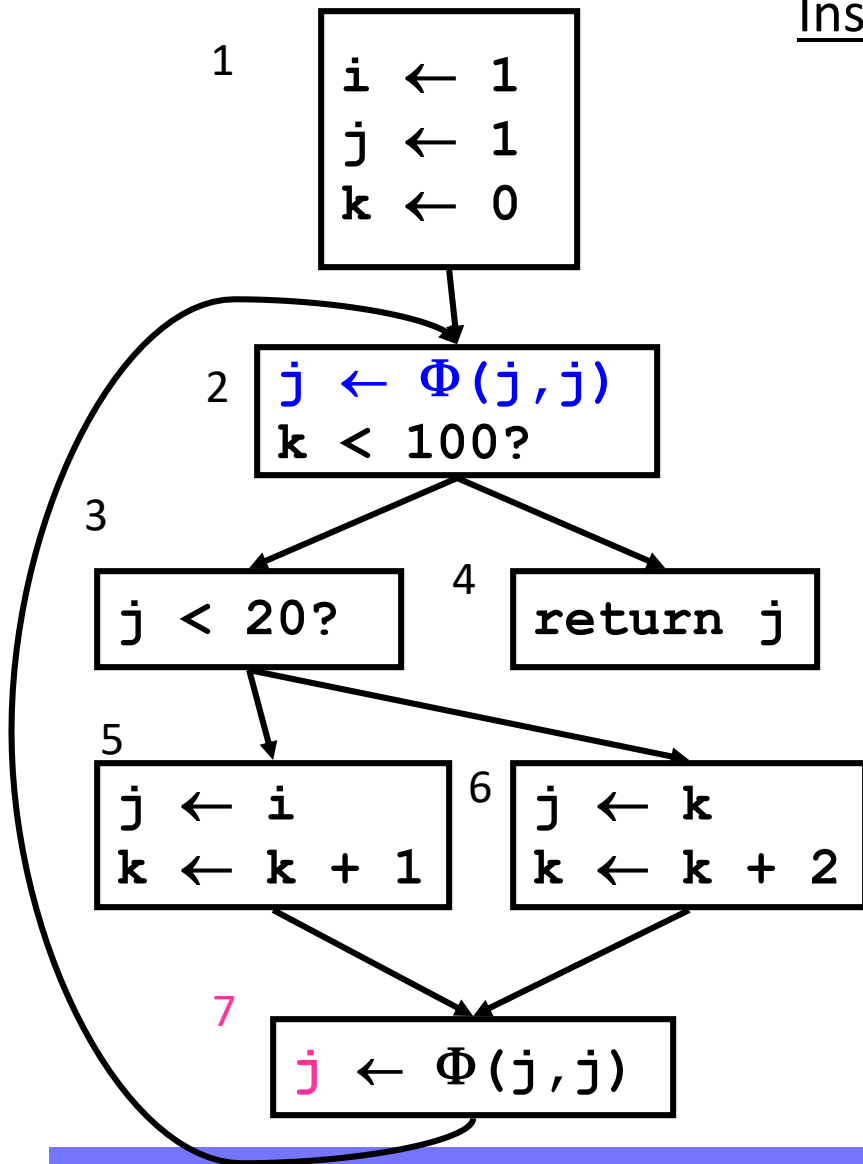
```

foreach y in DF[n]
  if y ∉ PHI[v] {
    insert "v ← Φ(v,v,...)" in y
    PHI[v] = PHI[v] ∪ {y}
    if v ∉ orig[y]: W = W ∪ {y}
  }
  
```

var j: W={5,6}

~~DF{1}~~

DF{5}



Insert $\Phi()$

DFs

1	{}
2	{2}
3	{2}
4	{}
5	{7}
6	{7}
7	{2}

orig[n]

1	{i,j,k}
2	{}
3	{}
4	{}
5	{j,k}
6	{j,k}
7	{}

PHI[v]

j	{2,7}
k	{}
defsites[v]	
i	{1}
j	{1,5,6}
k	{1,5,6}

```

foreach y in DF[n]
  if y ∉ PHI[v] {
    insert "v ← Φ(v,v,...)" in y
    PHI[v] = PHI[v] ∪ {y}
    if v ∉ orig[y]: W = W ∪ {y}
  }

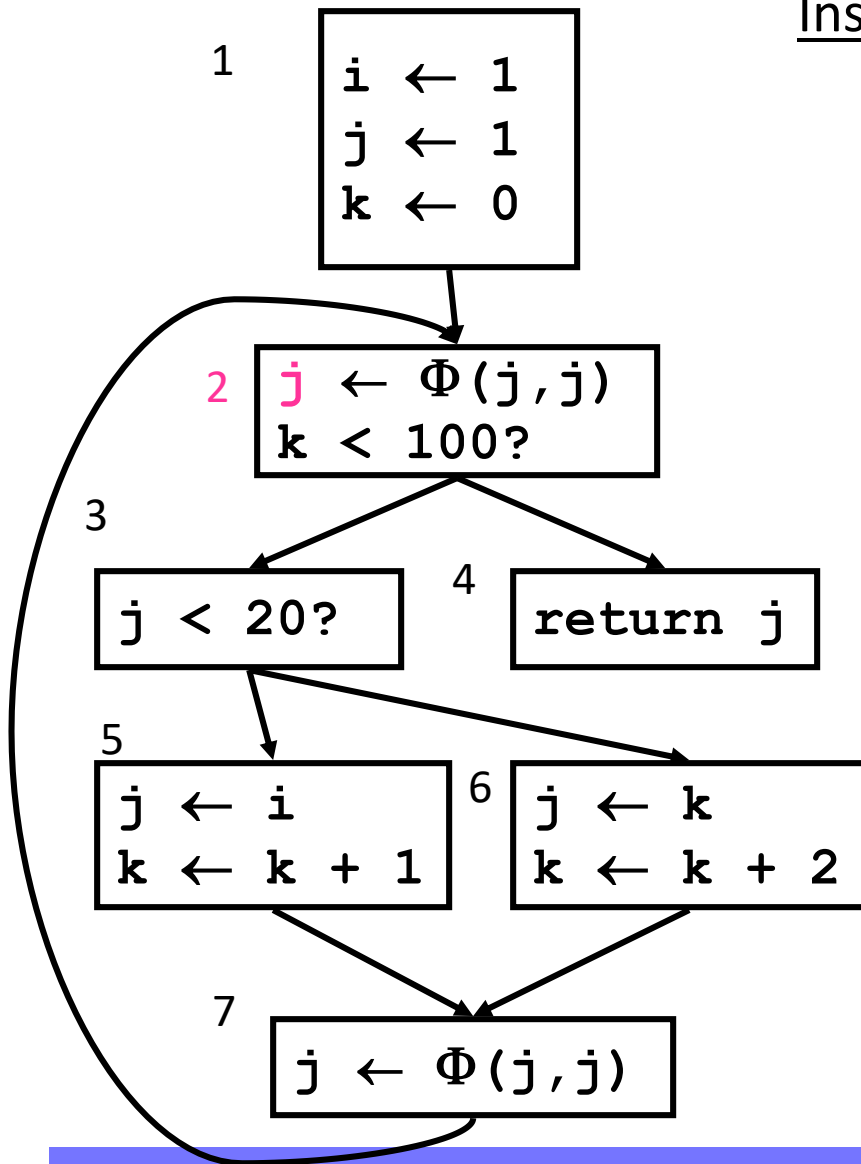
```

var j: W={7,6}

~~DF{1}~~

~~DF{2}~~

DF{7}



Insert $\Phi()$

DFs

1	{}
2	{2}
3	{2}
4	{}
5	{7}
6	{7}
7	{2}

orig[n]

1	{i,j,k}
2	{}
3	{}
4	{}
5	{j,k}
6	{j,k}
7	{}

PHI[v]

j	{2,7}
k	{}

defsites[v]

i	{1}
j	{1,5,6}
k	{1,5,6}

```

foreach y in DF[n]
  if y ∉ PHI[v] {
    insert "v ← Φ(v,v,...)" in y
    PHI[v] = PHI[v] ∪ {y}
    if v ∉ orig[y]: W = W ∪ {y}
  }
  
```

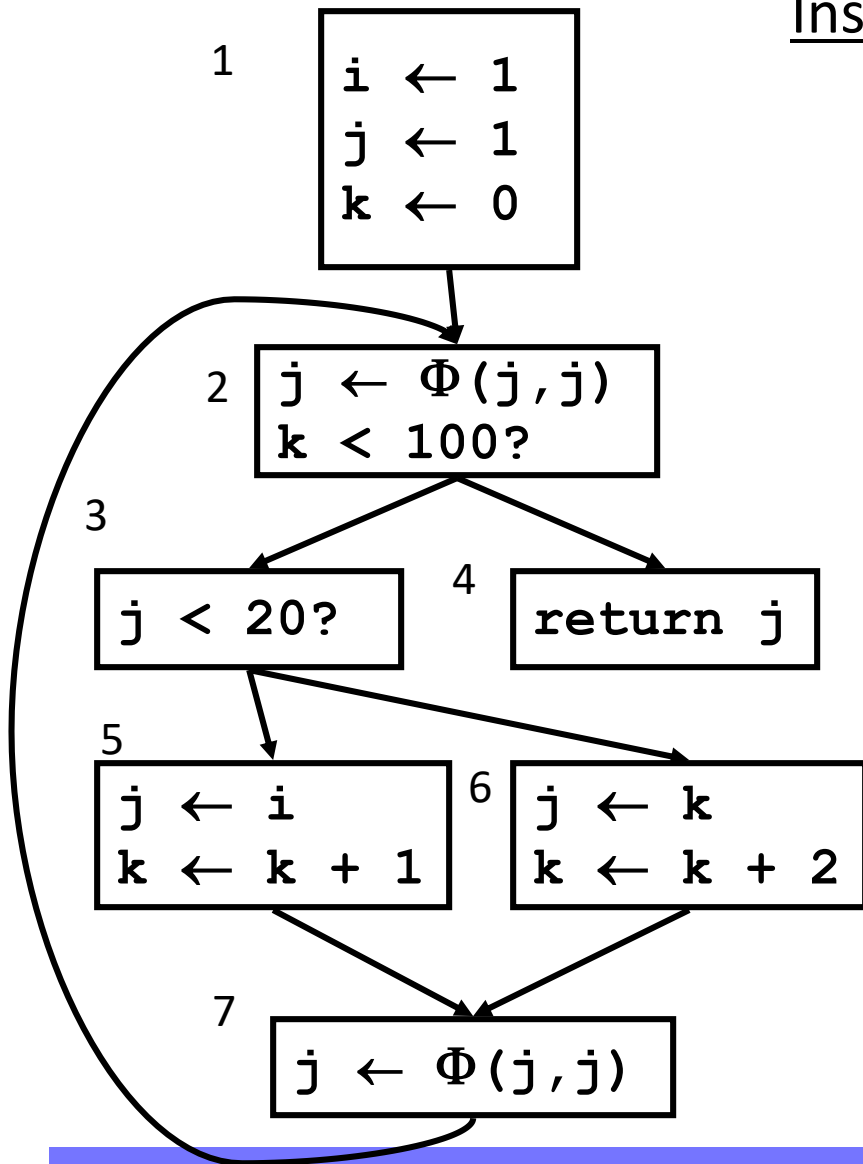
var j: W={2,6}

~~DF{1}~~

~~DF{3}~~

~~DF{7}~~

DF{2}



Insert $\Phi()$

DFs

1	{}
2	{2}
3	{2}
4	{}
5	{7}
6	{7}
7	{2}

orig[n]

1	{i,j,k}
2	{}
3	{}
4	{}
5	{j,k}
6	{j,k}
7	{}

PHI[v]

j	{2,7}
k	{}

defsites[v]

i	{1}
j	{1,5,6}
k	{1,5,6}

```

foreach y in DF[n]
  if y ∉ PHI[v] {
    insert "v ← Φ(v,v,...)" in y
    PHI[v] = PHI[v] ∪ {y}
    if v ∉ orig[y]: W = W ∪ {y}
  }
  
```

var j: W={6}

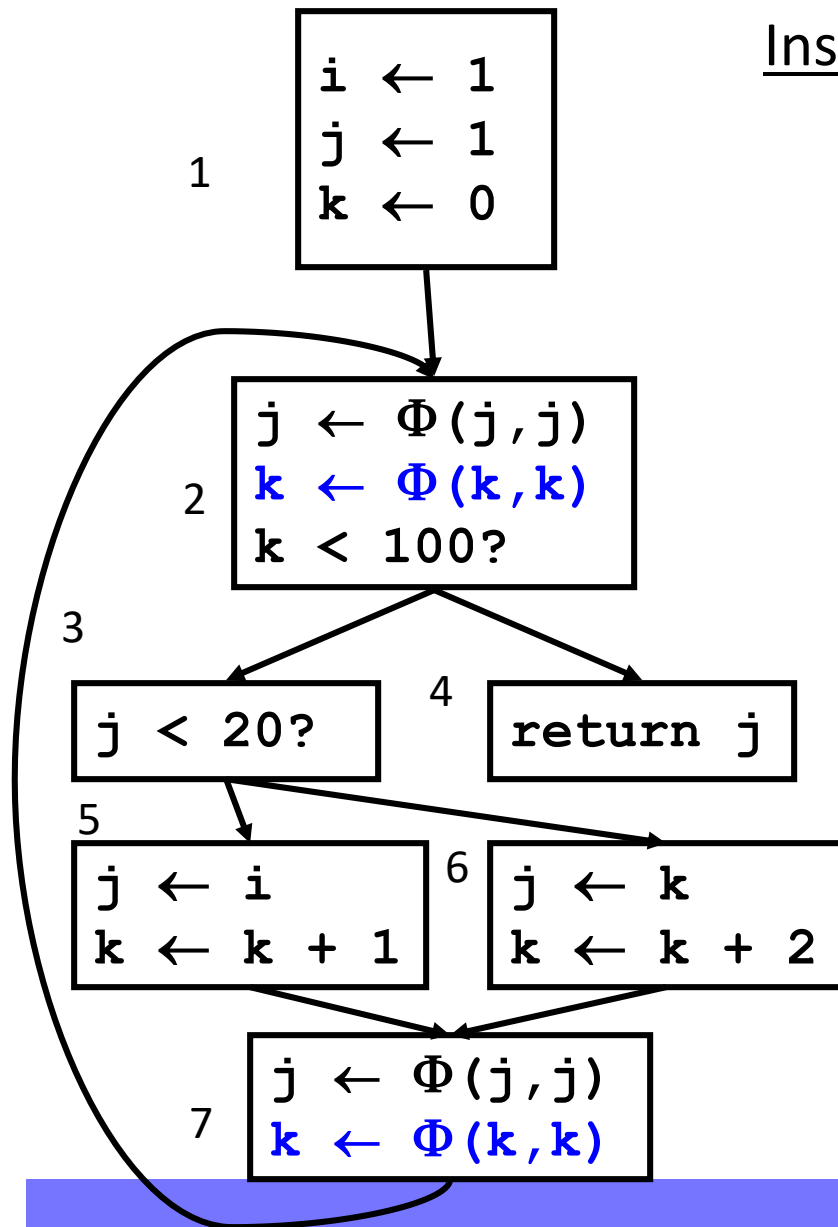
~~DF[1]~~

~~DF[3]~~

~~DF[2]~~

~~DF[6]~~

~~DF[4]~~



Insert $\Phi()$

DFs

1	{}
2	{2}
3	{2}
4	{}
5	{7}
6	{7}
7	{2}

orig[n]

1	{i,j,k}
2	{}
3	{}
4	{}
5	{j,k}
6	{j,k}
7	{}

PHI[v]

j	{2,7}
k	{2,7}

defsites[v]

i	{1}
j	{1,5,6}
k	{1,5,6}

```

foreach y in DF[n]
  if y ∉ PHI[v] {
    insert "v ← Φ(v,v,...)" in y
    PHI[v] = PHI[v] ∪ {y}
    if v ∉ orig[y]: W = W ∪ {y}
  }
  
```

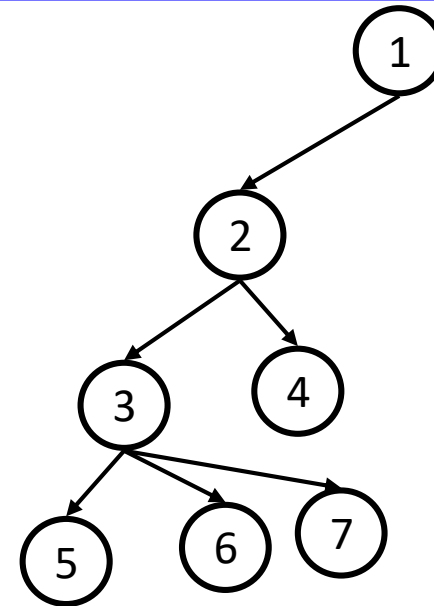
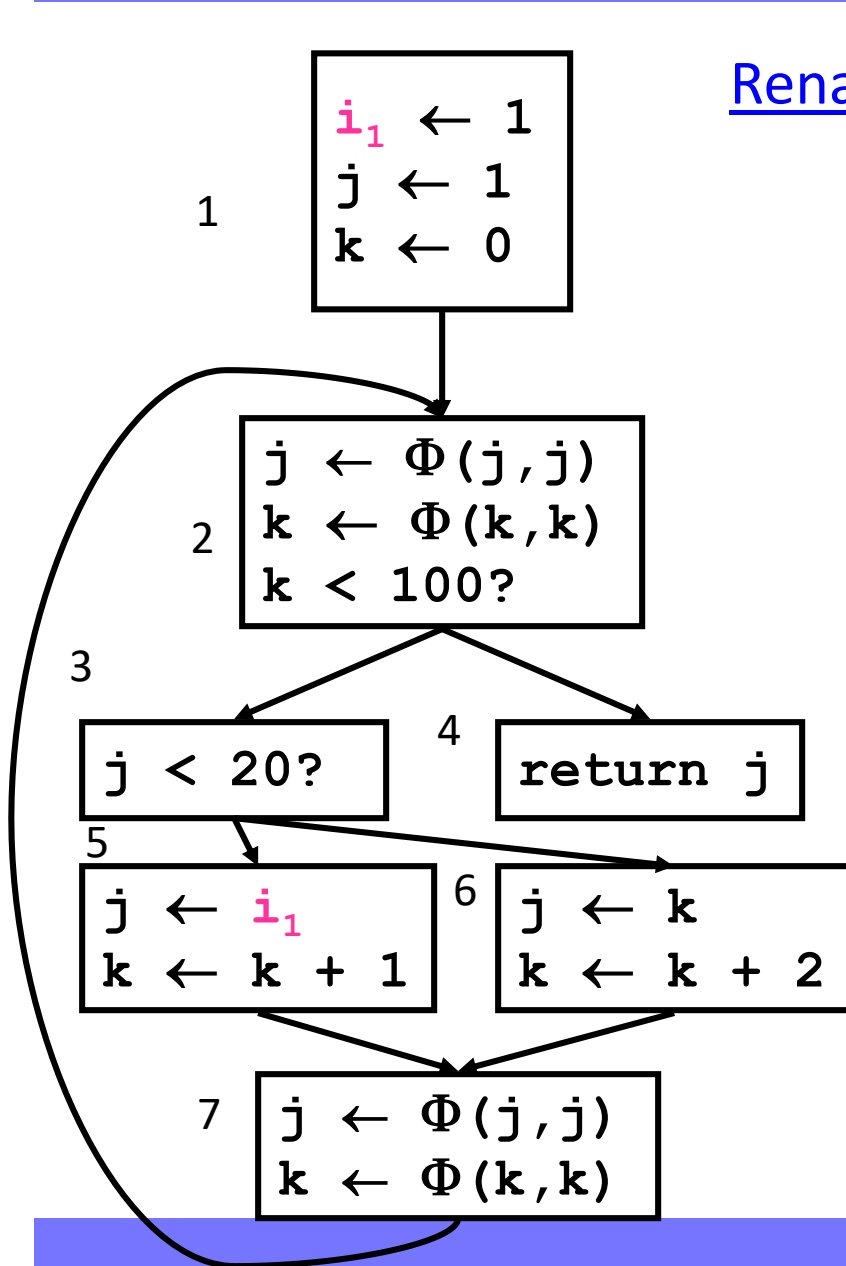
var k: W={1,5,6}

Same as var j: adds Φ to 7 and 2

Done inserting $\Phi()$ s...Time to rename vars

Carnegie Mellon

Rename Vars



i_1
stack(i)

rename(B):

for each statement in B:

replace (non- Φ) use of v with top of stack(v)

replace def of v with v_{new} ,

push v_{new} onto stack(v)

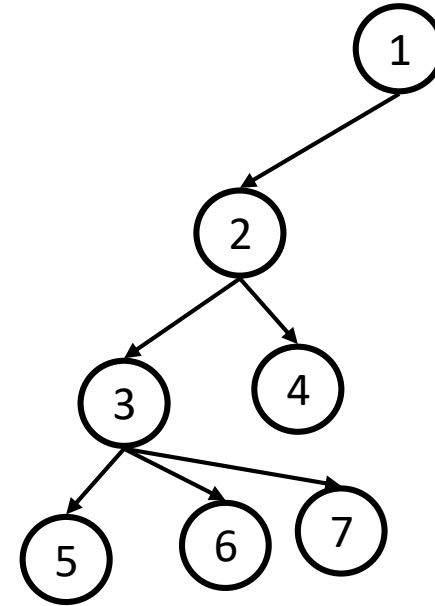
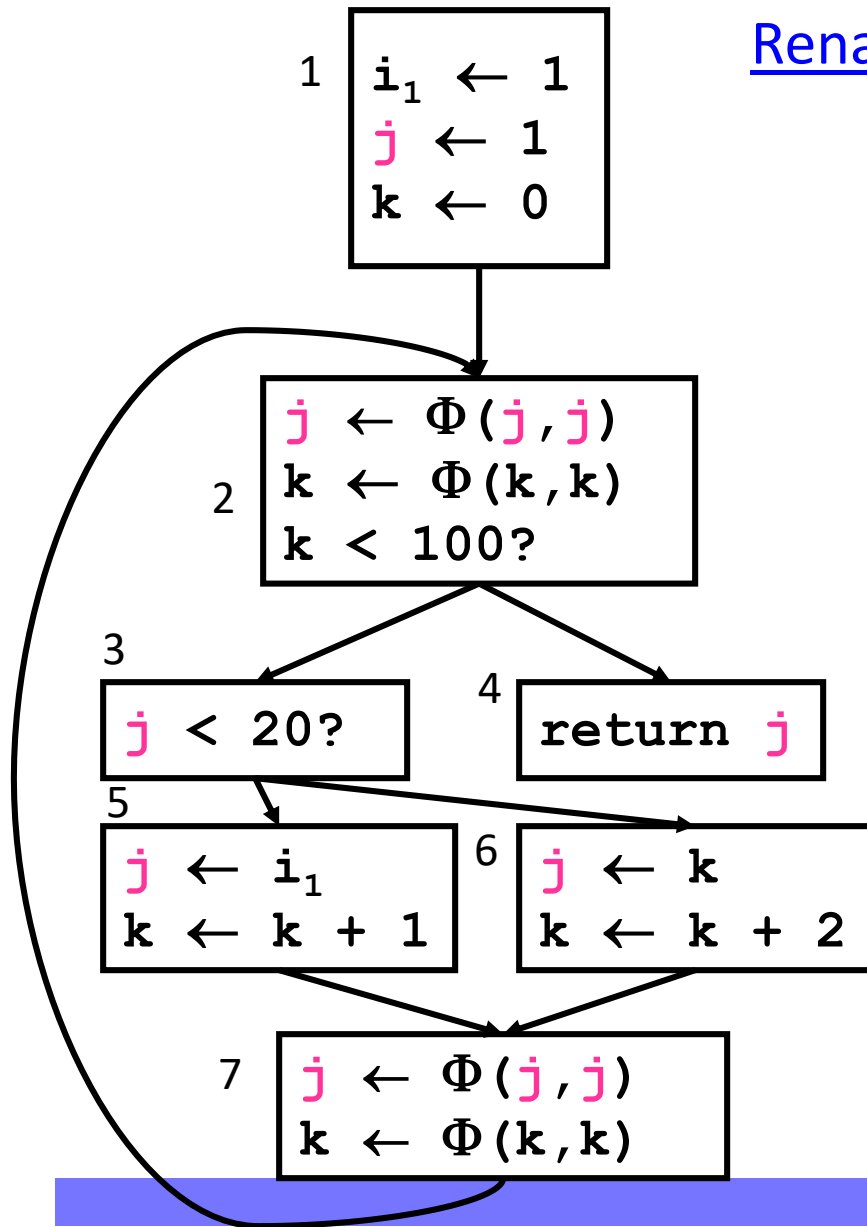
for each successor S of B in CFG:

replace k 'th arg. of $\Phi(\mathbf{v}, \dots \mathbf{v})$ with top of stack(v), where B is k 'th predecessor of S
call rename(C) on all children C of B

in Dominance Tree

pop all defs in B from stacks

Rename Vars



rename(B):

for each statement in B:

replace (non- Φ) use of v with top of stack(v)

replace def of v with v_{new} ,

push v_{new} onto stack(v)

for each successor S of B in CFG:

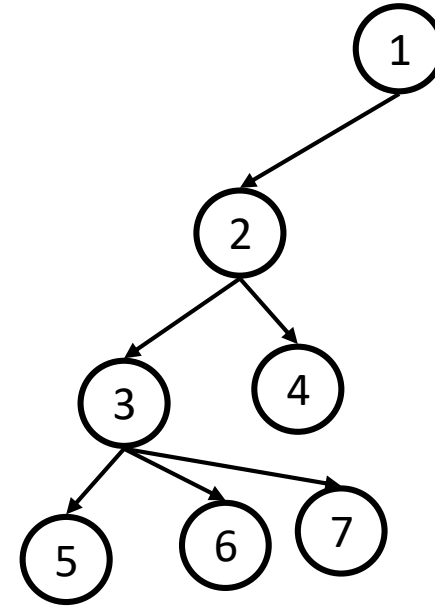
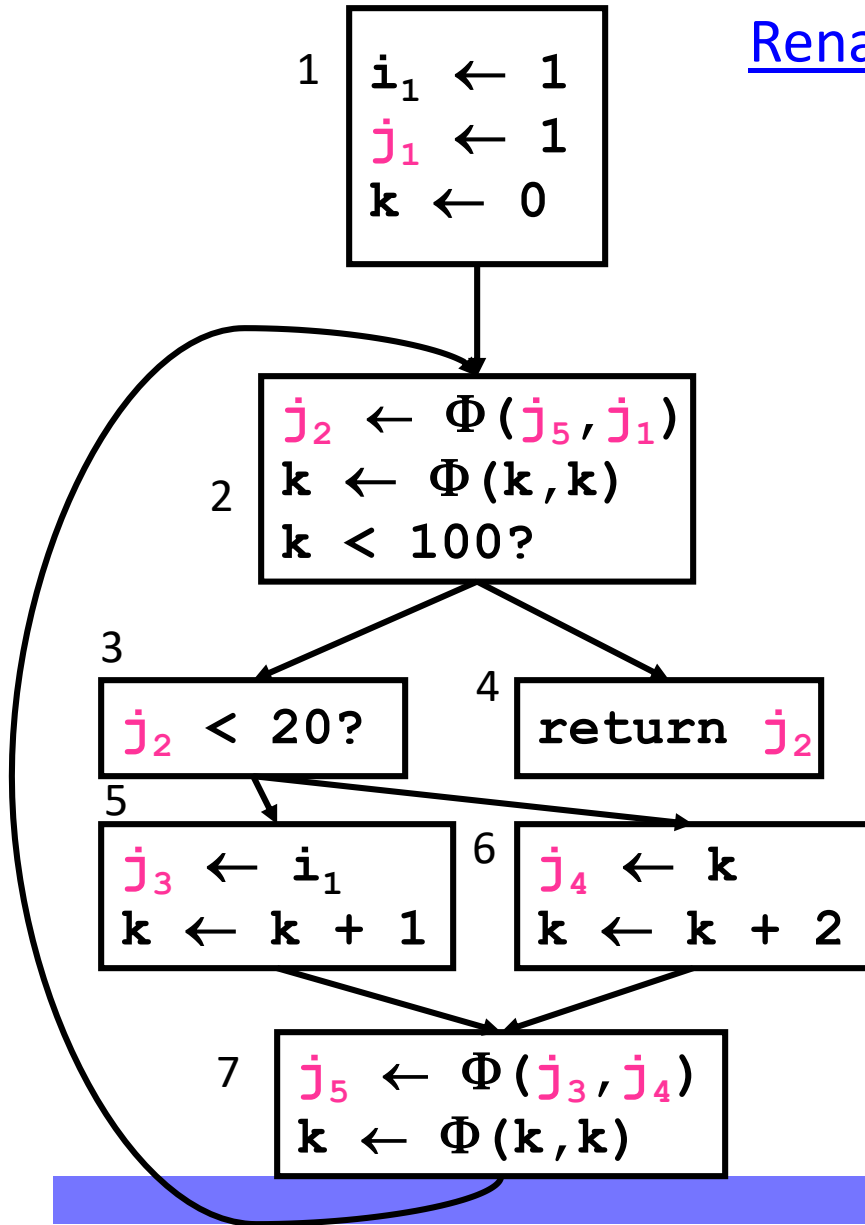
replace k 'th arg. of $\Phi(\mathbf{v}, \dots \mathbf{v})$ with top of stack(v), where B is k 'th predecessor of S

call rename(C) on all children C of B

in Dominance Tree

pop all defs in B from stacks

Rename Vars



rename(B):

for each statement in B:

replace (non- Φ) use of v with $\text{top of stack}(v)$

replace def of v with v_{new} ,

push v_{new} onto $\text{stack}(v)$

for each successor S of B in CFG:

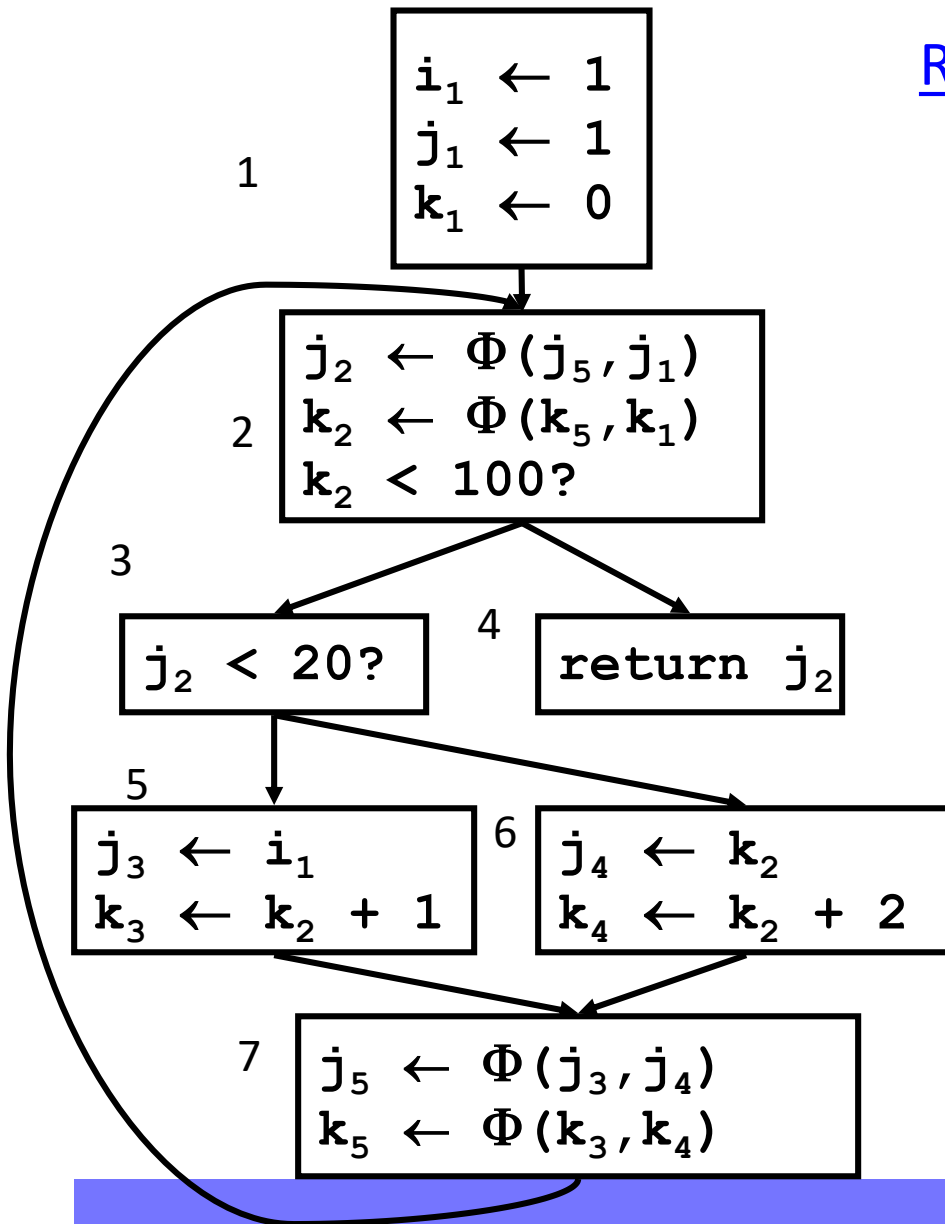
replace k 'th arg. of $\Phi(v, \dots, v)$ with $\text{top of stack}(v)$, where B is k 'th predecessor of S

call $\text{rename}(C)$ on all children C of B

in Dominance Tree

pop all defs in B from stacks

Rename Vars: Final Result

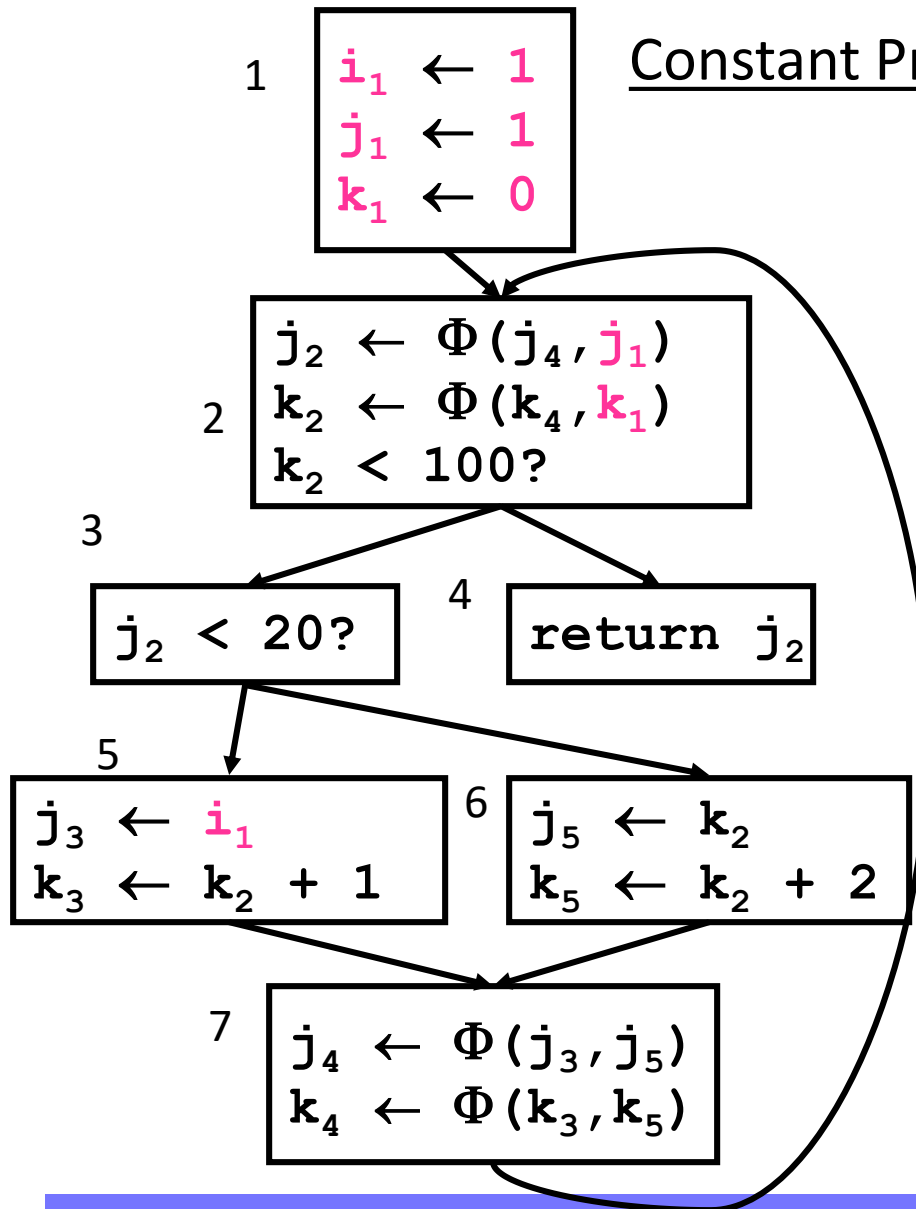


IV. Constant Propagation with SSA

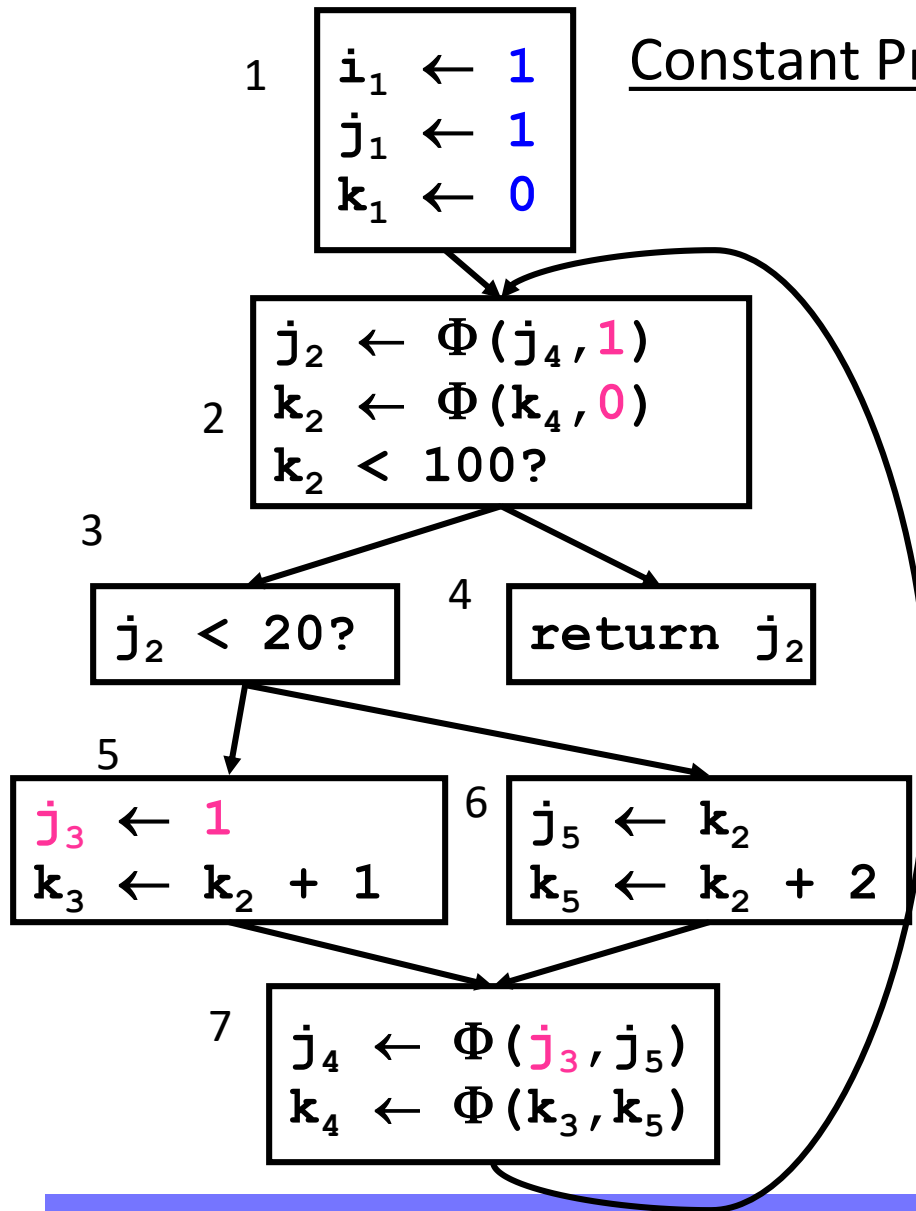
- If " $v \leftarrow c$ ", replace all uses of v with c
- If " $v \leftarrow \Phi(c, c, c)$ " (each input is the same constant), replace all uses of v with c

```
W ← list of all defs
while !W.isEmpty {
    Statement S ← W.removeOne
    if ((S has form " $v \leftarrow c$ ") ||
        (S has form " $v \leftarrow \Phi(c, \dots, c)$ ")) then {
        delete S
        foreach statement U that uses v {
            replace v with c in U
        }
        W.add(U)
    }
}
```

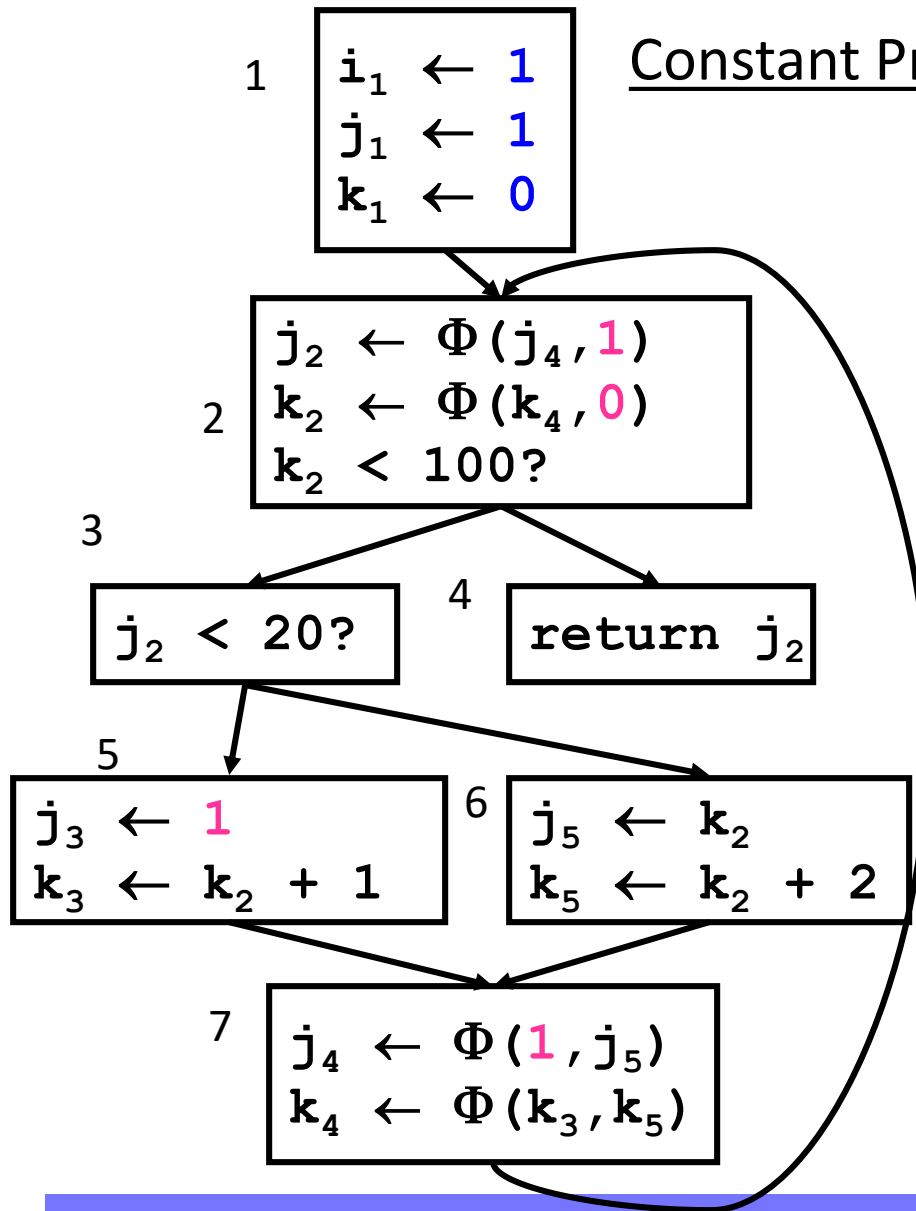
Constant Propagation



Constant Propagation

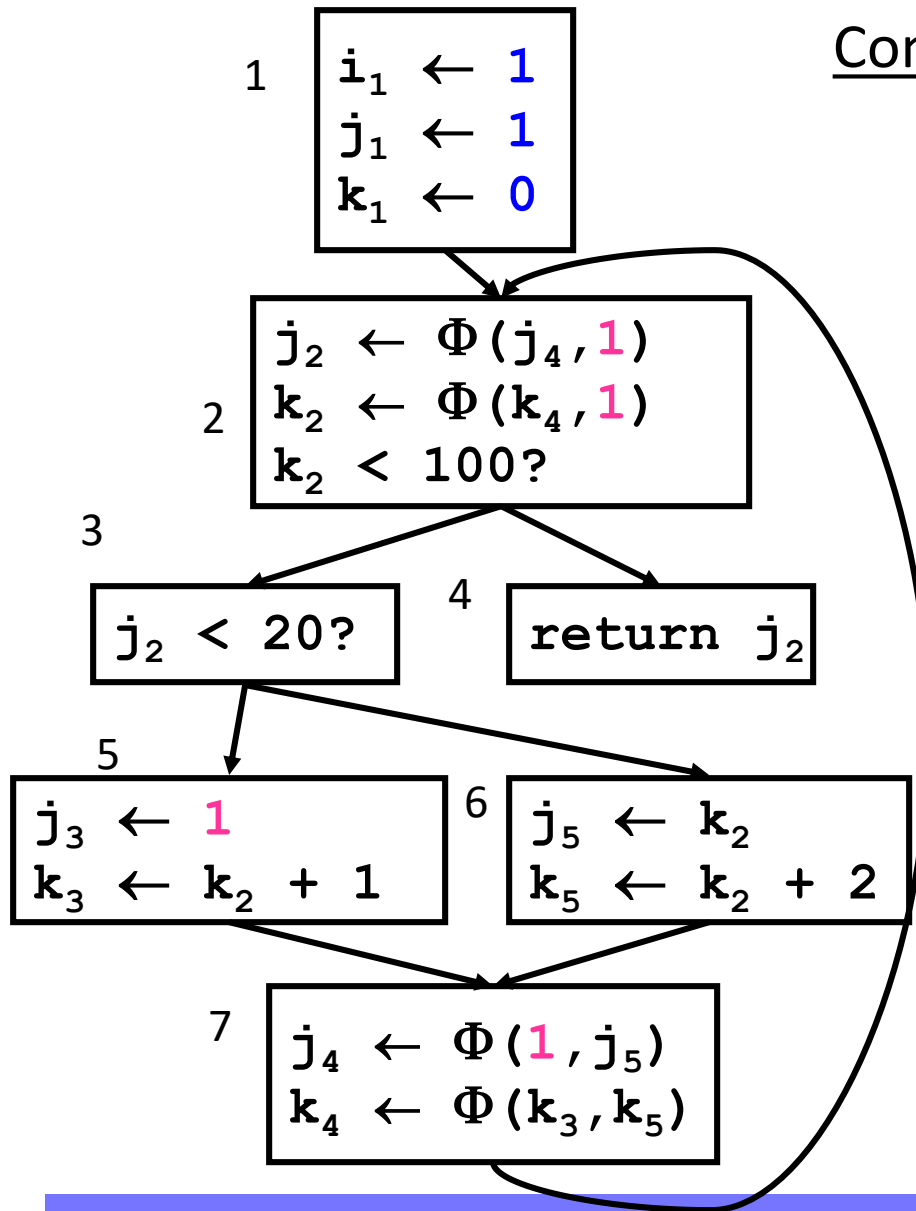


Constant Propagation



Not a very exciting result (yet)...

Conditional Constant Propagation



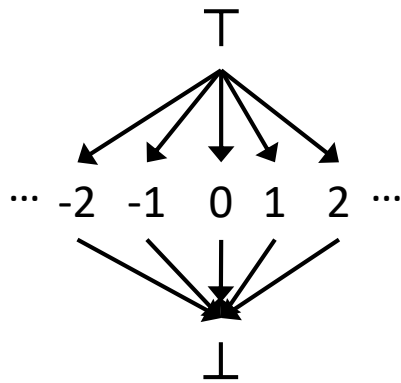
- Does block 6 ever execute?
- Simple Constant Propagation can't tell
- But “Conditional Const. Prop.” *can* tell:
 - Assumes **blocks don't execute** until proven otherwise
 - Assumes **values are constants** until proven otherwise

Conditional Constant Propagation Algorithm

Keeps track of:

- **Blocks**
 - assume unexecuted until proven otherwise
- **Variables**
 - assume not executed (only with proof of assignments of a non-constant value do we assume not constant)

Lattice for representing variables:

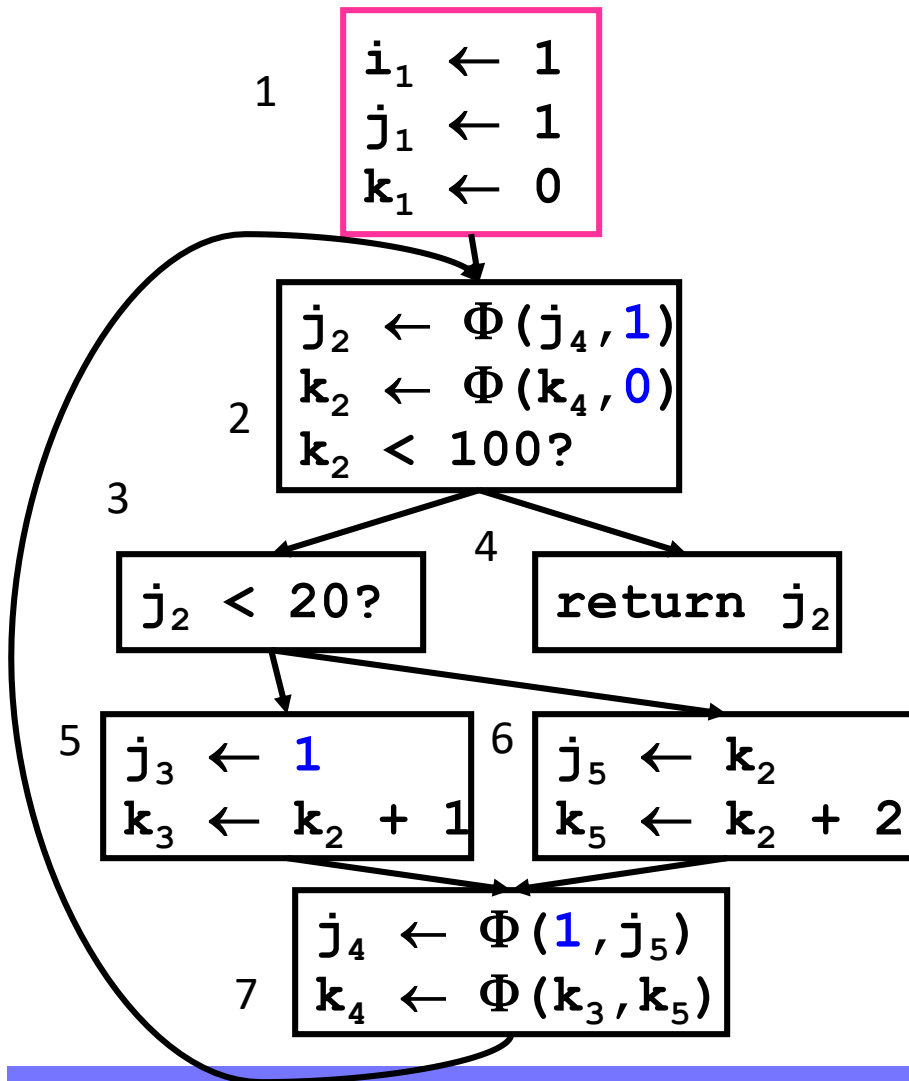


not executed

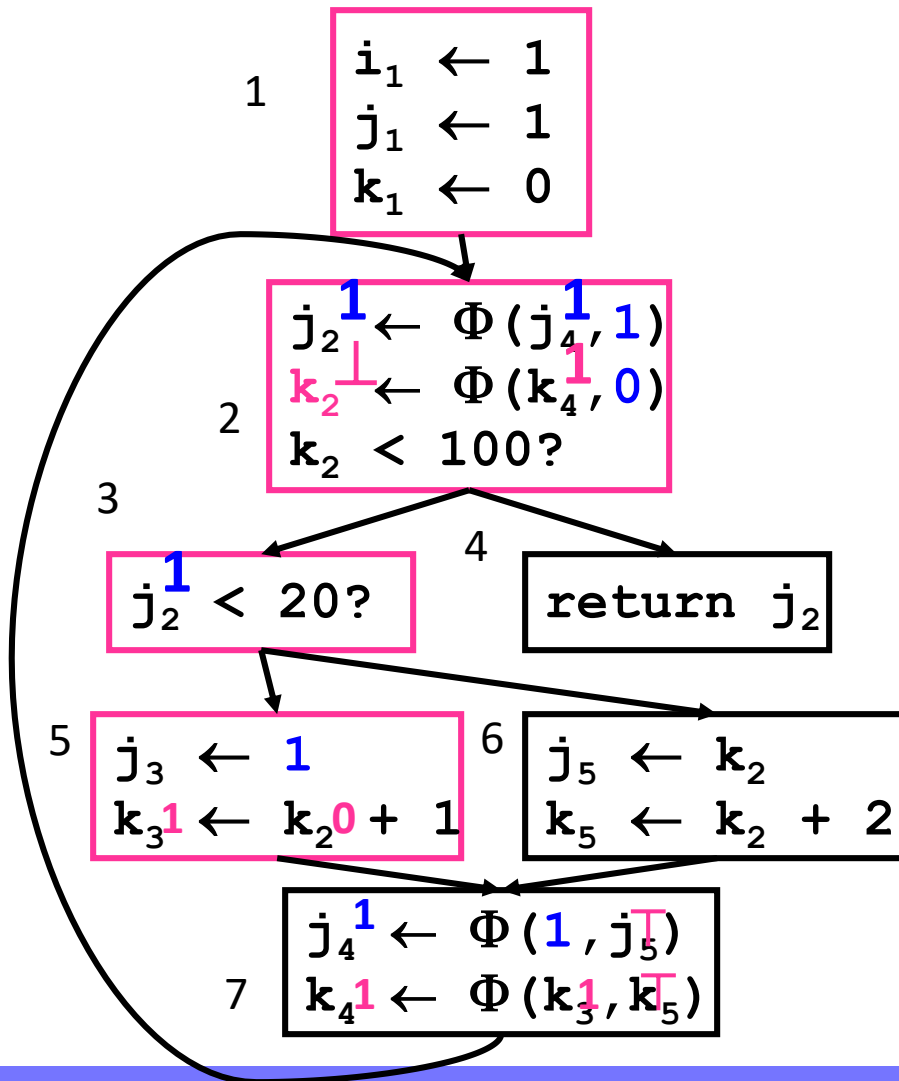
we have seen **evidence** that the variable has been **assigned a constant** with the value

we have seen **evidence** that the variable **can hold different values** at different times

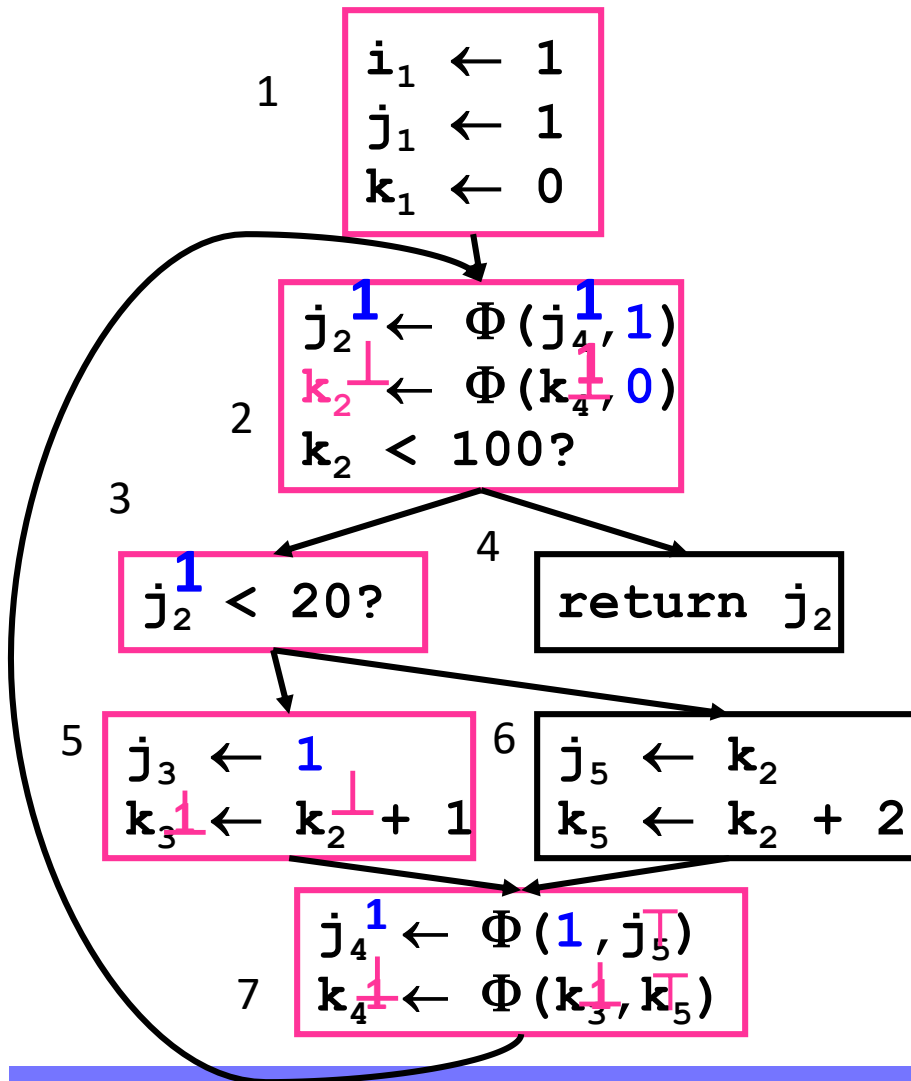
Conditional Constant Propagation



Conditional Constant Propagation



Conditional Constant Propagation



Conditional Constant Propagation

